

Streaming Data Pipelines with Kafka

Stefan Sprenger



MEAP



MANNING



**MEAP Edition
Manning Early Access Program**

**Streaming Data Pipelines with Kafka
Version 6**

Copyright 2024 Manning Publications

For more information on this and other Manning titles go to manning.com.

© Manning Publications Co. To comment go to liveBook

Licensed to Ruslan Hrushchak <hrushchak@appplant.de>

Welcome

Thank you for purchasing *Streaming Data Pipelines with Kafka* in MEAP.

We are used to getting groceries delivered in under ten minutes and we can access almost any information at any time via search engines, yet most data integrations are implemented with batch pipelines that leave data systems out of sync at most times. Streaming data pipelines are a modern alternative to batch pipelines: They replicate data between data systems like database systems and APIs in real-time and can process them on the way. As a foundational technology of modern, real-time data architectures, streaming data pipelines enable downstream data consumers to always work with fresh data. Unfortunately, you still need to become an expert in many technologies to successfully apply them.

I am writing this book for data engineers, software developers, and others who face use cases for data-integration at their workplace. It aims to provide a complete introduction to the concepts, development, and deployment of streaming data pipelines, with a strong focus on the open-source technology Apache Kafka, the de-facto industry standard for event streaming. The book not only teaches foundational concepts of streaming data pipelines but prepares the reader to successfully implement them at their workplace. Throughout the book, we will help an imaginary e-commerce company to migrate its legacy batch pipelines to a modern streaming architecture and overcome the typical challenges of such migrations.

My goal is to develop the best book possible. Your feedback on early versions of the book is essential for its development and is highly appreciated. Please don't hesitate to post your questions, comments, or suggestions in the [liveBook discussion forum](#).

-Stefan Sprenger

brief contents

PART 1: FUNDAMENTALS

1 Getting into data streaming

2 A walk through Kafka and its ecosystem

PART 2: INTEGRATING STREAMING DATA PIPELINES

3 Integrating data systems in real-time with Kafka Connect

4 Building custom connectors with Kafka Connect

PART 3: TRANSFORMING DATA IN REAL-TIME

5 Stateless transformations with Kafka Streams

6 Stateful transformations with Kafka Streams

PART 4: DEPLOYING STREAMING DATA PIPELINES

7 Packaging and deployment

8 Monitoring

9 Performance

10 Handling errors in production

1

Getting into data streaming

This chapter covers

- An introduction to streaming data pipelines and their building blocks
- The shortcomings of traditional batch data pipelines
- The relationship between data streaming and batch processing
- Use cases for stream processing

In the last decades, software has eaten the world. Across all industries, companies have adopted software solutions at their core to unlock new business models, improve their efficiency, and provide more value to customers. Businesses use CRM tools to manage customer relationships, drive their decision-making with reporting and dashboards, and predict the impact of price changes with AI models. Similar to cars requiring fuel to drive, these software systems run on data. Modern businesses employ a plethora of different software solutions that they cannot operate as isolated silos but need to integrate with each other to derive the maximum value. Integrating software systems has never been an easy task but is becoming even more complex these days.

We are used to getting groceries delivered in ten minutes and can access almost any information at any time via search engines - Why are most data integrations neither working in real-time nor syncing data changes instantly after their occurrence? At the same time, businesses operate an ever-growing number of different systems and easily spend six to seven digits on cloud computing per year, making them favor efficient data integrations that sync only relevant data changes instead of performing resource-wasting bulk loads.

Fortunately, there is a modern, event-driven approach to cope with the demanding requirements of data-driven businesses: Streaming data pipelines.

1.1 Introducing streaming data pipelines

Data pipelines integrate different data systems. They replicate data from data sources, like database systems, to data sinks, like data warehouses, and can process the data on the way.

Most datasets are unbounded. Whether it is a transactional database system empowering an online shop, a cloud object store holding sales data, or an API exposing user data; it is very likely that these datasets are not static but experience changes frequently. To this end, most data pipelines are not one-off deployments but must be recurringly or continuously executed.

1.1.1 Going one step back: Batch data pipelines

Traditionally, data pipelines are implemented with batch processing: They recurringly extract all data from data sources, processing them in one go, and publish the processed data to data sinks. Figure 1.1 shows a batch data pipeline that integrates MySQL, a transactional database system, with ClickHouse, a data warehouse. Each night at 2 AM, the batch pipeline (1) extracts all data from MySQL, (2) cleans up the data, and (3) ingests the processed data into ClickHouse. While batch pipelines are fairly straightforward to implement, they put a tremendous load on all components of the architecture, which precludes a frequent execution. As a consequence, data consumers, like dashboards, reporting tools, or analytical applications, do not have access to up-to-date data.

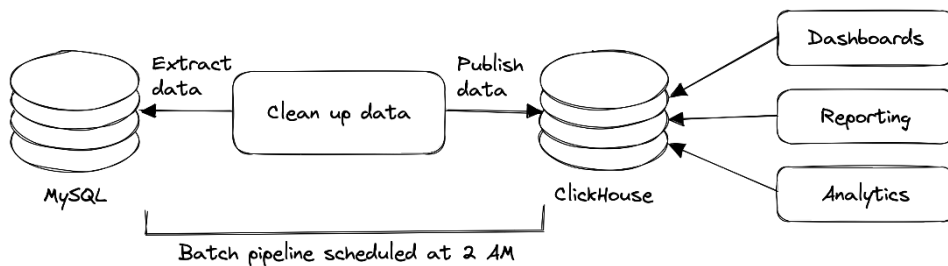


Figure 1.1 Each night at 2 AM, a batch data pipeline replicates data from MySQL to ClickHouse and processes them on the way.

NOTE While most database systems provide the same interface, the Structured Query Language (SQL), they typically do not optimize for all kinds of workloads but focus on either transactional, write-heavy operations or analytical, read-heavy use cases. For instance, MySQL is a solid choice for managing the continuously-changing state of an application, like an online shop, but does not cater well to the analytical queries executed by dashboards, reporting tools, and data-intensive applications.

The simple and straightforward implementation of batch data pipelines comes at the cost of several drawbacks in the timely, efficient, and robust processing of data:

1. Batch data pipelines cannot be executed with a high frequency. Business teams cannot support decisions with current information but are almost always working with outdated data, which might lead to missed business opportunities.
2. Batch pipelines are quite heavy and put a lot of load onto data source systems because they extract all data at each execution. To avoid impacting the performance of data sources, batch pipelines are typically executed outside of business hours. Debugging production issues is a challenge and rollouts of bug fixes must wait until the next execution.
3. Single runs of batch pipelines can easily take multiple hours because they are processing very large datasets. If batch pipelines fail while executing, they might leave data sinks in a corrupted state until the next execution of the pipeline. It might also happen that pipelines finish their execution after the start of the business day.
4. Batch pipelines see data sources at the time of data extraction and might miss incremental updates that occurred in between two runs.

1.1.2 Building blocks of streaming data pipelines

Similar to batch pipelines, streaming data pipelines replicate data from data sources to data sinks and can process them on the way. The big difference is that they are being executed continuously, work in real-time, and process incremental updates. Once a change has been applied to a data source, streaming data pipelines extract the change event (basically a row-level `INSERT`, `UPDATE`, or `DELETE` event), process it, and publish the processed change event to the data sink.

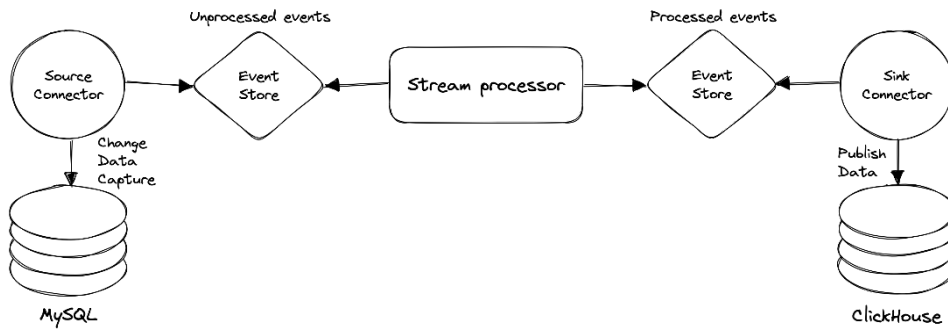


Figure 1.2 Employing a streaming data pipeline to replicate data from MySQL to ClickHouse in real-time and process them on the way.

Figure 1.2 illustrates how the example from the prior section could be implemented with a streaming data pipeline:

- Connectors** integrate the streaming data pipeline with external data sources and external data sinks, here MySQL and ClickHouse. The data source connector employs change data capture (CDC) for extracting row-level data change events from the replication log of MySQL in real-time. The data sink connector publishes processed change events to ClickHouse. Connectors may apply lightweight data transformations to events after extracting them from a data source or before publishing them to a data sink, e.g., for converting between data formats. The open-source community provides a few projects that implement connectors for data streaming use cases; the most popular and complete one is Kafka Connect, which is part of the official Apache Kafka release. Kafka Connect provides hundreds of different connectors for integrating Apache Kafka with external data systems, like MySQL or ClickHouse. While using existing connectors helps you to save time and minimize potential errors, developing custom connectors can make sense if you are dealing with uncommon data systems that are not (yet) covered by Kafka Connect.

- **Stream processors** handle the processing of events while replicating them from data sources to data sinks. They often integrate with an event store, like Apache Kafka. Stateless stream processing operations, such as transformations or filters, process event by event and do not consider the order or time of the events. Stateful stream processing operations, such as aggregations or joins, perform computations over windows of events. They are more complex than stateless operations, require maintaining and persisting the state of the computations, and need a notion of time. They also need mechanisms for dealing with scenarios, like the late arrival of events. Traditionally, stream processors are implemented with projects, such as Kafka Streams or Apache Flink, and in programming languages, like Java, Scala, or Python. Recently, a lot of tools started introducing SQL layers on top of their programmatic APIs, which simplifies the development of stream processing logic at the cost of flexibility. If you are heavily building on top of Apache Kafka, Kafka Streams might be a reasonable choice.
- **Event stores**, like Apache Kafka, persist events in a reliable and scalable way. Streaming data pipelines employ event brokers to persist (1) events that have been extracted by source connectors from an external data source, like MySQL, and are passed to the stream processor and (2) events that have been processed by the stream processor and are consumed by the sink connector for publishing them to an external data sink, like ClickHouse. Using dedicated storage for events instead of passing them directly between the connectors and the stream processor is very useful for a number of reasons, like the mitigation of backpressure, which occurs when consumers cannot keep up with the pace of the producers, or the replaying of events. Event stores also enable streaming data pipelines to achieve a high tolerance towards failures and temporary outages of single components. Apache Kafka is the most popular storage solution in the industry. Even most of its competitors, like Azure Event Hubs or RedPanda, provide a Kafka-compatible API. Apache Kafka provides low-level interfaces for consuming and producing events; Kafka Connect and Kafka Streams build on top of these interfaces and may help you save time when implementing connectors or stream processing logic.

1.1.3 Integrating data in real-time with Apache Kafka

Let us have a closer look at how you can use Apache Kafka, Kafka Connect, and Kafka Streams to build a data pipeline that streams order data from an online shop to a data warehouse in real-time.

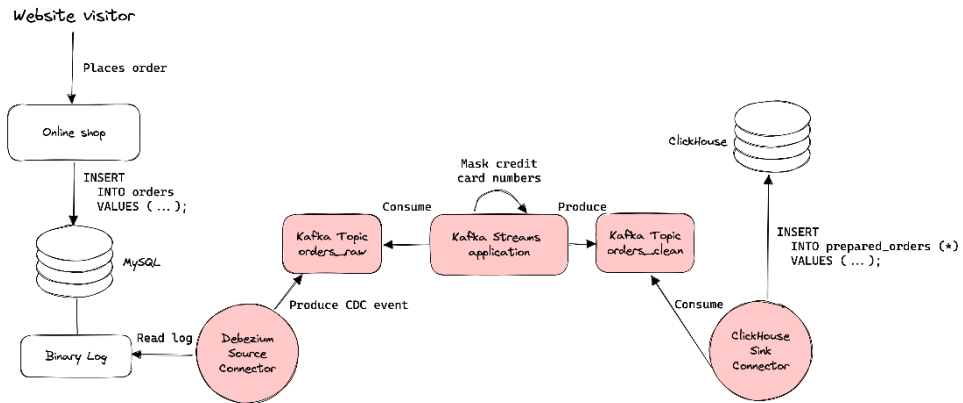


Figure 1.3 Visualization of a streaming data pipeline capturing CDC events from a MySQL database with Kafka Connect, processing them with Kafka Streams, and inserting them into ClickHouse with Kafka Connect; Kafka topics are used for persisting records.

When a website visitor places an order on the website of an e-commerce shop, the shop software executes an `INSERT` statement for adding a new row to the table `orders` of the underlying database system, here MySQL. Most database systems offer log files for replicating data between multiple database instances; one database instance can subscribe to the log file of another database instance and consume all data change events in real-time, allowing different database instances to stay in sync. The great thing about replication logs is that subscribing to them does not put any load onto the database system, because it bypasses the database system's query layer and lets subscribers directly access the log file. MySQL's replication log is called the Binary Log.

The open-source project Debezium, which provides Kafka Connect source connectors for various database systems, leverages this mechanism for detecting and capturing changes in real-time and without impacting the performance of MySQL. Once MySQL has flushed the change event to its Binary Log, Debezium extracts the event and produces it to an Apache Kafka topic, here called `orders_raw`. Internally, Debezium maintains the latest position of the Binary Log that it has successfully processed and persists that information in another Kafka topic, which allows resuming data extraction after restarts or downtimes without reprocessing data.

The Kafka Streams application subscribes to the Kafka topic `orders_raw`, consumes new records immediately after their occurrence, and applies a custom data masking function to the records for masking credit card numbers. After processing the records, the Kafka Streams application produces them to another Kafka topic, called `orders_clean`. The Kafka Streams application is a regular Java application.

A Kafka Connect sink connector subscribes to the Kafka topic `orders_clean`, consumes new records immediately after their occurrence, and publishes them to ClickHouse by executing `INSERT` statements.

The Kafka Connect connectors and the Kafka Streams application are loosely coupled, share data via Kafka topics, and are deployed independently of each other. That is, restarting the Kafka Streams application does not affect the uptime of the connectors and vice versa.

1.1.4 Benefits and challenges of streaming data pipelines

Streaming data pipelines represent a much more natural, robust, and efficient approach to dealing with unbounded datasets than their batch-based counterparts. Instead of performing recurring bulk loads at fixed times, streaming data pipelines continuously replicate data change events from the consumed data sources to the downstream data sinks.

Notably, streaming data pipelines offer the following benefits:

- **Data sinks are always in sync with data sources:** Streaming data pipelines minimize the gap between the time a change event occurs in the data source and the time the processed event arrives at the data sink, often offering sub-second latencies. They allow downstream data consumers, regardless of whether these are humans manually interpreting the data or applications processing the data in a programmatic manner, to work with up-to-date data.
- **Capturing of all changes occurring in data sources:** Log-based CDC connectors make all changes from a data source available to streaming data pipelines and downstream data consumers, while batch pipelines observe the state of the data source at the time of data extraction. Having access to high-fidelity data is beneficial for a number of use cases, including audit logs.
- **Better resource footprint:** When employing CDC, streaming data pipelines must process only the data that have changed, i.e., incremental changes. They keep all other data untouched and help to reduce the load on all involved systems and improve the overall usage of computing resources, which potentially leads to a reduction in cost.
- **Minimal impact on production database systems:** Processing incremental changes instead of performing bulk loads is beneficial for reducing the load on the consumed data source, because much less data is touched at execution time. When employing log-based CDC, streaming data pipelines further reduce the load on the consumed database system because they do not need to execute `SELECT * FROM` queries but can extract data from the replication log files of the database system.

- **Predictable workload patterns:** Batch pipelines show peak resource consumption at execution time and are stale for the rest of the time, which wastes a lot of compute resources. Streaming data pipelines have rather flat workload patterns as they continuously replicate change events. They reduce the cost and allow for the usage of elastic workload scaling, as available in cloud computing technologies like Kubernetes.

There is no free lunch. While providing clear benefits and advantages over batch processing, the architectures of streaming systems are more complex and require expertise in multiple technologies. This book helps you to overcome the hurdles associated with data streaming and prepares you for the successful adoption of streaming data pipelines at your workplace.

1.2 Is batch just a subset of streaming?

Batch and stream processing are not as opposed to each other as you might think. The big differentiator is that stream processing can handle unbounded datasets by having a notion of event time while batch processing cannot; batch processing is forced to treat datasets as bounded and finite. In other words, batch processing defines one global window over all events, while stream processing provides sophisticated means of assigning events to windows. Is batch processing thus just a subset of stream processing?

1.2.1 The stream-table duality

Let us have a look at the differences between streams of change events, like insertions or updates of single rows, and materialized tables.

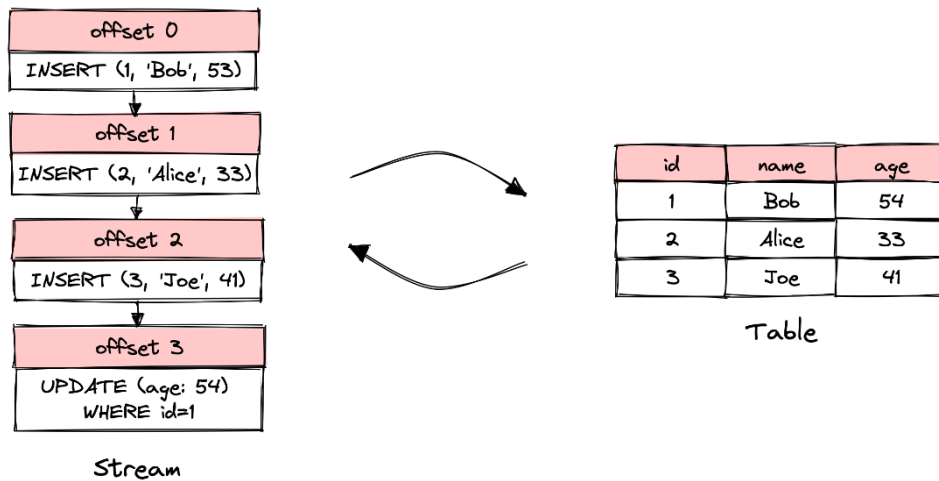


Figure 1.4 Four change events (left side) extracted from a database table (right side).

The left side of Figure 1.4 shows a sequence of four change events, three insertions and one update, which have been extracted from a transactional database. They are ordered by their occurrence; the offset resembles the position of an event in the sequence. These change events could be aggregated to build up the table that they have been extracted from, shown on the right side of Figure 1.3.

Evidently, there is a strong relationship between streams and tables: Streams resemble a historical changelog, while tables can be understood as a snapshot of a dataset at a specific time (for instance, the time when the stream has been aggregated to build up the table). That is, streams could be used to restore the state of a dataset at a specific point in time from the past. While streaming data pipelines process streams of data change events, batch pipelines work with snapshots of tables that are extracted at execution time.

1.2.2 Batching streams of data

Of course, we could extend batch pipelines with knowledge about the state of the processing. At each run, we could leverage this knowledge to extract only the records that have changed since the last execution. Replication logs typically do not hold the entire history of a database system but only those changes that occur within a specific time period, for instance, the last 24 hours. If the execution interval is shorter than the retention period of the replication log, we could employ a log-based CDC approach. If using the replication log is not feasible but the schema of the consumed dataset includes a field that holds the last change time of a row (typically called `updated_at`, `last_update`, or similar), we could also extend the `SELECT * FROM` query performed to extract data such that it reads only data that has been changed since the last execution (for instance, by adding the clause `WHERE updated_at >= $LAST_EXECUTION_TIME`).

Once we no longer process entire datasets at each run, we could schedule the batch pipeline with a higher frequency. While we would not reach the sub-second latencies of stream processing technologies, the changes would still improve the capabilities of batch pipelines in processing unbounded frequently-changing datasets.

1.2.3 Streaming batches of data

The components of a data streaming architecture are loosely coupled. For instance, connectors and stream processors are deployed independently from each other and are integrated via an event broker. Similar to how batch pipelines can process streams of events, we could also let streaming data pipelines handle batch workloads. All we need to do is replace the source connector with a batch job that is started at fixed times and fills batches of data into the event broker. The rest of the architecture would stay as is.

While such adoptions would allow streaming data pipelines to cater to batch workloads, it is probably not the most efficient implementation. Most components, e.g., the stream processors, would be idle most of the time while consuming compute resources and thus incurring cost. Evidently, streaming data pipelines are better suited for use cases with a continuous flow of data.

1.2.4 Practical examples that favor streaming over batch

In general, data streaming is the preferred implementation of data pipelines that consume unbounded datasets. You might want to choose a streaming data pipeline if you need to

- Replicate data from a transactional database system, like PostgreSQL, to a cloud data warehouse, like Google Cloud BigQuery, to empower downstream data consumers with current data.
- Monitor the database powering your SaaS application for changes and send these changes to a third-party API, like Slack. This would allow you to implement event-driven use cases, like a Slack bot, without introducing changes to your application code.
- Work with native event data, like records exposed by internet of things (IoT) sensors.
- Migrate from one database system to another one and want to avoid a "big-bang" migration. You could seamlessly stream data from your current database to the new one while preparing the migration and then just point your application to the new database once you feel ready.

1.2.5 Practical examples that favor batch over streaming

While data streaming can be used to implement almost any use case, there are a few use cases where traditional batch processing might be favorable. You should consider batch processing if you need to

- Process a static, finite CSV file once per day and perform a custom API call for each row of the CSV file.
- Generate a report for your accounting data at midnight sharp that covers the data from one specific day.

1.3 What you will learn in this book

Over the course of this book, we take on a journey through the world of streaming data pipelines. We focus on Apache Kafka as foundational technology; Kafka has been adopted by many businesses, is available as a managed service at all major cloud platforms, and is backed by an ever-growing open-source community. We cover implementing connectors with Kafka Connect and developing stream processors with Kafka Streams.

This book is a complete hands-on guide that prepares you to successfully develop, deploy, and operate streaming data pipelines with Apache Kafka and its ecosystem at your workplace. It not only covers the different building blocks of streaming data pipelines but also discusses topics, like deployment, operations, and monitoring. To stay as close as possible to the real world, a realistic case study from an e-commerce company, which migrates its error-prone batch data pipelines to streaming data pipelines, is developed throughout the book.

The next chapter provides an introduction to the data streaming landscape and discusses popular technologies for implementing storage, connectors, processing, and analytics, putting a focus on the ones covered in this book.

Part 2 of this book is about integrating streaming data pipelines with external data sources and data sinks using Kafka Connect. Chapter 3 shows how to use existing connectors from the open-source community. Chapter 4 walks you through building custom connectors with Kafka Connect.

Part 3 of this book covers processing data in real-time with Kafka Streams. Chapter 5 discusses implementing stateless transformation logic, like mapping data, filtering data, and routing data. Chapter 6 shows how to apply stream processors to windows of events and develop stateful transformation logic, e.g., deduplicating events, aggregating data, and joining data.

Part 4 of this book completes preparing you for the real world by discussing the deployment of streaming data pipelines. Chapter 7 shows how to package and deploy streaming data pipelines, using container technologies like Docker and Kubernetes. Chapter 8 covers the monitoring of the different components of streaming data pipelines, while Chapter 9 shows you how to tune their performance for different latency and throughput requirements. Finally, Chapter 10 discusses potential errors in streaming data pipelines and provides strategies for handling them.

After reading this book, you will be able to

- Serve real-time data to business departments of your organization
- Understand essential concepts of streaming data pipelines, e.g., CDC, backfilling snapshots, consumer lags, etc.

- Overcome the challenges associated with adopting streaming data pipelines
- Set up open-source connectors with Kafka Connect and develop your own custom connectors in Java
- Implement stateless and stateful stream processors with Kafka Streams
- Tune the performance of streaming data pipelines for low-latency and high-throughput requirements
- Scale streaming data pipelines to cope with different performance requirements
- Debug and monitor streaming data pipelines

1.4 Summary

- Streaming data pipelines replicate data from data sources to data sinks in real-time and can process them on the way.
- CDC extracts data change events (insertions, updates, and deletions) from data sources and makes them available to a stream processor.
- Most datasets in the real world are unbounded and change over time.
- Stream processing technologies provide sophisticated operators for working on windows of unbounded datasets, while batch processing must assume that a dataset is complete at execution time.
- Batch data pipelines are a subset of streaming data pipelines. Streaming data pipelines work on streams of change events, while batch data pipelines process snapshots of entire datasets at a time.
- Apache Kafka is an event store used for persisting unprocessed and processed data in streaming data pipelines.
- Kafka Connect lets you integrate streaming data pipelines with external data sources and data sinks. You can use existing connectors from the open-source community or develop custom connectors.
- Kafka Streams is a stream processing framework that can be used to implement stateless and stateful processing logic.

2

A walk through Kafka and its ecosystem

This chapter covers

- Storing events at scale with Apache Kafka
- Setting up streaming connectors with Kafka Connect
- Processing events in real-time with Kafka Streams
- Getting up and running with Apache Kafka on your local machine

The first chapter presented the building blocks of streaming data pipelines: Event storage, connectors, and stream processors. This chapter dives deeper into the technologies that can be used for implementing them. We strongly focus on storing events with Apache Kafka, implementing connectors with Kafka Connect, and processing events with Kafka Streams. For completeness, we cover alternative technologies as well.

We refrain from diving into all details of Apache Kafka, Kafka Connect, and Kafka Streams but focus on the aspects that are essential for successfully using these technologies in the context of streaming data pipelines. Throughout the book, we develop a practical case study. This chapter helps you set up the used technologies on your local computer using Docker containers.

IMPORTANT We assume that you have a running Docker installation on your local machine. If you need help getting up and running with Docker, please see the installation instructions on the official website: <https://docs.docker.com/get-docker/>. The code examples have been tested on Linux and OS X. If you are working with Windows, consider prepending the shell commands with `winpty` (<https://github.com/rprichard/winpty>), e.g., execute `winpty docker run` instead of `docker run`.

Let's get started with exploring how streaming data pipelines can store events with Kafka.

2.1 Storing events with Apache Kafka

Streaming data pipelines use event storage for integrating connectors with stream processors. First, pipelines persist unprocessed events that have been extracted from an external data system by a source connector and need to be processed by a stream processor. Second, pipelines store events that have been processed by a stream processor and need to be published to another data system by a sink connector. An event broker allows one to store events in a reliable, scalable, and efficient manner. It provides interfaces for consuming (or reading) and producing (or writing) events.

Apache Kafka is an open-source event broker developed by the Apache Software Foundation. Since its initial release in 2011, Apache Kafka gained tremendous traction in the developer community and has been deployed at many enterprises.

Apache Kafka allows one to read, write, and store event data at scale. It is a distributed system: One Kafka cluster is typically deployed onto multiple physical machines and can scale its performance with the number of machines. Figure [2.1](#) visualizes how consumers and producers interact with a Kafka broker.

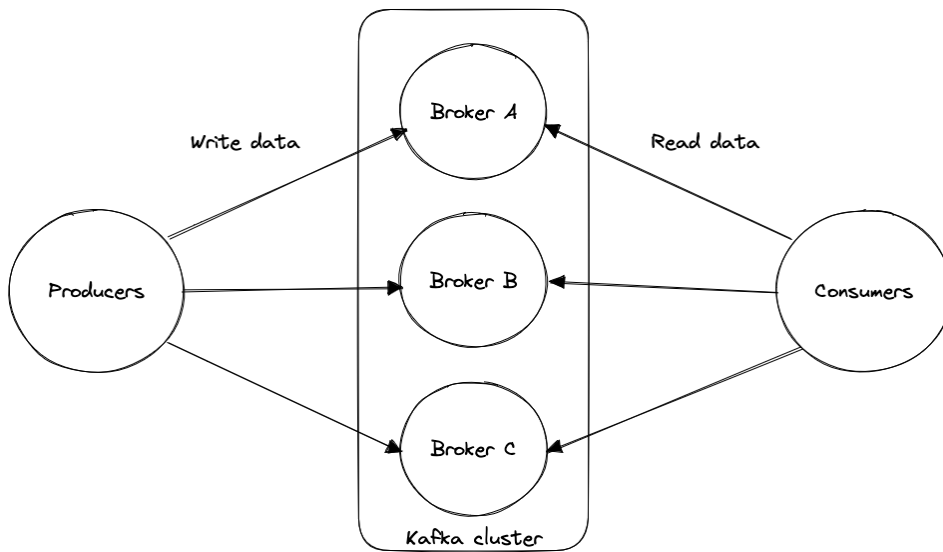


Figure 2.1 Consumers and producers interacting with an Apache Kafka cluster to read and write data.

Kafka provides low-level APIs for consuming and producing events, with official libraries being released for Java and Scala and unofficial libraries being available for almost any programming language. Kafka achieves a low latency and a high throughput, often satisfying the available network bandwidth, by employing sequential I/O, which provides much higher performance than accessing HDDs and SSDs randomly, and the zero-copy principle, which lets the kernel copy data directly from the disk to the socket without going through the application and leads to a much higher performance. As opposed to message queues that do not store data beyond consumption, Kafka persists data in a distributed log and allows users to configure the data retention: It can store data infinitely, for a specific period (e.g., for one week), or up to a specific size (e.g., up to one gigabyte).

Traditionally, Apache Kafka managed some metadata in Apache ZooKeeper and thus required operating both systems, which caused operational complexity. Since version 3.3, Apache Kafka offers KRaft, an internal consensus protocol for distributing metadata across the cluster, as a production-ready replacement for ZooKeeper. In this book, we cover KRaft and refrain from using ZooKeeper.

Let's have a closer look at the concepts of Apache Kafka.

2.1.1 Brokers

An Apache Kafka cluster consists of one or multiple *brokers* that are typically running on separate servers. When connecting to a Kafka cluster, you can use the configuration option `bootstrap.servers` to point your client to one or multiple brokers of your Kafka cluster. After successfully connecting to the broker, your client will fetch metadata and information about the structure of the cluster before it is ready to handle your requests.

Apache Kafka has been designed as a distributed system for multiple reasons:

- **High performance:** By running on multiple brokers, Apache Kafka can distribute the load of workloads across multiple physical machines and achieve higher performance. It is not limited by the network bandwidth and compute resources of a single machine but can scale its performance with the number of brokers.
- **Fault tolerance:** Apache Kafka can replicate data over multiple brokers to become robust towards potential outages and failures. If, for instance, one broker goes down because of a broken disk, other brokers can (temporarily) take over its roles in the cluster, which ensures that the Kafka cluster remains available.
- **High Availability:** Apache Kafka clusters are highly available by distributing work over multiple brokers and can remain online despite a single broker becoming unavailable.

Let's start a local Apache Kafka cluster with one broker in KRaft mode using Docker, as shown in Listing 2.1:

Listing 2.1 Starting a local Apache Kafka cluster with one broker.

```
$ docker run --rm \
--name kafka \ #1
--hostname kafka \ #2
--env 'CLUSTER_ID=ZGI1NTk0YmY3NzVjNDk5MD' \ #3
--env 'KAFKA_CONTROLLER_QUORUM_VOTERS=1@kafka:29093' \ #4
--env 'KAFKA_LISTENERS=PLAINTEXT://kafka:29092,
--CONTROLLER://kafka:29093' \ #5
--env 'KAFKA_ADVERTISED_LISTENERS=PLAINTEXT://kafka:29092' \ #6
confluentinc/confluent-local:7.5.0 #7
```

#1 Name of the container.

#2 Make the container accessible via the hostname kafka.

#3 KRaft-related config option that identifies the cluster (can be generated with `docker run confluentinc/confluent-local:7.5.0 /bin/kafka-storage random-uuid`).

#4 KRaft-related config option for assigning the quorum voters.

#5 Network interfaces that Kafka binds to. PLAINTEXT is used by clients and CONTROLLER is used by other brokers.

#6 Network interfaces that Kafka communicates with clients.

#7 A container image purpose-built for local development with Apache Kafka. Please don't get confused by the image tag, which equals the release of Confluent Platform; the image ships version 3.5 of Apache Kafka.

Of course, a cluster with only one broker is neither tolerant towards failures nor highly available. Also, its performance might be not very impressive. Nonetheless, you should now have an Apache Kafka cluster running on your local machine that you can use for development purposes.

Please keep this container running, so we can use it throughout the next sections.

2.1.2 Records

Apache Kafka lets you consume and produce *records*. Records hold your data.

NOTE You can use the terms events, records, and messages interchangeably.

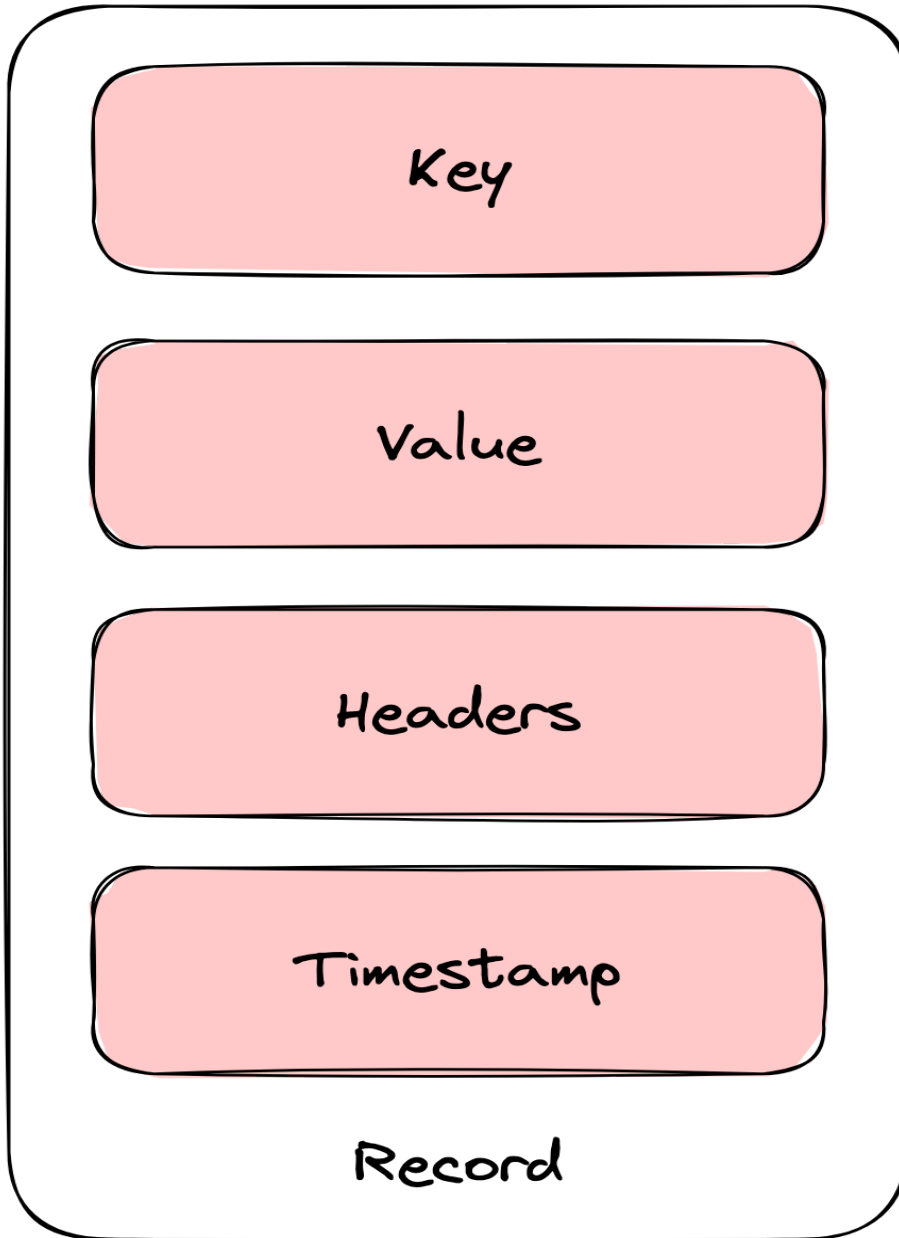


Figure 2.2 The structure of records in Apache Kafka.

As visualized in Figure [2.2](#), records consist of a key, a value, headers, and a timestamp:

- Keys hold data and are used for partitioning data, which is a prerequisite for parallelizing read workloads in Apache Kafka. When streaming CDC events, keys typically store the primary key of the associated row. Keys can be null.
- Values hold data and are typically the largest property of a record. When streaming CDC events, they typically hold the entire CDC event. Values can be null.
- Headers store user-provided metadata. They are optional and can be useful for storing data that should not become part of the key or the value.
- Timestamps can hold the time when an event has been created or processed. If they are not provided by a user, Kafka will automatically fill them with the time that the record has been produced. Kafka topics can be configured to use *LogAppendTime*; in this case, the Kafka broker overwrites the record timestamps with the time at which it persists the records in its log.

Kafka does not maintain any knowledge about the format of keys and values but treats them as byte arrays. Consumers and producers are responsible for knowing the data format and must take care of serializing and deserializing their data themselves. Timestamps are defined as long values and headers are defined as lists of key-value pairs where keys are strings and values are byte arrays. Listing [2.2](#) shows an exemplary record that has been produced by a source connector using change data capture. The record key holds a string and the record value holds a JSON object reflecting the complete CDC event.

Listing 2.2 An exemplary record that has been produced by a CDC source connector.

```

"732190" #1
{ #2
  "before":null,
  "after": {
    "pk":"1",
    "value":"New data"
  },
  "source": {
    "snapshot":"incremental"
  },
  "op":"r",
  "ts_ms":"1689180728000",
  "transaction":null
}
null #3
1689180728 #4

```

#1 The key of the record, which holds a string.

#2 The value of the record, which holds a JSON object.

#3 The headers of the record (here left empty).

#4 The timestamp of the record.

2.1.3 Topics

Similar to how database systems provide tables for organizing rows, Apache Kafka lets you store records in *topics*. Each topic has a name that is unique for a given Kafka cluster.

In Kafka, topics are partitioned. When creating a new topic, you can choose the number of partitions via the configuration option `num.partitions`. Partitions of the same topic should not all reside on the same broker but can be distributed by Kafka across different brokers (or physical machines), which is crucial for scaling the performance of the consumers and producers of a topic. By default, Kafka guarantees that records are ordered by their production time within each partition (but not at the topic level). When using Kafka's API to produce records to a topic, you can choose the to-be-used partition for each record. If you do not provide a partition, by default, Kafka will use the record key to determine the partition and guarantee that records with the same key (yes, different records can have the same key, like multiple CDC events associated with the same database row) end up in the same partition.

Let's imagine that we want to store activities from the visitors of our website in Kafka. Before being able to consume or produce the data, we need to create a topic. Listing [2.3](#) shows how to create a Kafka topic, named *visitors*, with three partitions and a replication factor of 1 on our local Kafka cluster:

Listing 2.3 Creating the Kafka topic visitors on the terminal.

```
$ docker exec kafka \
  /bin/kafka-topics \ #1
  --bootstrap-server=kafka:29092 \ #2
  --topic=visitors \ #3
  --partitions=3 \ #4
  --replication-factor=1 \ #5
  --create
```

#1 Shell script for managing topics.

#2 Connect with the local Kafka cluster.

#3 Name of the to-be-created topic.

#4 Number of partitions.

#5 Replication factor.

When listing the topics of the Kafka cluster (see Listing [2.4](#)), we should see the new topic :

Listing 2.4 Listing the topics of the Kafka cluster.

```
$ docker exec kafka \
  /bin/kafka-topics \
  --bootstrap-server=kafka:29092 \
  --list
```

If we would keep just one copy of each partition in the entire Kafka cluster, all partitions (and data) that reside on a specific broker would become unavailable if the broker goes down, e.g., for a restart. Kafka allows you to replicate topics/partitions across different brokers. You can choose the number of replicas of a topic by setting the configuration option `replication.factor` when creating the topic. A typical value for achieving high availability is 3, which allows two brokers to become unavailable while still keeping at least one replica online. For each partition, exactly one broker takes over the role of a *leader* while the brokers hosting the remaining replicas take over the role of a *follower*. While produce (or write) requests can only be sent to the leader, consume (or read) requests can be handled by followers as well.

NOTE We cannot choose a replication factor larger than the number of brokers available in our cluster. If we would do so, Kafka would throw an error when creating the topic.

Partitions store records sorted by their occurrence. Kafka assigns a unique, incrementing ID, the *offset*, to each record in a partition. The first record of the partition gets the offset 0, the second record gets the offset 1, etc. Offsets are only valid within the scope of a specific partition. Figure X visualizes a topic with three partitions holding a varying number of records. The shown numbers are the offsets of the records.

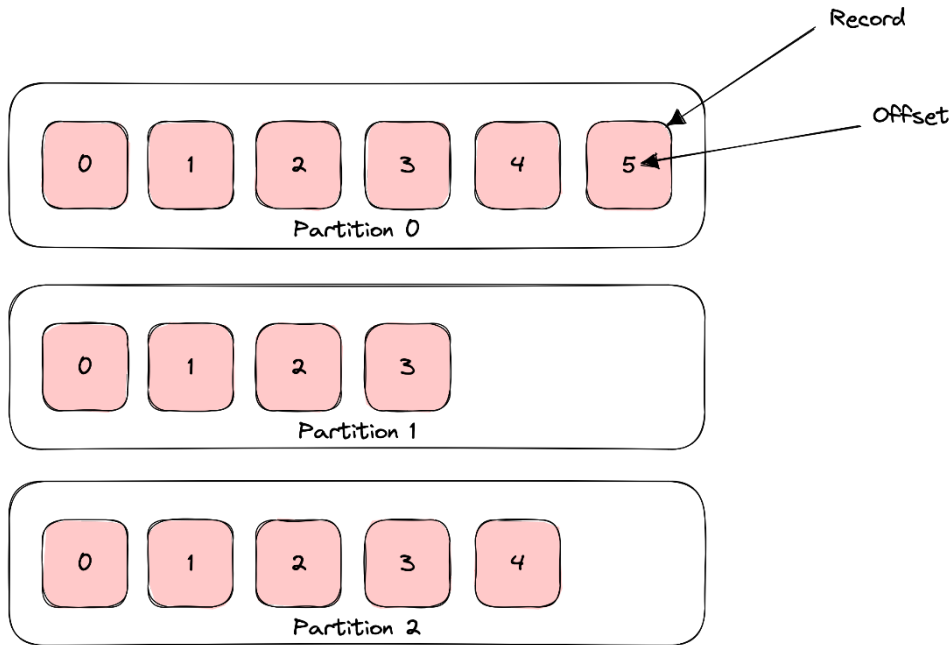


Figure 2.3 Three topic partitions with a varying number of records. The shown numbers are the offsets of the records.

2.1.4 Producers

Producers are client applications that produce (or write) records to a topic of a Kafka cluster. Unless the client application specifies the topic partition to use for the record, Kafka will automatically choose a partition such that records with the same key end up in the same partition. If the record key is not defined, Kafka assigns records to partitions using a round-robin approach; when producing records in batches, records of the same batch end up in the same partition though for performance reasons.

Let us produce two records, resembling website visits, to the topic that we created in the last section. We store the IP address of the visitor as a string in the record key, and more information about the event (e.g., the browser, country, and device of the visitor) as a JSON object in the record value. Listing 2.5 shows how to produce two records to the topic `visitors`. After executing the first command from the listing, `docker exec -it kafka /bin/kafka-console-producer ...`, in your terminal, you can interactively produce records by entering the key and the value of the to-be-produced record separated by a semicolon and followed by the enter key.

Listing 2.5 Producing two records to the topic visitors

```
$ docker exec -it kafka \
  /bin/kafka-console-producer \ #1
  --bootstrap-server=kafka:29092 \ #2
  --topic=visitors \ #3
  --property "parse.key=true" \ #4
  --property "key.separator=;" #5

> 192.158.1.38;{"browser":"Firefox","country":"USA","device":"Laptop"} #6
> 127.0.0.1;{"browser":"Chrome","country":"Germany","device":"Mobile"} #7
```

#1 Shell script for producing records.

#2 The broker that we want to connect to.

#3 The topic used for producing records.

#4 Enable the production of keys (if deactivated, keys are set to null).

#5 Separator between the record's key and value.

#6 The first record. Please press the *enter key* at the end of the line to produce the record.

#7 The second record. Please press the *enter key* at the end of the line to produce the record.

Kafka shifts a lot of responsibilities, like record durability or ordering, to client applications and lets them take care of these tasks. In the following, we walk through the most important concepts and configurations of producers in Apache Kafka.

In most cases, Kafka processes the records in the order that they have been produced in and maintains that sorting within the scope of a single partition, as indicated by the record offsets. There might be rare cases where records end up in a different order at the broker than they have been produced in because the client application had to retry producing single records while sending multiple produce requests at the same time. If you want to let the client retry failed requests but have strict requirements regarding the ordering of records, you can refrain from sending multiple produce requests at the same time by setting the producer configuration `max.in.flight.requests.per.connection` to the value 1. In the end, it is a tradeoff between performance and ordering guarantees.

Producers can be, for instance, implemented in Java using the official library of Apache Kafka. Unless a single producer already satisfies the capacities of your Kafka cluster, you can scale its performance by launching multiple instances of the client application and letting them produce data in parallel to the same topic but potentially different partitions. Please note that this strategy is only possible if you do not have strict requirements regarding the ordering of records. For instance, when producing multiple CDC events concerning the same database row, thus having the same record key, you typically want them to be produced, stored, and processed in the order of their occurrence.

Kafka lets your client application choose how many brokers must acknowledge its produce requests by setting the producer configuration `acks`. Let us assume that you have a cluster with multiple brokers and produce to a topic with 3 replicas. If you use `acks=0`, produce requests immediately succeed without waiting for the acknowledgment of the partition leader. When using `acks=1`, produce requests succeed once the partition leader has acknowledged the produce request. When using `acks=all`, produce requests succeed once all in-sync replicas have successfully processed and acknowledged the request. Again, Kafka lets you choose between a high performance and low-latency requests (`acks=0`) and strong processing guarantees (`acks=all`).

NOTE Depending on the configured replication factor, Apache Kafka replicates each partition onto different brokers of the cluster. An in-sync replica (ISR) is a broker that holds the latest version of a partition. You can configure the minimum number of in-sync replicas via the configuration option `min.insync.replicas`, both on the cluster and the topic level. If the number of in-sync replicas is below the value of `min.insync.replicas`, produce requests to the affected partition will be rejected by Kafka.

Client applications do not have to send record by record but can batch multiple records together. Producing records in batches is beneficial for achieving high throughput because you can send more data with fewer requests and you can achieve higher compression rates when using a compression algorithm, but might harm the latency of your producer because you need to wait until a batch fills up before sending the product request to the broker. Client applications can use two options to configure the batching of records in a Kafka producer. The option `batch.size` defines the minimum amount of data that are sent per request (default: 16384 bytes). The producer waits until a local buffer is filled up with at least `batch.size` bytes before initiating the request. There might be times when the local buffer remains partially filled for a long time. To this end, the option `linger.ms` lets the client application define how long the producer shall wait before initiating the produce request, regardless of whether `batch.size` bytes have been reached or not. By default, `linger.ms` is set to 0, therefore batching is not enabled in the standard configuration of a Kafka producer.

NOTE Kafka Connect and Kafka Streams sit one layer above Kafka's producer API and let you build connectors and stream processors without having to deal with the low-level producer API of Kafka.

2.1.5 Consumers

Consumers are client applications that consume (or read) records by subscribing to one, many, or all partitions of a Kafka topic. For each partition, the consumer maintains the most recent offset that it has processed successfully. When starting the consumer for the first time or, generally speaking, in the absence of a committed offset, a consumer can start reading from the earliest (`auto.offset.reset=earliest`) or the latest (`auto.offset.reset=latest`) available offset. Consumers can automatically commit their offsets to an internal Kafka topic by setting the configuration option `enable.auto.commit` to `true` or must manually commit their processing state via Kafka's consumer API. Offsets are persisted beyond the lifetime of a single consumer instance, such that, in the case of a restart, consumers can resume the processing at the offset where they stopped without having to reprocess records.

Let us consume the topic `visitors` from our local Kafka cluster, as shown in Listing

Listing 2.6 Consuming records from the topic visitors

```
$ docker exec kafka \
  /bin/kafka-console-consumer \ #1
  --bootstrap-server=kafka:29092 \ #2
  --topic=visitors \ #3
  --from-beginning \ #4
  --property print.key=true #5
```

#1 Shell script for consuming records.

#2 The broker that we want to connect to.

#3 The topic from which we want to consume records.

#4 Start consuming records from the beginning of the topic, i.e., from the earliest offset of each partition, which equals setting `auto.offset.reset=earliest` in the consumer API.

#5 Print the keys of records.

If you keep the script `./bin/kafka-console-consumer.sh` open, you should instantly see records that are being produced to the topic. Give it a try using the script from Listing [2.5](#)!

Each consumer is identified by a `group.id`, which can be passed as a configuration option to Kafka's consumer API. The `group.id` must be unique across an entire Kafka cluster. When starting multiple client applications that specify the same `group.id`, they become a *consumer group* and start consuming data in a cooperative, parallel manner. Each member of the consumer group is responsible for consuming data from at least one partition.

NOTE It would not make sense to run a consumer group with more members than partitions available, since no partition can be assigned to multiple consumers at the same time.

Figure 2.4 illustrates a topic with 3 partitions that are consumed by two consumer groups. Consumer group A consists of 2 consumers: One consumer is assigned to partitions A and B, and another consumer is assigned to partition C. Consumer group B consists of 1 consumer that is assigned to all three partitions.

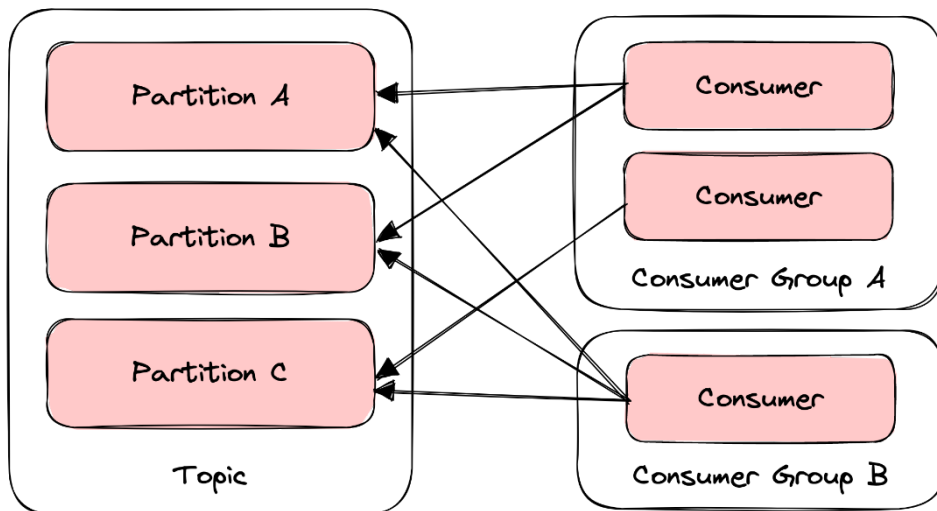


Figure 2.4 A Kafka topic with three partitions that are consumed by two consumer groups. Consumer group A has two members; consumer group B has one member.

Consumer groups are Kafka's approach to parallelizing the consumption of data. You might have noted that we cannot create consumer groups with an arbitrary number of consumers but are restricted by the number of partitions of the topic. Since the partitioning of data is typically not random in Kafka topics, we cannot easily change the number of partitions as needed. Thus, we need to carefully choose the number of partitions at topic creation time depending on the expected degree of parallelism that we need to achieve the desired performance.

TIP Partitions allow consumers and producers to parallelize their work. Given that you know the throughput T that you want to achieve, you can determine the optimal number of partitions by measuring the performance of your producer P_p and/or consumer C_p for a single partition. The optimal number of partitions equals the maximum of T / P_p and T / C_p . Let us have a look at an example, in which you want to achieve a throughput of 100 MB/s. If your producer can write to a single partition with a throughput of 10 MB/s and your consumer can read from a single partition with a throughput of 5 MB/s, you should create a topic with at least $\text{Max}(100/10, 100/5) = 20$ partitions.

Technically, Kafka consumers boil down to loops that recurringly poll the Kafka broker for new records. Consumers can define how many records should be polled per loop iteration via the option `max.poll.records` (default: 500), and can let Kafka know how long it shall wait for the completion of a loop iteration before marking the consumer as failed via the option `max.poll.interval.ms` (default: 300000, i.e., 5 minutes).

Assuming that we know (1) the latest offset of a topic partition and (2) the most recent offset that a consumer has processed, we can determine the number of records that the consumer has not yet processed, which is called *consumer lag*. Consumer lags are one of the most essential metrics that you need to monitor when running Kafka clients in production. While temporary fluctuations of consumer lags are normal, continuously growing consumer lags are worrying and indicate that the consumer is processing data at a slower pace than they are being produced. A practical example might be a streaming ETL pipeline replicating data from a database to an API, where the database produces two CDC events per second but the API can process only one call per second. This situation is also called *backpressure*. One of the ways to fix such a case is to increase the size of the consumer group, if possible, such that more data are being processed in parallel.

2.1.6 Deployment options

For local development, running Apache Kafka with Docker is a straightforward solution for quickly spinning up a non-production cluster. While you can start multiple brokers on your local machine, using Docker for running brokers on different physical machines is not possible.

If you are running Kubernetes and prefer to manage Apache Kafka yourself, we recommend using RedHat's open-source operator Strimzi. Strimzi automates the deployment and management of Kafka on Kubernetes, simplifies the operation of Kafka, and comes with great default configurations for various use cases.

If you neither have the requirements nor headcount to manage Apache Kafka yourself, we recommend using a managed service. Today, there are plenty of services out there, like Confluent Cloud, Aiven for Apache Kafka, Amazon MSK, etc. When choosing between the available options, you might consider questions, like:

- Do you require your data to reside within your cloud environment? If that is the case, using a pure SaaS solution, like Confluent Cloud, might not be possible.
- Do you need additional services, like MirrorMaker2 or Kafka Connect? If you answer that question with yes, you might want to choose a service that offers managed services for other Kafka-related products as well.
- Does your cloud provider offer a managed service for Apache Kafka? This might ease the procurement process as you do not need to enter a business relationship with yet another cloud business.

2.1.7 Other technologies

Besides the upstream distribution of Apache Kafka from the Apache Software Foundation, there are a couple of other event brokers available that offer a Kafka-compatible API and allow you to reuse client applications or open-source products developed for Kafka.

AZURE EVENT HUBS

Azure Event Hubs is a managed service by Microsoft that implements the Kafka API and can be used with any existing client application or tool from the Kafka community. You might want to consider Event Hubs if you are looking for a managed service and are already using Azure for other workloads.

REDPANDA

Redpanda is a C++ implementation of the Kafka API that reduces the operational complexity by shipping its own consensus protocol, allows to reduce cost by offloading data to cloud object storages, and claims to achieve a higher performance than the open-source distribution of Apache Kafka. You can manage Redpanda yourself on Kubernetes or bare metal, or use the managed service Redpanda Cloud. Redpanda Cloud supports the bring-your-own-cloud model that separates the control from the data plane: You can benefit from the simplicity and robustness of the managed service and, at the same time, keep your data within your cloud environment.

2.2 Integrating streaming data pipelines with Kafka Connect

So far you have learned about the core concepts of Apache Kafka, like brokers, topics, partitions, and records. You also saw how to produce and consume data with Kafka, and tried it out yourself.

In many use cases, like streaming data pipelines, Kafka is integrated with other technologies, like database systems, APIs, or data warehouses. Such integrations can be built as ordinary client applications that use the consumer and producer API of Kafka. That is a lot of repetitive work assuming that you (and the rest of the community) need to integrate the same systems over and over again - what a waste of time!

Kafka Connect is a tool for integrating Apache Kafka with external data systems. Building on top of the official consumer and producer APIs of Kafka, Kafka Connect provides a framework for implementing source and sink connectors and lets users manage them via a REST API.

- Source connectors extract data from external systems and publish them to Kafka topics; technically, they are Kafka producers.
- Sink connectors stream data from Kafka topics to external systems; technically, they are consumers.

Connectors are implemented in Java and can be distributed as Java archive (JAR) files. Kafka Connect has been designed with extensibility and reusability in mind; it allows you to add both your custom connectors and existing connectors from the community. When starting Kafka Connect, it loads all connectors that are included in its classpath and makes them available for usage.

Kafka Connect is part of the open-source distribution of Kafka but runs in separate processes (or containers), in parallel to a Kafka cluster. While Kafka Connect is typically deployed in a distributed setup, running on multiple physical machines, Listing [2.7](#) shows how to spin up a single-node cluster for development purposes (please note that we need to point it to a running Kafka cluster; we use the one created in the previous section):

Listing 2.7 Starting a local Kafka Connect cluster.

```
$ docker run --rm \
  --name kafka-connect \ #1
  --hostname kafka-connect \ #2
  -p 127.0.0.1:8083:8083 \ #3
  --link kafka:kafka \ #4
  --env 'BOOTSTRAP_SERVERS=kafka:29092' \ #5
  --env 'GROUP_ID=1' \ #6
  --env 'CONFIG_STORAGE_TOPIC=connect-configs' \ #7
  --env 'OFFSET_STORAGE_TOPIC=connect-offsets' \ #8
  --env 'STATUS_STORAGE_TOPIC=connect-statuses' \ #9
  quay.io/debezium/connect #10
```

#1 Name of the container.

#2 Make the container accessible via the hostname `kafka-connect`.

#3 Publish the port needed for connecting with Kafka Connect on the host machine.

#4 Link the container to the Kafka container.

#5 The Kafka cluster associated with this Kafka Connect cluster.

#6 Unique identifier of the Kafka Connect cluster.

#7 Name of the Kafka topic that shall be used for storing connector configs.

#8 Name of the Kafka topic that shall be used for storing connector offsets.

#9 Name of the Kafka topic that shall be used for storing connector statuses.

#10 A Docker image for Kafka Connect provided by the Debezium community.

NOTE In this book, we run Kafka Connect in the *distributed mode*, which stores metadata, like connector configs, in the associated Kafka cluster. For testing purposes and local development, Kafka Connect can be run in a *standalone mode*, which uses the local filesystem for storing the configs, offsets, and statuses of connectors.

You should now be able to access the API interface of your Kafka Connect cluster, which is available under <http://localhost:8083>, as shown in Listing 2.8:

Listing 2.8 Accessing the REST API of Kafka Connect.

```
$ curl http://localhost:8083
{"version":"3.4.0","commit":"2e1947d240607d53",
  "kafka_cluster_id":"ZGI1NTk0YmY3NzVjNDk5MA"}
```

The root endpoint of Kafka Connect's API, <http://localhost:8083/>, returns basic information about the cluster, e.g., the version of Kafka Connect or the ID of the associated Kafka cluster: Please see the Kafka website for a complete documentation in the OpenAPI format: https://kafka.apache.org/35/generated/connect_rest.yaml.

2.2.1 Tasks and workers

When executing connectors, Kafka Connect maps them to *tasks*. Each instance of a connector becomes one task. Kafka Connect supports parallelizing the processing of data, by letting connectors launch multiple instances (or tasks).

When running Kafka Connect in the standalone mode, there is only one JVM process that takes care of executing all connector tasks.

When running Kafka Connect in the distributed mode, Kafka Connect distributes all tasks to different *workers*. Workers are JVM processes that can be potentially executed on different physical machines. Each worker (or JVM process) can run multiple tasks (or connectors). Kafka Connect aims at equally distributing the tasks across the different workers.

Figure 2.5 visualizes a Kafka Connect cluster with three workers and four connectors. Connectors A and B have two tasks each; connectors B and C have one task each.

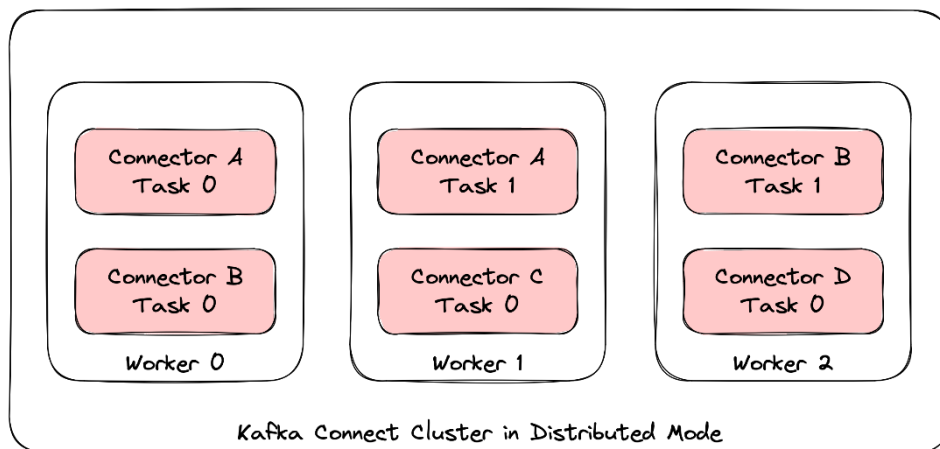


Figure 2.5 A Kafka Connect cluster with three workers and four connectors.

2.2.2 Configuring connectors

Connectors can be configured with JSON objects that are passed via API calls (We walk through the most important API endpoints in Chapter 3). While most configuration options are very specific to the used connector type, some options are available for all connectors.

First, each connector must provide a unique *name*, which is used for identifying the connector within a Kafka Connect cluster. Second, regardless of the used connector type, the following configuration options are available for the *config* field:

- **connector.class:** The config option *connector.class* specifies the name of the Java class of the connector, which must be available in the classpath of Kafka Connect. For instance, when using Debezium's MySQL source connector, you would set `connector.class = io.debezium.connector.mysql.MySqlConnector`.
- **key.converter:** The config option *key.converter* specifies the to-be-used data format for the record key by pointing the connector to a Java class that takes care of the (de)serialization. You can either use existing converter classes or configure your own implementation (after adding it to the classpath of Kafka Connect). For instance, you can set `key.converter=org.apache.kafka.connect.json.JsonConverter` for using JSON.
- **value.converter:** The config option *value.converter* specifies the to-be-used data format for the record value. The option works similarly to *key.converter*.
- **tasks.max:** The config option *tasks.max* lets you define the number of tasks that Kafka Connect shall start for the connector, which equals the degree of parallelism. Please note that some source connectors require the sequential processing of data and cannot be easily parallelized.
- **topics:** The config option *topics* is only required for sink connectors and lets the connector know which topic to use as the input. Alternatively, you can specify the config option *topics.regex* to point the connector to a list of topics, defined by a regular expression.
- **transforms:** The config option *transforms* can be used to define single-message transforms (SMT). Please see the next section for a detailed description.

2.2.3 Single-message transforms

Kafka Connect provides SMTs as a lightweight means to implement stateless processing logic in combination with connectors:

- Source connectors can define SMTs for transforming data before producing them to the Kafka topic.
- Sink connectors can specify SMTs for transforming data before publishing them to the external data system.

SMTs are rather simple functions that take a Kafka record as the input, apply some processing logic that does not require state (e.g., mask credit card numbers, drop obsolete information from values, or generate new record keys), and return a modified version of the record. Figure 7 visualizes the role of SMTs in source and sink connectors.



Figure 2.6 The role of SMTs in source and sink connectors.

Kafka Connect provides a couple of SMTs out of the box that you can apply by referencing them in the configuration of your connector, for instance:

- **Cast** casts fields of the record key or value.
- **ExtractField** extracts a field from a map object.
- **InsertField** inserts a field into the record key or value.
- **ValueToKey** copies the record value to the key.

Please see the Kafka Connect documentation for a complete list of the pre-defined SMT functions: https://kafka.apache.org/documentation/#connect_included_transformation

Similar to custom connectors, Kafka Connect allows you to implement your own SMT functions in Java. You can deploy them to a Kafka Connect cluster by adding the JAR file to the classpath of Kafka Connect.

We will have a closer look at SMTs in Chapter 3.

2.2.4 Deployment options

For local development, running Kafka Connect with Docker, as demonstrated in this chapter, helps you quickly spin up a non-production cluster.

If you are running Kubernetes and prefer to manage Kafka Connect yourself, we recommend using RedHat's open-source operator Strimzi. Strimzi automates the deployment and management of Kafka Connect on Kubernetes.

If you neither have the requirements nor the headcount to manage Kafka Connect yourself, we recommend using a managed service. Today, there are plenty of services out there, like Confluent Cloud, Aiven for Kafka Connect, Amazon MSK Connect, etc. When choosing between the available options, you might consider questions, like:

- Do you require your data to reside within your cloud environment? If that is the case, using a pure SaaS solution, like Confluent Cloud, might not be possible.
- Do you have your own connector implementations and require running custom JARs? If you can answer that question with a yes, managed services that do not support custom connectors might be easily ruled out.

CONDUIT: AN ALTERNATIVE TO KAFKA CONNECT

Conduit is an open-source connector tool implemented in Go and can be considered as an alternative to the Java-based Kafka Connect. Conduit provides its own software development kit (SDK) for implementing connectors. In addition to connectors implemented with its SDK, it can also run Java-based Kafka Connect connectors, which allows users to benefit from the existing set of connectors from the Kafka Connect community.

Conduit applies a few concepts from the Go community to the development and the deployment of streaming connectors. For instance, Conduit offers higher robustness by executing connectors in their own goroutines, while Kafka Connect runs multiple connectors potentially in the same JVM process.

Conduit has been recently published and, as a consequence, has far fewer production deployments than Kafka Connect. Nonetheless, it is a very interesting technology that you might want to follow or try out. Feel free to have a look at Conduit's GitHub repository: <https://github.com/ConduitIO/conduit>

2.3 Processing events with Kafka Streams

So far we have seen how to store event data in Apache Kafka and how to integrate Kafka topics with external data systems using Kafka Connect. We also learned that Kafka Connect provides lightweight means for transforming events while streaming them. In this section, we have a look at a more sophisticated, general-purpose technology for processing and transforming data in real-time: Kafka Streams.

Kafka Streams is a client library for building stateful streaming applications with Apache Kafka. Similar to Kafka Connect, Kafka Streams applications are associated with a specific Kafka cluster and build on top of the official consumer and producer APIs of Kafka. They consume their input data from Kafka topics, process the data with stateless and stateful operators, and produce the processed output data to other Kafka topics. Kafka Streams applications can be represented as a directed acyclic graph (DAG) with processing nodes and edges. *Source nodes* consume data from the input topics and make them available to the application. The data flows through a chain of user-defined *processing nodes* before being produced to output topics by *sink nodes*. Figure [2.7](#) visualizes an exemplary Kafka Streams application as a DAG. The application consumes the Kafka topic `orders`, applies a filter operator to select those records that have a value with the field `country` set to `UK`, and produces the records matching the filter to the Kafka topic `orders_uk`.

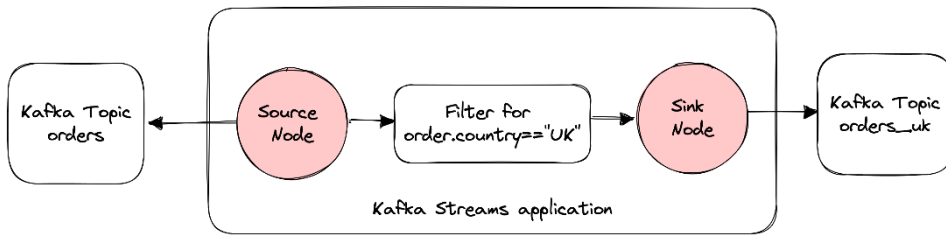


Figure 2.7 A Kafka Streams application with one filter operator visualized as a DAG.

The Kafka Streams client library, which is available for Java and Scala, provides two APIs for implementing processing logic:

- The **declarative, functional domain-specific language (DSL)**, which looks very similar to Scala's collection API or Java Streams, provides a set of well-designed building blocks that let users express processing logic in a few lines of code.
- The **imperative, low-level processors API** is more flexible than the high-level DSL but requires much more coding and is more complicated to use.

In this book, we cover the declarative DSL of Kafka Streams.

Kafka Streams applications can employ parallelism to scale their performance. Since they build on top of Kafka consumers, you can start multiple instances of the same Kafka Streams application to form a consumer group and process data records in parallel. That is, the maximum degree of parallelism (or the number of application instances) is limited by the number of partitions of the Kafka topics that are consumed as input. Additionally, you can employ multithreading to parallelize the execution of some operations of a Kafka Streams application by setting the configuration option `num.stream.threads` to the desired number of threads.

Figure 2.8 visualizes the Kafka Streams application from Figure 2.7 when being deployed with two instances and consuming an input topic with three partitions.

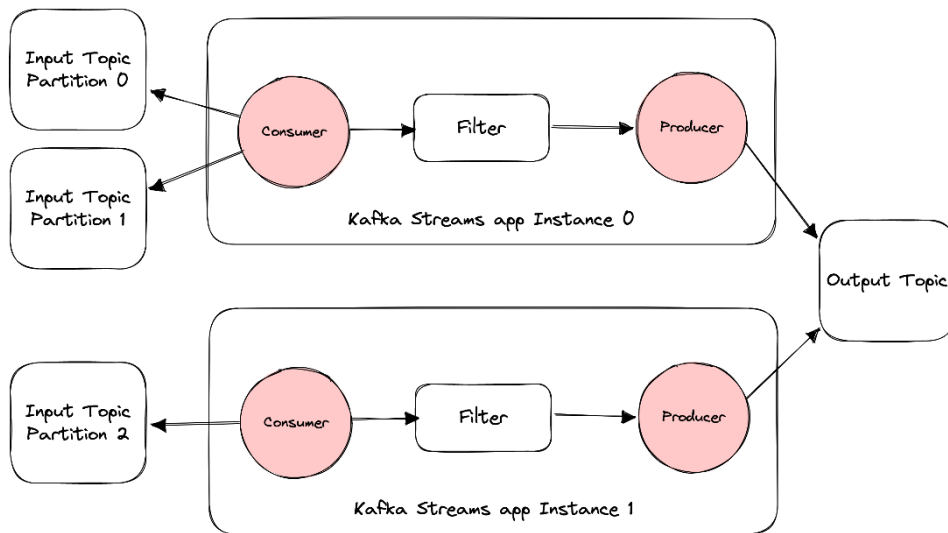


Figure 2.8 A Kafka Streams application with two instances processing records from an input topic with three partitions.

As mentioned above, one Kafka Streams application is always tied to one Kafka cluster. That is, it cannot stream data between two different Kafka clusters. Kafka Streams provides different abstractions for working with Kafka topics:

- **KStream:** A *KStream* represents a stream of Kafka records from one Kafka topic, in which records are not related. Records are processed independently from each other.
- **KTable:** A *KTable* represents a stream of Kafka records from one Kafka topic, where records with the same key are related to each other and represent updates. When using SQL terminology, the first occurrence of a specific key in a *KTable* represents an *INSERT* and the following occurrences of the same key represent *UPDATES*. *DELETES* are represented by records that have the value set to *null*. The *KTable* concept is very useful when working with a Kafka topic that resembles a changelog, e.g., a Kafka topic holding CDC events of a database table. It can be used to, for instance, perform lookups to get the most recent value of a specific key, which is very handy when joining multiple Kafka topics.

- **GlobalKTable:** When running a Kafka Streams application with multiple instances and reading a Kafka topic with the *KTable* concept, the *KTable* holds only data of the topic partitions that the Kafka Streams app is assigned to. The *GlobalKTable* concept is built on that of the *KTable* but always holds data from all topic partitions, regardless of the partition assignment of the application instance. By having all records (and their values) always available, using a *GlobalKTable* allows you to implement a use case, where you need access to all data, like join operators using the record values for the comparison. When using a *GlobalKTable*, each instance of a Kafka Streams application holds much more data than when using a *KTable*. Thus, it requires more memory and puts a higher load onto the network.

Given that streaming data pipelines operate on unbounded data sets, they must group data into windows using a notion of time when applying stateful operators that must process multiple records at a time, like aggregations or joins. Streaming architectures involve multiple components and are distributed by their nature. Single components can be scaled up, scaled down, started, or paused at any time. In many cases, streaming applications cannot rely on the wall-clock time but must use a notion of time that is independent from the actual time an event is eventually processed. To this end, Kafka Streams supports different notions of time, for instance:

- **Event time** equals the time when the record has been produced.
- **Ingestion time** equals the time when the record has been processed by the Kafka broker.
- **Processing time** equals the time when the record has been processed by the Kafka Streams app.

In Kafka Streams, you can define how these windows shall be computed. It supports different kinds of windows, for instance:

- **Tumbling time windows** group records into non-overlapping windows of equal size. They take a single configuration option, the size of the windows, e.g., five minutes. Data records can belong to only one window because windows do not overlap. Figure 2.9 visualizes how tumbling time windows would group the shown records into windows of 10 minutes each.

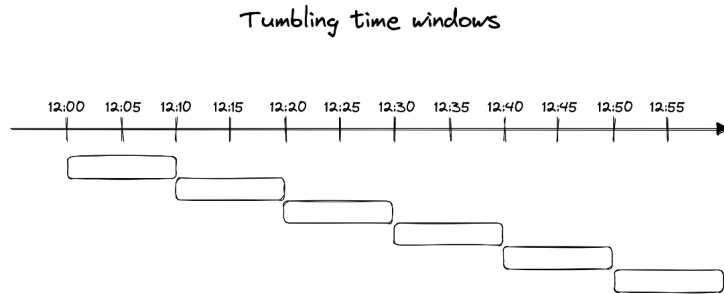


Figure 2.9 Tumbling time windows with a window size of 10 minutes.

- **Hopping time windows** group records into windows of equal size that can potentially overlap. They take two configuration options: the size of the windows, e.g., five minutes, and the advance interval (or the hop), e.g., 1 minute. The advance interval lets you define how much the window shall *move forward* compared to the previous one. Tumbling time windows can be implemented with a hopping time window that defines an advance interval that equals the window size. Figure 2.10 visualizes how tumbling time windows would group the shown records into windows of 10 minutes each with an advance interval of 5 minutes.



Figure 2.10 Hopping time windows with a window size of 10 minutes and an advance interval of 5 minutes.

Further window types are sliding windows and session windows. They are covered in the Kafka Streams documentation: <https://kafka.apache.org/35/javadoc/org/apache/kafka/streams/kstream/SlidingWindows.html> and <https://kafka.apache.org/35/javadoc/org/apache/kafka/streams/kstream/SessionWindows.html>

By default, Kafka Streams continuously updates the results of an operation and does not wait for the completion of a window, which is especially useful when working with the *KTable* concept. If your use case cannot cope with such intermediate results, you can configure that Kafka Streams shall *suppress* the emitting of results until the window has been completed.

We discuss developing stateful stream processors with Kafka Streams in more detail in Chapter 6.

2.3.1 Deployment options

Kafka Streams applications are regular Java or Scala applications that can be built with tools, such as Maven, and distributed as JARs. We recommend packaging them as container images and deploying them on a managed container service (e.g., Amazon Elastic Container Service or Google Cloud Run) or Kubernetes. We discuss packaging and deploying Kafka Streams applications in depth in Chapter 7.

2.3.2 Other technologies

Kafka Streams is the stream processing framework natively shipped with Apache Kafka. Besides Kafka Streams, there exist a couple of other technologies for building stateful streaming applications.

KSQldb

ksqldb introduces a SQL-based interface on top of Kafka Streams and Kafka Connect, which lets you build complete streaming applications with SQL statements. It ships its own orchestrator, which takes care of running ksqldb queries.

APACHE FLINK

Apache Flink is an open-source framework for batch and stream processing. Similar to Kafka Streams, it provides powerful means for the stateful processing of data streams. Besides Kafka, it supports other data sources and data sinks, such as Amazon S3 or PostgreSQL, as well and allows you to implement complete streaming data pipelines with a single Flink application. Flink applications are rather heavy in terms of resource consumption and operations. Flink ships its own orchestrator and is not a simple client application, like Kafka Streams. Flink might still be a viable choice if you do not need the native Kafka integration from Kafka Streams.

2.4 Summary

- Kafka is a distributed, open-source event broker that lets you read and write event data at scale. It is designed to satisfy the available network bandwidth and can scale its performance by running multiple brokers on different physical machines.
- Kafka organizes records in topics, which are partitioned for scaling the performance of operations and can be replicated for fault tolerance. Records consist of a key, a value, headers, and a timestamp. Keys are used for partitioning. Kafka guarantees the ordering of records by their occurrence within partitions.
- Kafka consumers can organize multiple consumers in a consumer group for scaling their performance.
- Kafka Connect is a scalable connector tool on top of the consumer and producer API of Kafka. It lets you integrate Kafka topics with external data systems.
- Kafka Streams is a stream processing library on top of the consumer and producer API of Kafka. Kafka Streams provides a declarative, functional DSL for building stream processing applications that apply stateful operators, e.g., aggregations, or joins, to windows of events with a few lines of code.
- Kafka Streams supports different notions of time that can be used to assign events to windows. Event time is the time when the event has occurred in the data source system, e.g., the transactional database consumed by a streaming data pipeline. Ingestion time is the time when the event has been written to Kafka. Processing time is the time when the event has been processed by Kafka Streams.
- Kafka Streams provides different types of windows for grouping events. Tumbling time windows group records into non-overlapping, temporal windows of equal size. Hopping time windows group records into potentially overlapping, temporal windows of equal size.

3

Integrating data systems in real-time with Kafka Connect

This chapter covers

- Extracting data from database systems with change data capture
- Turning data systems at rest into streams of change events
- Publishing change events to external data systems
- Transforming data with Kafka Connect

In the last chapter, we looked at Apache Kafka and its ecosystem from an architectural point of view and learned how one can combine Kafka, Kafka Connect, and Kafka Streams to build powerful streaming data pipelines. We walked through a few practical examples to understand how to use the low-level consumer and producer APIs for writing and reading data with Kafka.

Kafka Connect is a connector framework on top of Kafka's consumer and producer API. It plays an essential role in the Kafka ecosystem by acting as the entry and exit points of streaming architectures: *Source connectors* extract change events from external systems, like databases, and produce them to Kafka topics, while *sink connectors* consume events from Kafka topics and publish them to external systems, like data warehouses.

The community around Apache Kafka has developed and published hundreds of open-source connectors that can be used for integrating Kafka with other systems. There are connectors for database systems, cloud data warehouses, APIs, cloud object stores, etc. If you are tasked with connecting Kafka to a specific data system, it is very likely that you can reuse an existing connector. For getting up and running, all you need to do is to load the connector's JAR into the classpath of your Kafka Connect cluster and instantiate the connector with an API call.

Most source connectors implement *change data capture (CDC)*, a technique for identifying and extracting change events from data systems. CDC is a foundational design pattern in streaming data pipelines and enables them to process continuous streams of data changes instead of performing recurring loads of entire data sets.

Change data capture can be implemented using different techniques:

- **Replication logs:** Most database systems maintain their transaction logs as files on disk, which enables the recovery of the managed databases in case of failures. Transaction logs consist of the executed statements or changed rows. CDC connectors can leverage these logs for capturing and extracting change events. Transaction logs are typically maintained in real-time within the scope of the executed transaction. Accessing them does not create load on the database system, because it bypasses its query layer and is comparable to reading files from disk.
- **Triggers:** Some database systems provide trigger functions as a means to perform user-defined actions once events, like insertions of data, occur. For implementing CDC, the user-defined action could write a changelog record to a separate table. While the trigger-based approach supports the extraction of all kinds of change events, it has a negative impact on the performance because the trigger function is being executed before the statement finishes, often greatly increasing the execution time. It also requires the recurring polling of the changelog table, which puts further load onto the database system.

- Timestamp columns:** Some datasets specify a timestamp column that provides the time when a record has been most recently modified. CDC connectors can recurringly query that column for extracting the rows that have been changed recently, using a query similar to `SELECT * FROM table_name WHERE updated_at > TIMESTAMP_OF_LAST_RUN`. While database indexes might mitigate performance issues, this approach still needs to perform regular database queries for data extraction and can put significant load onto the database. The timestamp-based CDC implementation supports the extraction of insertions and updates but cannot detect deletions (soft deletions, which keep deleted rows in the database but mark them as deleted via a dedicated column, could help here). Another downside is that they cannot work with any data schema but require the presence of a timestamp column. Last but not least, timestamp-based CDC approaches miss events if multiple updates of the same row occur between two polls. Many data systems, like APIs, provide neither replication logs nor trigger functions and require the usage of timestamp fields for the implementation of CDC.

When comparing the different approaches to implementing CDC, replication logs are typically the preferred technique because they provide high performance and allow connectors to capture all kinds of change events, i.e., insertions, updates, and deletions.

Let's get ready to take the next steps. In this and the subsequent chapters, we will help the data team of the e-commerce business *Excellent Toys* to migrate their legacy batch data pipelines to a streaming approach, using Kafka, Kafka Connect, and Kafka Streams. [Section 3.1](#) introduces the case study and helps you prepare your local machine for the practical examples.

In [Section 3.2](#), we'll take a closer look at Debezium, one of the most popular implementations of log-based CDC for database systems. The case study from Excellent Toys will provide us with a practical scenario of streaming changes from PostgreSQL to Kafka topics.

When building streaming data pipelines with Kafka Connect and Kafka Streams, you have two options for implementing stateless processing logic: Kafka Streams offers a powerful DSL for developing stream processors, while Kafka Connect provides *single message transforms (SMT)* as a lightweight means of implementing stateless processing logic. In [Section 3.3](#), we cover when to use SMTs for processing data and when to avoid them.

The implementation of sink connectors is typically simpler than the one of source connectors. They consume records from a Kafka topic and publish them record by record or in batches to external data systems. We discuss sink connectors in detail in [Section 3.4](#) and use them to implement the integration between Kafka and ClickHouse.

This chapter focuses on working with existing connectors from the open-source community. We cover developing custom Kafka Connect connectors in the next chapter.

3.1 Meet our case study: Building a streaming data pipeline for an e-commerce business

Throughout the rest of the book, we will take practical examples of streaming data with Kafka from the operations of the data team at Excellent Toys, an imaginary e-commerce business that sells vintage toys. Excellent Toys employs around 100 people including a five-person data team. Excellent Toys runs their highly-frequented online shop on top of the database system PostgreSQL. The leadership team started gaining interest in their operational data and set out to support their decision-making with the latest intelligence about sales, customers, and marketing. They tasked the data team with introducing a business intelligence (BI) solution and building dashboards.

The data team chose ClickHouse as the data warehouse, which perfectly caters to the analytical workloads executed by the BI solution. They integrated PostgreSQL with ClickHouse by implementing a batch data pipeline run as a CRON job: Each night at 2 AM, the batch pipeline (1) extracts all data from PostgreSQL, (2) cleans up the data, and (3) ingests the processed data into ClickHouse. Figure 3.1 illustrates the different components of the pipeline.

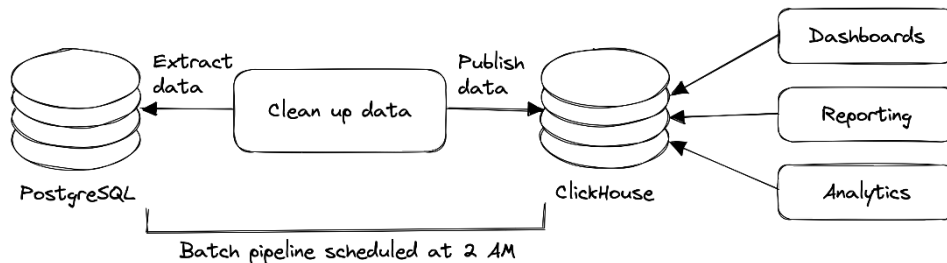


Figure 3.1 Excellent Toy's batch pipeline that replicates data from PostgreSQL to ClickHouse at fixed times.

The PostgreSQL database of Excellent Toys holds hundreds of GBs of data from many years of being in business. After a few months of the data pipeline being in production, frustration started to grow inside the organization and it became apparent that the batch-based data architecture fails to meet the expectations of the business:

- Business teams cannot support decisions with current information but are almost always working with outdated data from the last business day, which leads to missed business opportunities.
- The batch pipeline is too heavy and puts a lot of load on the PostgreSQL database because it uses a `SELECT * FROM` query to extract all data. To avoid impacting the customer experience, the pipeline must be executed outside of business hours. Debugging production issues becomes hard and rollouts of bug fixes must wait until the next execution.

- A single run of the batch pipeline takes multiple hours because it needs to process all data from the PostgreSQL database. On some days, the batch pipeline fails in the middle of the process, leaving ClickHouse in a corrupted state until the next execution of the pipeline. On other days, the pipeline finishes its execution after the start of the business day.

The data team has researched alternatives to batch processing. They stumbled upon streaming data pipelines and advanced techniques for data extraction, like change data capture. These modern approaches promise the real-time processing of data, the continuous replication of incremental updates from data sources to data sinks, and a much higher tolerance for failures. The data team convinced the leadership team to allocate a small budget for exploring a potential migration of their batch data pipelines to a streaming approach, which could potentially unlock much more value for their business. Let us go on this journey together throughout the rest of this book!

We provide a Docker Compose configuration for setting up all components needed for working on the examples provided in this book. Please download the configuration from the GitHub repository of the book (<https://github.com/flippingbits/streaming-data-pipelines-book>) and start the containers, as shown in Listing 3.1:

Listing 3.1 Downloading the Docker Compose configuration and starting the containers.

```
$ mkdir streaming-data-pipelines-book && cd streaming-data-pipelines-book
$ wget https://raw.githubusercontent.com/flippingbits/streaming-data-pipelines-book/main/docker-compose.yml
$ docker-compose up
```

NOTE The `docker-compose.yml` binds the Docker volume named `streaming-data-pipelines-book-kafka-data` to the Kafka container, so that you can persist topic data, including Kafka Connect connector configurations, across restarts of the container. You can reset your development environment by deleting the volume with the command `$ docker volume rm streaming-data-pipelines-book-kafka-data`.

3.2 Capturing changes from transactional databases with Debezium

Debezium is one of the most popular open-source projects from the Kafka ecosystem. It implements log-based change data capture for various database systems^[1], like MySQL, PostgreSQL, and MongoDB, and can be deployed as a Kafka Connect source connector. Debezium is maintained and sponsored by the company RedHat.

In this section, we use Debezium and Kafka Connect to capture change events from a PostgreSQL database; however, most concepts can be easily transferred to other database systems.

3.2.1 Debezium in action

Kafka Connect provides an API endpoint that lists all connectors of the given Connect cluster. Let's call it, as shown in Listing 3.2, and verify that our Kafka Connect cluster does not yet run any connectors:

Listing 3.2 Retrieving all Kafka Connect connectors.

```
$ curl http://localhost:8083/connectors  
> []%
```

Let's have a look at how Debezium works in action. With the curl command from Listing 3.3, we can instantiate a new Debezium source connector for the PostgreSQL database from our case study. We call the connector *online_shop*, point it to the PostgreSQL server that we deployed with Docker Compose, and choose to capture all events from the database called *postgres*.

Listing 3.3 Instantiating a Debezium source connector for the PostgreSQL database from our case study.

```
$ curl -XPOST \
-H'Content-Type:application/json' \
http://localhost:8083/connectors \
--data-raw '{
  "name": "online_shop", #1
  "config": { #2
    "connector.class":
      "io.debezium.connector.postgresql.PostgresConnector", #3
    "database.hostname": "postgres", #4
    "database.port": "5432", #5
    "database.user": "postgres", #6
    "database.password": "postgres", #7
    "database.dbname" : "postgres", #8
    "topic.prefix": "book" #9
  }
}'
```

#1 Set the connector name to *online_shop*.

#2 Configuration of the connector.

#3 Choose the PostgreSQL source connector from Debezium by passing its classname to `connector.class`.

#4 Hostname (or IP address) of the PostgreSQL server.

#5 Port of the PostgreSQL server.

#6 Username for connecting to the PostgreSQL server.

#7 Password for connecting to the PostgreSQL server.

#8 Name of the PostgreSQL database. Please note that we do not restrict Debezium to certain tables but replicate data from the entire database.

#9 Prefix for names of the topics, which helps to separate different Debezium connectors consuming data from the same database.

When listing the connectors again (see Listing [3.4](#)), you should now see that the Kafka Connect cluster features one connector with the name *online_shop*:

Listing 3.4 Retrieving all Kafka Connect connectors again.

```
$ curl http://localhost:8083/connectors
> ["online_shop"]%
```

Listing [3.5](#) shows how to use Kafka Connect's API for retrieving the config of a connector:

Listing 3.5 Retrieving the config of a Kafka Connect connector.

```
$ curl http://localhost:8083/connectors/online_shop/config
> {
  "connector.class":"io.debezium.connector.postgresql.PostgresConnector",
  "database.user":"postgres",
  "database.dbname":"postgres",
  "topic.prefix":"book",
  "database.hostname":"postgres",
  "database.password":"postgres",
  "database.port":"5432"
}%
```

The connector should now be up and running. You can check on its health by calling the API of Kafka Connect, as shown in [Listing 3.6](#):

Listing 3.6 Retrieving the health status of a Kafka Connect connector.

```
$ curl http://localhost:8083/connectors/online_shop/status
> {
  "name":"online_shop",
  "connector": {
    "state":"RUNNING",
    "worker_id":"172.18.0.5:8083"
  },
  "tasks": [
    {
      "id":0,
      "state":"RUNNING",
      "worker_id":"172.18.0.5:8083"
    }
  ],
  "type":"source"
}%
```

In the case of an error, the connector would have thrown an exception; the `/connectors/:name/status` endpoint would then report the *state* as `FAILED` and include the stacktrace of the exception in the response body. The execution of failed connectors is halted until a (manually executed) restart occurs.

If you want to dive deeper into the status and progress of your connector, you could view and follow the logs of the Docker container, as shown in [Listing 3.7](#):

Listing 3.7 Viewing the logs of the Kafka Connect container.

```
$ docker logs -f connect
```

When starting up, the source connector creates the Kafka topics, extracts all existing data from the consumed database by taking a snapshot, and produces the events from the snapshot to the Kafka topics. Feel free to follow Listing [3.8](#) and start a consumer on the console to take a look at the records produced to the Kafka topic `book.online_shop.orders`:

Listing 3.8 Consuming records from the topic `book.online_shop.orders`.

```
$ docker exec kafka \
  /bin/kafka-console-consumer \
  --bootstrap-server=kafka:9092 \
  --topic=book.online_shop.orders \
  --from-beginning \
  --property print.key=true
```

You should now see a bunch of records that have been extracted from the table of the PostgreSQL database. Let us keep the console consumer open and start `psql` in a second terminal, as shown in Listing [3.9](#). When `psql` asks you for the password, enter `postgres`.

Listing 3.9 Using the tool `psql` for working with the PostgreSQL instance.

```
$ docker exec -it postgres psql -hpostgres -p5432 -Upostgres -W
```

If you now enter a few statements for inserting, updating, and deleting rows, as shown in Listing [3.10](#), you should see the corresponding CDC events in the console consumer.

Listing 3.10 Inserting, updating, and deleting rows via `psql`.

```
INSERT INTO online_shop.orders
  VALUES (default, '2023-07-01', 1001, 5, 101);

INSERT INTO online_shop.orders
  VALUES (default, '2023-07-01', 1001, 3, 102);

UPDATE online_shop.orders
  SET quantity = 10 WHERE customer_id=1001 and product_id=101;

DELETE FROM online_shop.orders WHERE customer_id=1003;
```

Congratulations! You have successfully set up a CDC connector for a PostgreSQL database that captures change events in real-time.

3.2.2 Format of change events

Debezium produces one change event for each row that has been affected by a `INSERT`, `UPDATE`, or `DELETE` statement. For instance, assuming an `UPDATE` statement that changes four rows, Debezium would produce four records to the associated Kafka topic.

Debezium converts the change events sent by the logical decoding plugin to its own format before producing them to Kafka. What does the Debezium format look like in detail? Let us discuss the format of record keys and record values separately. We use the following table schema shown in Listing [3.11](#) for the discussion:

Listing 3.11 Schema of a PostgreSQL table called orders.

```
CREATE TABLE orders (  
  id          SERIAL NOT NULL PRIMARY KEY,  
  order_date  DATE    NOT NULL,  
  purchaser   INTEGER NOT NULL,  
  quantity    INTEGER NOT NULL,  
  product_id  INTEGER NOT NULL  
);
```

RECORD KEYS

Debezium uses record keys that contain the column of the primary key of the consumed table. If the table specifies multiple primary keys, all columns in the primary key are included. The data format and the inclusion of schemas can be configured via Kafka Connect's converters. By default, Debezium produces records in the JSON format and includes the schema of the produced data. When streaming change events for the row with the primary key `id=42`, Debezium would create a record key as shown in Listing [3.12](#):

Listing 3.12 Example record key in the JSON format.

```
{
  "schema": { #1
    "type": "struct", #2
    "name": "book.online_shop.orders.Key", #3
    "optional": false, #4
    "fields": [ #5
      {
        "name": "id", #6
        "index": "0", #7
        "schema": {
          "type": "INT32", #8
          "optional": "false" #9
        }
      }
    ]
  },
  "payload": { #10
    "id": 42 #11
  },
}
```

#1 The schema of the payload.

#2 The type of the payload.

#3 The name of the schema of the payload.

#4 Whether the payload is optional or not.

#5 Fields of the schema of the payload.

#6 Name of the first field of the payload, here the name of the primary key, *id*.

#7 Index of the field in the list of fields.

#8 Data type of the field *id*, here *INT32*.

#9 Whether the field *id* is optional or not.

#10 The payload.

#11 The primary key column of the consumed table, here instantiated with the value 42.

RECORD VALUES

Similar to the record keys, record values hold a `schema` and a `payload`. Compared to the record key, however, the `payload` of record values is much more complex and holds a so-called *envelope*. The envelope consists of five main fields:

- The field **before** holds the state of the row before changing its state. In the case of an insertion, *before* is set to `null`.
- The field **after** holds the state of the row after changing its state. In the case of a deletion, *after* is set to `null`.

- The field **source** provides information about the source of the CDC event. For instance, it provides the Debezium version, the type and the name of the source connector, the transaction id, and further information.
- The field **op** holds the type of the operation that changed the state of the row. It can be one of `c` (create or insertion), `u` (update), `d` (deletion), `r` (read; reflects an event produced by a snapshot), `t` (truncate), and `m` (message).
- The field **ts_ms** holds the time when the connector has processed the CDC event.

Listing [3.13](#) shows an example of a record value for a CDC event that describes the statement `INSERT INTO orders (id, order_date, customer_id, quantity, product_id) VALUES (10005, 19539, 1001, 5, 101)`. The schema has been omitted to improve readability:

Listing 3.13 Example record value in JSON format.

```
{
  "schema": { ... },
  "payload": {
    "before": null, #1
    "after": { #2
      "id": 10005,
      "order_date": 19539,
      "customer_id": 1001,
      "quantity": 5,
      "product_id": 101
    },
    "source": { #3
      "version": "2.3.0.Final",
      "connector": "postgresql", #4
      "name": "online_shop", #5
      "ts_ms": 1690408108817, #6
      "snapshot": "false", #7
      "db": "postgres",
      "sequence": "[null,\"23406704\"]", #8
      "schema": "online_shop",
      "table": "orders",
      "txId": 761, #9
      "lsn": 23406704, #10
      "xmin": null
    },
    "op": "c", #11
    "ts_ms": 1690408109203 #12
  },
}
```

#1 The state of the row before applying the operation (here null because it is an insertion).

#2 The state of the row after applying the operation.

#3 Information about the source of the CDC event.

#4 Type of the Debezium connector, here postgresql.

#5 Name of the Kafka Connect connector.

#6 Timestamp when the operation has been executed in the database.

#7 Whether the event has been extracted in a snapshot procedure.

#8 The first value is the LSN that has been most recently committed by the connector. The second value is the current LSN.

#9 ID of the database transaction.

#10 The LSN of the change event.

#11 Type of the operation, here c (create or insertion).

#12 Timestamp when the event has been processed by Debezium.

3.2.3 Logical decoding and replication slots in PostgreSQL

PostgreSQL writes all database transactions in the order of their execution to its *transaction log*, named write-ahead log (WAL). PostgreSQL employs the WAL as a foundational concept for storing data reliably and preventing data loss, even in the case of power loss, hardware failures, or other malfunctions. The WAL describes changes at the storage level; its data format is not easily readable.

Since version 9.4, PostgreSQL provides logical decoding as a means of converting the low-level WAL format into a human-readable format, e.g., statements or rows. In the beginning, logical decoding had to be implemented by a PostgreSQL extension, in this case often referred to as an output plugin. For instance, the Debezium community has released the output plugin `decoderbufs` that converts the WAL format into Protocol Buffers (read more on <https://github.com/debezium/postgres-decoderbufs>). Since version 10, PostgreSQL ships with a native implementation of logical decoding, `pgoutput`, and does not require the installation of an extension.

Technically, users subscribe to the changes of a PostgreSQL cluster by creating *replication slots* and consuming them. Each replication slot has a unique identifier and allows the consumer to read the WAL in the format of the output plugin. The logical decoding plugin sends new changes to the consumer once they have been committed and written to the WAL. Figure 3.2 shows the relationships between the WAL, the output plugins, the replication slots, and the consumers.

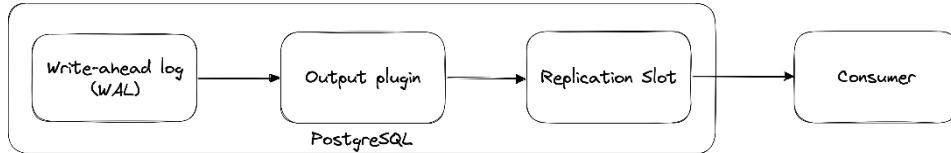


Figure 3.2 Interaction of the write-ahead log, the output plugins (e.g., `pgoutput`), the replication slots, and the consumers.

PostgreSQL allows consumers to define the log sequence number (LSN), i.e., the position, from which they want to read the WAL. It is possible to have one PostgreSQL cluster with multiple replication slots that read from different LSNs, which could be beneficial if you employ multiple consumers that process data independently from each other.

NOTE You can create and consume replication slots only on the *primary servers* of a PostgreSQL cluster. If your PostgreSQL cluster is distributed across multiple servers, please make sure that you work with the primary server.

3.2.4 Streaming record-level change events

Debezium makes use of PostgreSQL's logical replication feature for capturing row-level insertions, updates, and deletions. It requires its own replication slot and consumes all change events sent by the logical decoding plugin. Internally, the Debezium connector remembers the last LSN that it has consumed and persists that information in the offset topic of Kafka Connect (if you use our Docker Compose configuration, that topic is called `my_connect_offsets`). When being restarted, Debezium can resume the streaming of change events at the persisted LSN, which avoids the reprocessing of data and the production of duplicate events.

Debezium converts change events from the logical decoding plugin into its own format, which contains the state of the row before the change, the state of the row after the change, and some metadata, and produces them to a Kafka topic. The name of the Kafka topic is determined by the connector-specific configuration option, `topic.prefix`, the name of the PostgreSQL schema, and the name of the PostgreSQL table; it follows the pattern `topicPrefix.schemaName.tableName`. For instance, when setting up a connector with the config `topic.prefix=book` that consumes the table `orders` from the schema `online_shop`, Debezium would produce data to the topic `book.online_shop.orders`.

By default, Debezium produces records with a record key that contains the database row's primary key, which ensures that change events associated with the same database row always end up in the same topic partition. The logical decoding plugin sends the events to Debezium in the order of their occurrence. Given that Kafka guarantees the ordering of records by the time they were produced at the partition level, change events are ordered in Kafka by their occurrence.

One connector instance can use one replication slot to stream change events from multiple database tables or schemas at the same time. Debezium defines the tables and schemas of interest by creating a *publication* in PostgreSQL. Debezium consumes the data of the publication by instantiating a *subscriber*. By default, Debezium captures changes from all non-system schemas and tables from PostgreSQL. You can restrict the streaming of change events to specific schemas and tables via the configuration of the connector. Figure 3.3 shows a Debezium connector that streams changes from the PostgreSQL tables `orders` and `visitors` to Kafka.

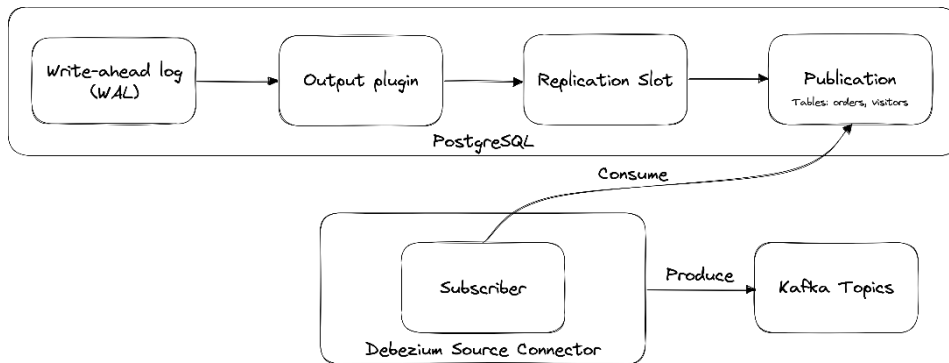


Figure 3.3 A Debezium connector using logical replication to stream change events from the tables orders and visitors to Kafka.

3.2.5 Snapshots

PostgreSQL does not store the complete history of the database in the WAL for efficiency reasons; if it did, the WAL would require a tremendous amount of storage, much more than the actual database. As a consequence, the WAL can only be used for streaming current or recent changes.

Debezium aims at replicating entire databases. When a connector is started for the first time, Debezium takes a consistent *snapshot* of the consumed database. Debezium implements the snapshot as follows:

1. Start a new transaction to ensure that subsequent reads are performed against a consistent view of the consumed database.
2. Take note of the current LSN in the transaction log of the database.
3. Extract all data from the consumed database, using a `SELECT * FROM table` query, and generate one event for each row.
4. Commit the transaction that has been started in the first step.
5. Store the completion of the snapshot in the offset of the connector

After completing the snapshot, Debezium starts streaming change events from the transaction log at the LSN noted in the second step of the snapshot procedure.

Snapshots are typically only executed when starting a connector for the first time. There might be cases where you need to take a snapshot again after running the connector for some time, e.g., because you need to reprocess all data due to a fundamental update of the downstream stream processor. In such a case you can, for instance, reset the offset of the connector stored in the Kafka topic holding the Kafka Connect offsets and restart the connector to force the connector instance to fetch the updated offset. Let us have a look at the records of the topic `my_connect_offsets`, as shown in Listing [3.14](#):

Listing 3.14 Investigating the records from the topic `my_connect_offsets`.

```
$ docker exec kafka \
  /bin/kafka-console-consumer \
  --bootstrap-server=kafka:9092 \
  --topic=my_connect_offsets \
  --from-beginning \
  --property print.key=true
>
["online_shop",{"server":"book"}]      {"last_snapshot_record":true,
➤ "lsn":23409160,"txId":764,"ts_usec":1690410175959714,"snapshot":true}
["online_shop",{"server":"book"}]      {"transaction_id":null,
➤ "lsn_proc":23410376,"messageType":"INSERT","lsn":23410376,
➤ "txId":766,"ts_usec":1690410503239635}
["online_shop",{"server":"book"}]      {"transaction_id":null,
➤ "lsn_proc":23411776,"messageType":"INSERT","lsn_commit":23411720,
➤ "lsn":23411776,"txId":767,"ts_usec":1690410588747841}
```

Debezium stores the name of the connector and the value of the configuration option `topic.prefix` in the key of the record, in our case set to `["online_shop", {"server":"book"}]`. Debezium stores information about the most recent position of the LSN that it has successfully processed in the value of the record. The most recent Kafka record reflects the current offset, here the record with `lsn` set to 23411776.

NOTE In Kafka, tombstone records are records that have their value set to `null`. They indicate the logical deletion of the data for the associated key.

Listing 3.15 walks you through resetting the offset of our Debezium connector. First, we produce a tombstone record with the key `["online_shop", {"server":"book"}]` and an empty value to the topic `my_connect_offsets` to delete the offset information of our connector. Unfortunately, the CLI script shipped with Apache Kafka, `kafka-console-producer.sh`, does not support producing records with the value set to `null`. In this case, we use the open-source CLI tool `kcat` (<https://github.com/edenhill/kcat>). Second, we restart the connector to force the connector to read the offset from the Kafka topic. If you want to follow the impact and process of the offset reset, you could watch the logs of the Docker container and consume the Kafka topics filled by the connector.

Listing 3.15 Producing a new record to the Kafka topic `my_connect_offsets` and restarting the connector.

```
$ echo '["online_shop",{"server":"book"}]'. ' | \ #1
  docker run -i --network=streaming-data-pipelines-book \
  edenhill/kcat:1.7.1 \
  -b kafka:29092 \
  -t my_connect_offsets -Z -K.
$ curl -XPOST \
  "http://localhost:8083/connectors/online_shop/restart?includeTasks=true" #2
```

#1 Produce a tombstone record to the topic `my_connect_offsets`.

#2 Restart the connector by calling the Kafka Connect API.

You can configure the snapshot behavior of Debezium via its configuration property `snapshot.mode`. By default, Debezium takes a snapshot only when no offset exists, i.e., when the connector is being started for the first time (`snapshot.mode = initial`). You can, for instance, configure that the connector shall take a snapshot every time it starts (`snapshot.mode = always`) or that the connector should never take a snapshot but solely stream change events from the WAL (`snapshot.mode = never`).

Please see the Debezium documentation for more information about snapshots: <http://mng.bz/M952>

3.2.6 Configuring Debezium

You can configure Debezium when creating (or updating) the connector using Kafka Connect's API. In the following, we discuss the most important configuration options of Debezium's PostgreSQL connector. For the full documentation of the configuration properties, please visit the official Debezium website: <http://mng.bz/yZvJ>

PLUGIN.NAME

The name of the PostgreSQL logical decoding plugin. You can choose between `pgoutput` (recommended for PostgreSQL 10 or newer) and `decoderbufs` (recommended for PostgreSQL 9.4 or newer; requires installing a PostgreSQL extension) (default: `decoderbufs`).

SLOT.NAME

The name of the replication slot that is being used for logical replication (default: `debezium`).

DATABASE.HOSTNAME

The hostname of the PostgreSQL server.

DATABASE.PORT

The port of the PostgreSQL server (default: 5432).

DATABASE.USERNAME

The name of the PostgreSQL user for connecting to the PostgreSQL server.

DATABASE.PASSWORD

The password of the PostgreSQL user for connecting to the PostgreSQL server.

DATABASE.DBNAME

The name of the PostgreSQL database.

TOPIC.PREFIX

The prefix to use for topic names.

SCHEMA.INCLUDE.LIST

If you want to restrict the capturing of events to specific PostgreSQL schemas, you can pass them to Debezium as a comma-separated list using the configuration property `schema.include.list`. Similarly, you can define the schemas that you want to exclude via the property `schema.exclude.list`. You should not set both options at the same time.

TABLE.INCLUDE.LIST

If you want to restrict the capturing of events to specific PostgreSQL tables you can pass them to Debezium as a comma-separated list using the configuration property `table.include.list`. Similarly, you can define the tables that you want to exclude via the property `table.exclude.list`. You should not set both options at the same time. In both cases, you must provide the tables in the format *schemaName.tableName*.

COLUMN.INCLUDE.LIST

If you want to include only specific columns in the CDC events, you can pass them to Debezium as a comma-separated list using the configuration property `column.include.list`. Similarly, you can define the columns that you want to exclude from the CDC events via the property `column.exclude.list`. You should not set both options at the same time. In both cases, you must provide the columns in the format *schemaName.tableName.columnName*.

SNAPSHOT.SELECT.STATEMENT.OVERRIDES

By default, Debezium executes a `SELECT * FROM schemaName.tableName` query for taking the initial snapshot. In the case of very large tables, this might become problematic because the snapshot could take hours to days to complete. You can pass the tables, for which you want to provide a custom snapshot query, via the configuration property `snapshot.select.statement.overrides` and then define the custom query in a second key-value mapping. Let us assume that you want to execute the query `SELECT * FROM online_shop.orders WHERE quantity > 5` to take a snapshot of the table `orders` in the schema `online_shop`. You could do this by setting the configuration property `snapshot.select.statement.overrides` to `online_shop.orders` and setting the property `snapshot.select.statement.overrides.online_shop.orders` to `SELECT * FROM [online_shop].[orders] WHERE quantity > 5`.

3.3 Single message transforms in Kafka Connect: When to use and when to avoid

With the current setup, we are streaming CDC events with a lot of information about the before and after state, the source of the event, etc. Given that Excellent Toy's downstream use cases only require the state of the row after applying the change event, their data team decides to get rid of all other information to save space and reduce the structural complexity of the change events.

While they could develop a custom stream processing application using Kafka Streams, they choose to explore Kafka Connect's single message transforms (SMT) for implementing this task. Fortunately, Debezium provides a SMT, called *ExtractNewRecordState*, which flattens CDC events and reduces them to their field after. In a nutshell, the payload from Listing [3.13](#) would be converted to the one shown in Listing [3.16](#).

Listing 3.16 Payload of the CDC event after applying the SMT *ExtractNewRecordState*.

```
{
  "schema": { ... },
  "payload": {
    "id": 10005,
    "order_date": 19539,
    "customer_id": 1001,
    "quantity": 5,
    "product_id": 101
  }
}
```


Let's add the SMT *ExtractNewRecordState*, which is provided by the Debezium project, to the configuration of our connector. To this end, we need to update the connector configuration, as shown in [3.17](#):

Listing 3.17 Adding the SMT *ExtractNewRecordState* to the connector configuration.

```
$ curl -XPUT \
-H'Content-Type:application/json' \
http://localhost:8083/connectors/online_shop/config \
--data-raw '{
  "connector.class": "io.debezium.connector.postgresql.PostgresConnector",
  "database.hostname": "postgres",
  "database.port": "5432",
  "database.user": "postgres",
  "database.password": "postgres",
  "database.dbname" : "postgres",
  "topic.prefix": "book",
  "transforms": "unwrap", #1
  "transforms.unwrap.type":
  ➔ "io.debezium.transforms.ExtractNewRecordState" #2
}'
```

#1 Comma-separated list of the names of the SMT transforms that shall be applied to the records in the provided order.

#2 Configuration of the SMT named `unwrap`.

Let us watch the SMT in action. Please insert a new order into the PostgreSQL database and then start the console consumer, as shown in [Listing 3.18](#). You should now see CDC events with a much simpler schema.

Listing 3.18 Inserting a new row into the PostgreSQL database and consuming the Kafka topic.

```
$ docker exec -it postgres psql -hpostgres -p5432 -Upostgres -W \
-c "INSERT INTO online_shop.orders VALUES (default,
➔ '2023-07-03', 1003, 1, 101);"

$ docker exec kafka \
  /bin/kafka-console-consumer \
  --bootstrap-server=kafka:9092 \
  --topic=book.online_shop.orders \
  --from-beginning \
  --property print.key=true
```

Kafka Connect's single message transforms allow extending connectors with processing capabilities. Whether it's the flattening of a CDC event, the masking of string values, or the rerouting of records - Kafka Connect and the wider open-source community provide a set of ready-to-use SMTs that can be used to implement lightweight transformations without setting up a separate streaming application. In the case of flattening Debezium's CDC events, it would probably be too much overhead to set up a Kafka Streams application.

The simplicity of single message transforms comes at the cost of poor developer experience. SMTs are rather hard to debug and remain a black box once they are deployed. It is hard to understand their business logic without analyzing the underlying source code. The development and deployment of SMTs is not a simple task. You need to build a full-fledged Java project, package the compiled source code as a JAR, and load the JAR into the classpath of your Kafka Connect installation.

3.4 Streaming records to data sinks

So far, we have looked at source connectors and single message transforms. We used Debezium for streaming changes from PostgreSQL to Kafka. Because the CDC events generated by Debezium are rather complex and contain a lot of information, we simplified their structure with an SMT. We do not want to leave the data in Kafka but stream them to downstream applications and consumers. Kafka Connect offers sink connectors as a means for streaming data from Kafka to other systems. Figure 3.4 visualizes how they integrate topics with external data systems:

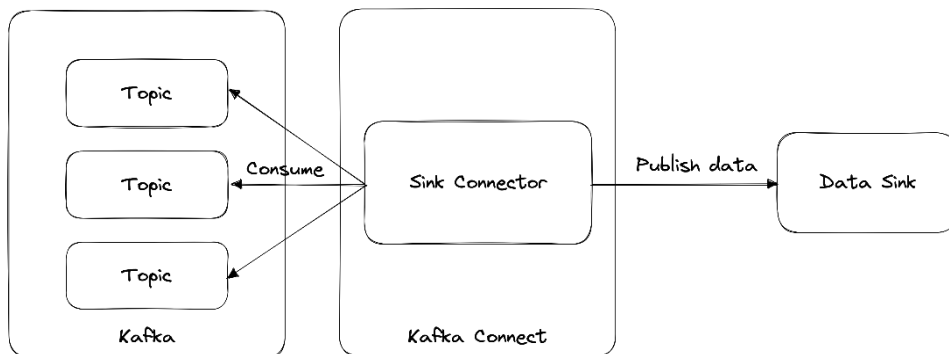


Figure 3.4 A Kafka Connect sink connector that integrates three Kafka topics with an external data sink.

In this section, we will dive deeper into sink connectors and learn how to use them for streaming records to downstream applications. While Kafka is great at persisting events in a scalable, fault-tolerant, and robust manner and offers the consumer API for reading data from topic partitions, its sequential data access pattern is not suited for analytical workloads. As a consequence, the data team of Excellent Toys decides to stream the CDC events stored in Kafka to the data warehouse ClickHouse.

ClickHouse is an open-source data warehouse that caters very well to analytical workloads. It is built for handling even complicated SQL queries over very large data sets with great performance and excels at serving data to reporting tools, dashboards, and BI solutions.

The Docker Compose file from our case study defines a service for ClickHouse - you should already have it up and running. ClickHouse offers an official sink connector for Kafka Connect, which you can download on their GitHub page (<https://github.com/ClickHouse/clickhouse-kafka-connect/releases>). We placed the JAR from the download in the folder `/kafka/connect/clickhouse` of the Kafka Connect container image.

Before we can stream data to ClickHouse, we must create the according table structure. Please start a ClickHouse client by executing `docker exec -it clickhouse clickhouse-client -d clickhouse` and run the following statement to create a ClickHouse table for storing orders data:

Listing 3.19 Setting up a ClickHouse table for storing orders data.

```
CREATE TABLE "book.online_shop.orders" (
  id Int32,
  order_date Int32,
  customer_id Int32,
  quantity Int32,
  product_id Int32
) ENGINE = MergeTree() ORDER BY id;
```

Let us try integrating Kafka with ClickHouse by creating a sink connector on our Kafka Connect cluster, as shown in Listing 3.20. We call the connector *clickhouse*, point it to the database *clickhouse* running on the ClickHouse server that we deployed with Docker Compose, and choose to sync all records from the Kafka topic *book.online_shop.orders* to the ClickHouse table with the same name (the mapping between Kafka topic names and ClickHouse table names is implicitly defined by the ClickHouse sink connector). Technically, sink connectors are Kafka consumers. Recall that, by default, consumers start streaming records from the earliest available offset in the absence of a committed offset. The topic *book.online_shop.orders* features records with different schemas, because we added a SMT to the source connector in Section 3.3 after performing the initial snapshot. To avoid potential errors, we set the consumer setting `auto.offset.reset = latest`, so the sink connector ignores existing records but starts streaming from the latest available offset.

Listing 3.20 Setting up a sink connector for ClickHouse.

```
$ curl -XPOST \
-H'Content-Type:application/json' \
http://localhost:8083/connectors \
--data-raw '{
  "name": "clickhouse",
  "config": {
    "connector.class":
      "com.clickhouse.kafka.connect.ClickHouseSinkConnector", #1
    "consumer.override.auto.offset.reset": "latest", #2
    "hostname": "clickhouse", #3
    "port": "8123", #4
    "username": "clickhouse", #5
    "password": "clickhouse", #6
    "ssl": "false", #7
    "database": "clickhouse", #8
    "topics": "book.online_shop.orders", #9
    "errors.retry.timeout": "30",
    "schemas.enable": "false", #10
    "key.converter":
      "org.apache.kafka.connect.storage.StringConverter", #11
    "value.converter":
      "org.apache.kafka.connect.json.JsonConverter", #12
    "value.converter.schemas.enable": "true"
  }
}'
```

#1 Choose the ClickHouse sink connector by passing its classname.

#2 Start streaming records from the *end* of the topic.

#3 Host of ClickHouse.

#4 Port of ClickHouse.

#5 Username for connecting to ClickHouse.

#6 Password for connecting to ClickHouse.

#7 Disable SSL for connecting to ClickHouse.

#8 Name of the ClickHouse database.

#9 Comma-separated list of the topics that shall be synced to ClickHouse.

#10 Disable the validation of the Kafka Connect schemas.

#11 Interpret the record keys as strings.

#12 Interpret the record values as JSON.

For the complete documentation of the ClickHouse sink connector, please see the official website: <https://clickhouse.com/docs/en/integrations/kafka#clickhouse-kafka-connect-sink>

Since the sink connector starts streaming records from the latest available offset, the ClickHouse table will stay empty until new change events occur in the PostgreSQL database. Trigger a new snapshot of the source connector by executing the commands from Listing [3.15](#).

Let us query the order data in ClickHouse, after opening a ClickHouse Client with `docker exec -it clickhouse clickhouse-client`, as shown in Listing [3.21](#):

Listing 3.21 Querying the order data in ClickHouse.

```
USE clickhouse;  
SELECT * FROM "book.online_shop.orders";
```

Congratulations! You successfully integrated PostgreSQL with ClickHouse. Your streaming data pipeline replicates data between the two systems in real-time and applies a stateless transformation function, which flattens the structure of the CDC events, to the data.

Please feel free to play with the integration. You could apply changes to PostgreSQL before querying ClickHouse again. You should be able to see the changes become available in ClickHouse instantly.

3.5 Summary

- Change data capture can be implemented using replication logs, trigger functions, or timestamp columns. Replication logs are the preferred and most efficient way to implement CDC.
- Debezium subscribes to the changes of PostgreSQL by leveraging its logical replication feature that makes the WAL available in a readable format.
- Replication logs hold all changes from a fixed time period, e.g., the last seven days. When starting up for the first time and before streaming data using the replication log, Debezium takes a snapshot of the source database to capture all data.
- Debezium persists the connector state in Kafka and can recover from outages or restarts without reprocessing data.
- Debezium defines a sophisticated format for CDC events that includes the state of the row before and after applying the change, as well as information about the source and the time of the CDC event.
- Single message transforms are handy for taking care of implementing small, stateless transformations but offer a poor developer experience, because it's hard to deploy and debug them, and should be used with caution.
- Sink connectors take care of streaming data from Kafka topics to external data systems, like data warehouses or APIs.

- [1] At the time of writing, Debezium version 3.4 has been released, which offers log-based CDC connectors for Apache Cassandra, Db2, Google Cloud Spanner, MongoDB, MySQL, Oracle, PostgreSQL, SQL Server, and Vitess.

4

Building custom connectors with Kafka Connect

This chapter covers

- Developing a custom source connector with Kafka Connect
- Handling errors in connectors
- Persisting connector offsets with Kafka Connect
- Testing connectors with unit tests

The last chapter introduced Kafka Connect and showed how to use community connectors for integrating Kafka topics with external data systems without writing code. In particular, we applied the Debezium source connector for streaming order data from Excellent Toy's PostgreSQL database to Kafka and used a sink connector for integrating Kafka with Excellent Toy's data warehouse.

The open-source community around Apache Kafka provides and maintains hundreds of ready-to-use connectors for various data systems that you can install in your Kafka Connect instance by adding the connector's JAR file to the classpath. However, in practice, you might need to integrate data systems that are not (yet) supported by an existing open-source connector. Excellent Toys uses an analytics tool for tracking their website traffic. The analytics tool exposes an HTTP API that can be used for extracting the tracked visitor data. Unfortunately, the Kafka community has not yet built a connector for the API of the analytics tool. In this chapter, we develop a custom connector for the API of Excellent Toys' analytics tool, which allows us to stream the visitor data to Kafka, integrate them with other data systems of Excellent Toys, and make them available to downstream consumers.

[Section 4.1](#) discusses how to use change data capture for extracting data from the HTTP API of the analytics tool.

[Section 4.2](#) shows how to set up the Java project of a Kafka Connect connector and explains its structure. It walks you through implementing the most important Java classes of your connector. After completing the section, you will have the first version of your connector ready that can extract data from the API endpoint and produce them to a Kafka topic.

Building on this minimal version, [Section 4.3](#) explains how to handle errors in Kafka Connect.

[Section 4.4](#) presents how to keep track of the state of the connector. In our case, we will persist the timestamp of the most recent API call, even beyond complete restarts of the connector, and dynamically fill it into the query parameter `since`.

Finally, [Section 4.5](#) walks you through verifying the functional correctness of connectors with unit tests.

4.1 Using change data capture with HTTP APIs

If you have installed this book's dependencies via Docker Compose, as instructed in Chapter 3, the HTTP API of the analytics tool should be available on <http://localhost:8080/visitors>. As shown in Listing 4.1, each call of the API returns information about the visits that occurred after the timestamp provided in the query parameter `since`:

Listing 4.1 Calling the API endpoint to retrieve data about visits since a given timestamp.

```
$ curl http://localhost:8080/visitors?since=2023-09-07T04:22:00.000000
> [
  { "timestamp": "2023-09-07T04:22:19.321543", "domain":
→ "excellenttoys.corp", "page": "/", "ip": "192.168.123.132",
→ "device": "Firefox" },
  { "timestamp": "2023-09-07T04:22:22.391113", "domain":
→ "excellenttoys.corp", "page": "/products", "ip": "192.168.123.132",
→ "device": "Firefox" },
  { "timestamp": "2023-09-07T04:22:25.074089", "domain":
→ "excellenttoys.corp", "page": "/", "ip": "85.214.132.117", "device":
→ "Chrome" }
]%
```

APIs do not provide replication logs or any similar mechanism that exposes data change events. We can still implement change data capture for extracting changes from the analytics tool by recurringly calling the API. By setting the query parameter `since` to the most recent timestamp that we have processed, each request would extract the visitor data that occurred since the last call. Assuming that we call the API every 60 seconds, the API calls would look as follows when starting at the timestamp `2023-09-07T04:22:00.000000`:

Listing 4.2 How to fill the query parameter `since` when calling the API endpoint every 60 seconds.

```
$ curl http://localhost:8080/visitors?since=2023-09-07T04:22:00.000000
$ curl http://localhost:8080/visitors?since=2023-09-07T04:23:00.000000
$ curl http://localhost:8080/visitors?since=2023-09-07T04:24:00.000000
$ curl http://localhost:8080/visitors?since=2023-09-07T04:25:00.000000
$ curl http://localhost:8080/visitors?since=2023-09-07T04:26:00.000000
$ ...
```

NOTE Change data capture is limited to extracting insertions and updates from HTTP APIs. It is not possible to extract deletions because the API response does not include deleted records unless using soft deletions. We can accept this limitation in the case of Excellent Toys' analytics tool because we do not expect visitor data to be deleted.

Let's move on to setting up our custom connector that implements the discussed approach to extracting data from the analytics tool.

4.2 Under the hood of custom connectors

In Kafka Connect, connectors are implemented in Java. They can be packaged as JAR files and can be installed onto clusters by adding the JAR file to their classpath. Once a connector is available on a cluster, you can instantiate it via the HTTP API of Kafka Connect.

When designing APIs or building products, I find it very intuitive to start from the perspective of the potential user. Let's imagine that our connector is implemented and available on our local Kafka Connect cluster. How would we configure and instantiate it? Listing 4.3 shows what creating a new instance of our connector via Kafka Connect's API could look like.

Listing 4.3 How creating an instance of the to-be-implemented source connector for the analytics API could look like.

```
$ curl -XPOST \
-H'Content-Type:application/json' \
http://localhost:8083/connectors \
--data-raw '{
  "name": "analytics",
  "config": {
    "connector.class":
      "com.manning.streamingdatapipelines.AnalyticsApiSourceConnector", #1
    "base.uri": "http://fake-analytics-api:8080", #2
    "poll.interval.ms": "30000", #3
    "topic.name": "online_shop.analytics.visitors" #4
  }
}'
```

#1 The classname of our connector.

#2 The base URI of the API.

#3 The interval between subsequent calls of the analytics API, defined in milliseconds.

#4 The name of the Kafka topic that the connector should produce data to.

4.2.1 Setup

As visualized in Figure 4.1, when creating a new connector via an API call, Kafka Connect creates a new instance of the Java class specified in the connector configuration under `connector.class`, here `AnalyticsApiSourceConnector`. This class acts as the entry point of a connector, accepts the configuration provided in the request body of the API call, and passes it to the connector tasks, which are implemented by separate classes (here called `AnalyticsApiSourceTask`). Kafka Connect does not provide any mechanism for separating the workloads of different connectors: Both Kafka Connect and all instantiated connector classes of a specific worker of the Kafka Connect cluster run in the same JVM process.

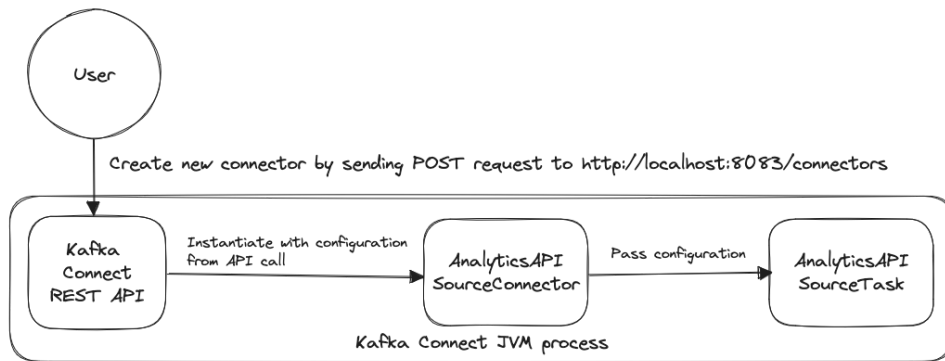


Figure 4.1 What happens when creating a new instance of our custom connector via Kafka Connect's REST API.

Let's get started with developing a custom source connector for the analytics API. To speed things up, I prepared a project template that you can download from GitHub:

Listing 4.4 Downloading the project template for the source connector.

```
$ wget https://github.com/flippingbits/streaming-data-pipelines-book
--connect-template/archive/refs/heads/main.zip
$ unzip main.zip
$ mv streaming-data-pipelines-book-connect-template-main analytics
--api-connector
```

After downloading the .zip file, unpack it and rename the directory to a name of your choice, e.g., `analytics-api-connector`.

4.2.2 Implementing the custom connector

Change to the directory holding the project template and navigate to the file `src/main/java/com/manning/streamingdatapipelines/AnalyticsApiSourceConnector.java`. The file holds the class `AnalyticsApiSourceConnector`, which serves as the entry point of our connector and is referenced by the configuration shown in Listing 4.3.

As shown in Listing 4.5, the class `AnalyticsApiSourceConnector` must inherit from Kafka Connect's class `SourceConnector`. It can specify the behavior of the connector by overwriting specific methods from its base class. For instance, the method `start` is called by Kafka Connect when the connector is being started and can take care of setup tasks, like the handling of the configuration. Similarly, the method `stop` is called when shutting down a connector and can handle cleanup tasks, such as closing connections or other resources. Additionally, our connector class can overwrite methods for returning its configuration, version, or implementation of connector tasks.

Listing 4.5 The class `AnalyticsApiSourceConnector` from the project template.

```

package com.manning.streamingdatapipelines;

import java.util.List;
import java.util.Map;
import org.apache.kafka.common.config.ConfigDef;
import org.apache.kafka.connect.connector.Task;
import org.apache.kafka.connect.source.SourceConnector;
import static java.util.stream.Collectors.toList;
import static java.util.stream.IntStream.range;

public class AnalyticsApiSourceConnector extends SourceConnector {
    private Map<String, String> settings;

    @Override
    public void start(Map<String, String> settings) {
        this.settings = settings;
    }

    @Override
    public ConfigDef config() { #1
        return AnalyticsApiSourceConnectorConfig.config();
    }

    @Override
    public Class<? extends Task> taskClass() { #2
        return AnalyticsApiSourceTask.class;
    }

    @Override
    public List<Map<String, String>> taskConfigs(int maxTasks) { #3
        return range(0, maxTasks).boxed().map(__ -> settings).collect(toList());
    }

    @Override
    public void stop() {
    }

    @Override
    public String version() { #4
        return "0.0.1";
    }
}

```

```
}
}
```

#1 Returns the configuration definition of the connector as an instance of Kafka Connect's `ConfigDef` class.

#2 Returns the class that implements the tasks of the connector.

#3 Takes care of passing the connector configuration to the task instances of the connector.

#4 Returns the version of the implementation of the connector.

The connector delegates the handling of the configuration to the class `AnalyticsApiSourceConnectorConfig`, which is defined in the file `src/main/java/com/manning/streamingdatapipelines/AnalyticsApiSourceConnectorConfig.java`.

Connectors can define their configuration options using Kafka Connect's class `ConfigDef`. By calling the instance method `define`, they can specify the properties of a configuration option, e.g., its name, the expected data type of the value, a default value, etc. Listing 4.6 shows how the method `config` returns an instance of `ConfigDef` that defines three configuration options: `base.uri`, `poll.interval.ms`, and `topic.name`:

Listing 4.6 The class `AnalyticsApiSourceConnectorConfig` from the project template.

```
package com.manning.streamingdatapipelines;

import java.util.Map;
import org.apache.kafka.common.config.AbstractConfig;
import org.apache.kafka.common.config.ConfigDef;

public class AnalyticsApiSourceConnectorConfig extends AbstractConfig {
    ...

    public static ConfigDef config() {
        return new ConfigDef()
            .define(
                "base.uri", #1
                ConfigDef.Type.STRING, #2
                "http://localhost:8080", #3
                ConfigDef.Importance.HIGH, #4
                "The base URI of the API, e.g., http://localhost:8080." #5
            )
            .define(
                "poll.interval.ms",
                ConfigDef.Type.LONG,
                30000,
                ConfigDef.Importance.HIGH,
                "The frequency of the polling in milliseconds."
            )
    }
}
```

```

    )
    .define(
        "topic.name",
        ConfigDef.Type.STRING,
        ConfigDef.NO_DEFAULT_VALUE,
        ConfigDef.Importance.HIGH,
        "The name of the Kafka topic."
    );
}
}

```

#1 The name of the configuration option.

#2 The type of the value of the configuration option

#3 The default value of the configuration option.

#4 The importance of the configuration option.

#5 The description of the configuration option, which serves documentation purposes.

So far, we looked at the entry point of our connector, the class `AnalyticsApiSourceConnector`, and the handling of the configuration, performed by the class `AnalyticsApiSourceConnectorConfig`. Next, we discuss the implementation of the connector tasks, which are responsible for extracting data from the API and sending them to a Kafka topic. Similar to `AnalyticsApiSourceConnector`, the class implementing the source task inherits from a class provided by Kafka Connect, here `SourceTask`, and can define actions that shall be performed when starting or stopping a source task by overwriting methods of the base class. The actual processing logic that shall be performed for extracting data from the API of the analytics tool can be defined in the method `poll`. When executing the connector (task), Kafka Connect continuously calls that method, `poll`, until the connector stops. The method `poll` must return the extracted data as a `List` of instances of `SourceRecord`, a class provided by Kafka Connect. In our project template, connector tasks are implemented by the class `AnalyticsApiSourceTask`, which can be found in the file `src/main/java/com/manning/streamingdatapipelines/AnalyticsApiSourceTask.java`:

Listing 4.7 The class AnalyticsApiSourceTask from the project template.

```

package com.manning.streamingdatapipelines;

import org.apache.kafka.connect.source.SourceRecord;
import org.apache.kafka.connect.source.SourceTask;

import java.util.List;
import java.util.Map;
import java.util.ArrayList;

public class AnalyticsApiSourceTask extends SourceTask {
    private AnalyticsApiSourceConnectorConfig config;
    private ObjectMapper objectMapper;

    @Override
    public String version() {
        return "0.0.1";
    }

    @Override
    public void start(Map<String, String> settings) {
        this.config = new AnalyticsApiSourceConnectorConfig(settings);
        this.objectMapper = new ObjectMapper();
    }

    @Override
    public List<SourceRecord> poll() throws InterruptedException {
        List<SourceRecord> sourceRecords = new ArrayList<>();

        // TODO: To be implemented

        return sourceRecords;
    }

    @Override
    public void stop() {
    }
}

```

Before implementing the core functionality of our custom connector, let's think through Excellent Toys' requirements on the connector. Excellent Toys wants to extract the visitor data from their analytics tool and produce them to Kafka, such that they can integrate the data with other data sources and make them available to downstream consumers. As discussed in [Section 4.1](#), we could implement change data capture by polling the API of the analytics tool at fixed intervals and extracting the data that occurred since the last access. Technically, these requirements could be implemented as follows. Each time the method `poll` of the class `AnalyticsApiSourceTask` is called by Kafka Connect, it should perform the following steps:

1. Implement the pause in between two accesses of the Analytics API by waiting for the number of milliseconds defined in the configuration option `poll.interval.ms`.
2. Send an HTTP request to the Analytics API to extract the latest visitor data.
3. Parse the JSON-formatted response body and turn it into a `List` of `SourceRecord` objects.
4. Return the `List` of `SourceRecord` objects. Kafka Connect takes care of producing them to the configured Kafka topic.

In the following paragraphs, we walk through the implementation of each step separately before presenting the complete implementation of the `poll()` method in Listing [4.11](#).

In the first step, we need to implement the pause in between two accesses of the API. Listing [4.8](#) shows a potential implementation that waits for the number of milliseconds defined in the configuration property `poll.interval.ms`:

Listing 4.8 Pause the connector for the number of milliseconds defined in the configuration option `poll.interval.ms`.

```
Thread.sleep(config.getPollIntervalMs());
```

For sending the HTTP request to the Analytics API and reading the JSON response, we create an instance of the Java class `URL` and make use of the Java library Jackson, as shown in Listing [4.9](#):

Listing 4.9 Call the Analytics API and read the JSON-formatted response body.

```
try {
    String fullPath = String.format("%s/visitors", config.getBaseUri()); #1
    URL apiUri = new URL(fullPath);
    JsonNode apiResponse = this.objectMapper.readTree(apiUri);
} catch (IOException e) { #2
    throw new InterruptedException(e.getMessage());
}
```

#1 If the connector configuration `base.uri` is set to <http://localhost:8080>, the concatenation will build the string value <http://localhost:8080/visitors>.

#2 Catch exceptions of type `IOException` and re-throw them as one of type `InterruptedException`. Kafka Connect provides an exception handler for the latter one.

Let's recall what the response of the API looks like (see Listing 4.1 for more details). The API returns a JSON array of objects. Each object represents a website visit. Let's iterate over the JSON array and build a `SourceRecord` from each JSON object:

Listing 4.10 Call the Analytics API and read the JSON-formatted response body.

```
List<SourceRecord> sourceRecords = new ArrayList<>();
try {
    Schema keySchema = SchemaBuilder.string().build(); #1
    Schema valueSchema = SchemaBuilder.struct() #2
        .field("domain", string().build()) #3
        .field("timestamp", string().build())
        .field("page", string().build())
        .field("ip", string().build())
        .field("device", string().build())
        .build();

    for (JsonNode visit : apiResponse) { #4
        String ipAddress = visit.get("ip").asText(); #5
        String timestamp = visit.get("timestamp").asText();
        Struct recordValue = new Struct(valueSchema) #6
            .put("domain", visit.get("domain").asText()) #7
            .put("timestamp", timestamp)
            .put("page", visit.get("page").asText())
            .put("ip", ipAddress)
            .put("device", visit.get("device").asText());

        SourceRecord sourceRecord = new SourceRecord( #8
            Map.of("baseUri", config.getBaseUri()), #9
            Map.of("timestamp", timestamp), #10
```

```

        config.getTopicName(), #11
        null, #12
        keySchema, #13
        ipAddress, #14
        valueSchema, #15
        recordValue, #16
        System.currentTimeMillis() #17
    );

    sourceRecords.add(sourceRecord);
}
} catch (IOException e) {
    throw new InterruptedException(e.getMessage());
}

```

#1 Kafka Connect Schema of the record key of type `String`.

#2 Kafka Connect Schema of the record value of type `Struct`.

#3 Add a field with the name `domain` and the type `string` to the schema of the record value.

#4 Iterate over the JSON array returned by the Analytics API.

#5 Access the key `ip` from the JSON object and interpret it as a `String`.

#6 Build an instance of a struct that uses the schema of the record value and holds all information of a visit.

#7 Fill the key domain with the data from the key domain from the JSON object.

#8 Build an instance of the `SourceRecord` class.

#9 Partition of the record in the data source, here set to the base URI of the Analytics API.

#10 Offset of the record in the data source, here set to the timestamp attribute.

#11 Name of the Kafka topic.

#12 Partition of the Kafka topic. We set it to `null` to let Kafka choose the partition.

#13 Kafka Connect schema of the record key.

#14 The record key (here of type `String`).

#15 Kafka Connect schema of the record value.

#16 The record value (here of type `Struct`).

#17 Timestamp of the Kafka record. We use the current clock time.

When combining the different code snippets, the implementation of the `poll` method would look as follows:

Listing 4.11 Implementation of the connector task's poll method.

```

@Override
public List<SourceRecord> poll() throws InterruptedException {
    List<SourceRecord> sourceRecords = new ArrayList<>();

    // Wait for next call of the API
    Thread.sleep(config.getPollIntervalMs());

    try {
        // Call the API
        String fullPath = String.format("%s/visitors", config.getBaseUri());
        URL apiUri = new URL(fullPath);
        JsonNode apiResponse = this.objectMapper.readTree(apiUri);

        // Convert API response to list of source records
        Schema keySchema = SchemaBuilder.string().build();
        Schema valueSchema = SchemaBuilder.struct()
            .field("domain", string().build())
            .field("timestamp", string().build())
            .field("page", string().build())
            .field("ip", string().build())
            .field("device", string().build())
            .build();

        for (JsonNode visit : apiResponse) {
            String ipAddress = visit.get("ip").asText();
            String timestamp = visit.get("timestamp").asText();
            Struct recordValue = new Struct(valueSchema)
                .put("domain", visit.get("domain").asText())
                .put("timestamp", timestamp)
                .put("page", visit.get("page").asText())
                .put("ip", ipAddress)
                .put("device", visit.get("device").asText());

            SourceRecord sourceRecord = new SourceRecord(
                Map.of("baseUri", config.getBaseUri()),
                Map.of("timestamp", timestamp),
                config.getTopicName(),
                null,
                keySchema,
                ipAddress,
                valueSchema,
                recordValue,
            );
        }
    }
}

```

```

        System.currentTimeMillis()
    );

    sourceRecords.add(sourceRecord);
}
} catch (IOException e) {
    throw new InterruptedException(e.getMessage());
}

return sourceRecords;
}

```

Congratulations! You implemented a minimum-viable source connector for the Analytics API. Next, for shipping the connector, we need to package it as a JAR file.

4.2.3 Packaging and deployment

Open your terminal and navigate to the root folder of the connector project. You can compile the connector project and package it as a JAR by executing `mvn clean package`.

You should now see the JAR file when executing the following command in the root folder of the project:

Listing 4.12 Listing the content of the target folder of the connector project.

```

$ ls target
analytics-api-source-connector-0.0.1-jar-with-dependencies.jar
analytics-api-source-connector-0.0.1.jar
...

```

PACKAGING FAT JAR FILES

By default, `mvn package` creates a JAR file that includes only the classes and files from the project but leaves out those from dependencies, e.g., Jackson in our example. When deploying a connector as such a JAR, we would have to load the JARs from the connector's dependencies into the classpath of Kafka Connect separately. We configured Maven in the project template such that it creates a *fat JAR*, which includes all dependencies. While this eases the deployment of the connector, it comes with the downside of producing larger JAR files and does not allow us to update dependencies without re-packaging the connector.

Copy the JAR file `target/analytics-api-source-connector-0.0.1-jar-with-dependencies.jar` to the folder where you stored the `docker-compose.yml` that you downloaded when setting up your development environment at the beginning of Chapter 3. Next, mount the JAR file into the Kafka Connect container by adding the `volumes` section to the configuration of the Kafka Connect service in the file `docker-compose.yml` as follows:

Listing 4.13 Updated configuration of the Kafka Connect service.

```
connect:
  image: flippingbits/connect:latest
  container_name: connect
  ports:
    - 8083:8083
  links:
    - clickhouse
    - fake-analytics-api
    - kafka-1
    - postgres
  environment:
    - BOOTSTRAP_SERVERS=kafka-1:9092
    - GROUP_ID=1
    - CONFIG_STORAGE_TOPIC=my_connect_configs
    - OFFSET_STORAGE_TOPIC=my_connect_offsets
    - STATUS_STORAGE_TOPIC=my_connect_statuses
  volumes:
    - ./analytics-api-source-connector-0.0.1-jar-with-dependencies.jar:
    - /kafka/connect/analytics-api-connector.jar
```

Finally, stop your existing Docker Compose instance and restart it by executing `docker-compose up` in the folder of the `docker-compose.yml`. When listing all connector plugins of the Kafka Connect instances, as shown in Listing 4.14, your custom connector should show up. Kafka Connect might need a few minutes to restart.

Listing 4.14 List the connector plugins that are available in your Kafka Connect cluster.

```
$ curl http://localhost:8083/connector-plugins
> [{"class":"com.manning.streamingdatapipelines
➤ .AnalyticsApiSourceConnector","type":"source","version":"0.0.1"}, ...]%
```

You should now be able to create an instance of your connector with the code shown in Listing 4.3. You can observe the results by creating a consumer that subscribes to the topic `online_shop.analytics.visitors`:

Listing 4.15 Consuming records from the topic `online_shop.analytics.visitors`.

```
$ docker exec kafka \
  /bin/kafka-console-consumer \
  --bootstrap-server=kafka:29092 \
  --topic=online_shop.analytics.visitors \
  --from-beginning \
  --property print.key=true
```

4.3 Handling errors in connectors

By default, Kafka Connect handles errors that occur in running connectors by stopping the connector and changing their state from `RUNNING` to `FAILED`. Connectors can be restarted by restarting the Kafka Connect service or by calling the API endpoint `/connectors/:connector_name/restart?include_tasks=true` with the HTTP method `POST` (replace `:connector_name` with the name of your connector instance):

Listing 4.16 Restarting the connector `analytics` and all of its tasks.

```
$ curl -XPOST \
  "http://localhost:8083/connectors/analytics/restart?include_tasks=true"
```

Depending on the error, you might want to employ retry mechanisms instead of letting the connector immediately fail after the occurrence of an error. In the last section, we built a custom connector that extracts data from an HTTP API. APIs can become unavailable from time to time, e.g., because of a deployment that rolls out a new version of the backend. The current implementation of the connector throws an exception when the HTTP call to the Analytics API is not successful and lets the connector immediately fail.

Let's have a look at how we could improve the resilience of our connector. Instead of immediately failing in the case of an error, we could retry performing the API request and throw an `InterruptedException` only if we have exceeded a specific number of attempts. To this end, let's introduce a new attribute for storing the number of retry attempts to the instance of `AnalyticsApiSourceTask`, called `retryAttempts`. Let's also introduce a constant, called `MAX_RETRY_ATTEMPTS`, which holds the maximum number of retry attempts that shall be performed before ultimately failing. We set `MAX_RETRY_ATTEMPTS = 5` by default. We also need to change the method `poll` of the class `AnalyticsApiSourceTask` such that it (1) increases the counter whenever an error occurs, (2) fails when `MAX_RETRY_ATTEMPTS` attempts have been performed, and (3) resets the counter when the data extraction works normally. Listing 4.17 shows the new variables and a new revision of the `poll` method of the class `AnalyticsApiSourceTask`:

Listing 4.17 Adding a retry mechanism to the connector.

```

public class AnalyticsApiSourceTask extends SourceTask {
    private Integer retryAttempts = 0; #1
    private static final Integer MAX_RETRY_ATTEMPTS = 5; #2

    ...

    @Override
    public List<SourceRecord> poll() throws InterruptedException {
        List<SourceRecord> sourceRecords = new ArrayList<>();

        // Wait for next call of the API
        Thread.sleep(config.getPollIntervalMs());

        try {
            ...

            retryAttempts = 0; #3
        } catch (IOException e) {
            retryAttempts++; #4
            if (retryAttempts >= MAX_RETRY_ATTEMPTS) { #5
                throw new InterruptedException(e.getMessage());
            }
        }

        return sourceRecords;
    }
}

```

#1 Track the number of retry attempts in a class attribute.

#2 Maximum number of retry attempts to perform.

#3 Reset the number of retry attempts in the case of a successful data extraction.

#4 Increase the number of retry attempts in the case of an error.

#5 Let the connector fail by throwing an `InterruptedException` if the maximum number of retry attempts have been exceeded.

NOTE Whenever you introduce a change to the connector's source code, re-package it by executing `mvn clean package`, copy the updated JAR file to the folder of your `docker-compose.yml`, and restart Docker Compose. It makes sense to version your connector, so you can verify that the connector update has been deployed successfully by calling <http://localhost:8083/connector-plugins> and investigating the entry for your connector.

Due to the `Thread.sleep` invocation, the connector waits the number of milliseconds specified by `poll.interval.ms` between each attempt. Do you feel ready to practice your skills in developing custom connectors? Try out introducing a new configuration option, called `retry.interval.ms`, which holds the number of milliseconds that the connector shall wait before retrying a failed API call, and adapt the implementation of the `poll` method in the `AnalyticsApiSourceTask` such that it takes the new configuration value into account. Alternatively, have a look at Java libraries, like `resilience4j` (<https://resilience4j.readme.io>), which provide implementations of more sophisticated retry policies.

4.4 How to keep track of the state of the data processing

The current implementation of the connector fetches the most recent visitor data from Excellent Toys' analytics tool at fixed intervals. It does not set the query parameter `since` for restricting the fetched visitor data to those that occurred after a specific timestamp but relies on the default behavior of the API of the analytics tool, which returns the data from the last 30 seconds. While this works perfectly fine as long as the connector is being executed, issues could come up when the connector becomes unavailable, e.g., because of a restart of Kafka Connect, a roll-out of a new connection version, or any other reason. The connector does not persist the state of the data processing, like the timestamp of the most recently processed record, and thus cannot continue the data processing after a restart at the point at which it stopped. When starting up, it must either re-process all data from the analytics tool, which leads to duplicate data, or rely on the default value of the query parameter `since` and fetch data from the last 30 seconds, which leads to data loss. Either case would probably frustrate the users of the data within Excellent Toys.

As visualized in Figure 4.2, we will adapt the connector implementation such that the connector persists the most recently processed timestamp as the *connector offset*. In the case of a restart, the connector could use that timestamp to continue the data processing without producing duplicate data or losing data. Fortunately, we do not have to build the management of the connector offset from scratch but can build on top of the functionality provided by Kafka Connect.

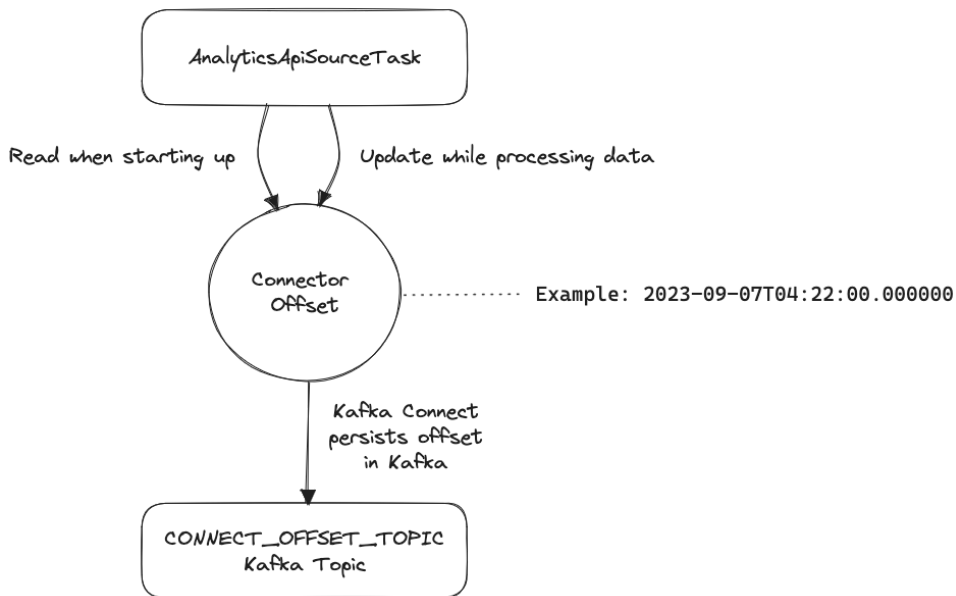


Figure 4.2 The connector keeps track of the state of the data processing in the connector offset, which is persisted in a Kafka topic by Kafka Connect.

Recall that, when we built a new `SourceRecord` in Listing 4.10, we had to set the data source's partition and offset. Kafka Connect persists this information in a dedicated Kafka topic, which is configured via the Docker container's environment `CONNECT_OFFSET_TOPIC` and, in our case, set to the value `my_connect_offsets`. Kafka Connect takes care of both reading the committed connector offset from the Kafka topic upon connector startup and writing new connector offsets to the Kafka topic.

Navigate to the class `AnalyticsApiSourceTask` in the project of your source connector and add the instance variable `lastProcessedTimestamp`, which will be used for keeping the connector offset in memory. Let's modify the method `start` of the `AnalyticsApiSourceTask` such that it loads the most recent connector offset from that topic, if there is any, and assigns the offset to the variable `lastProcessedTimestamp`:

Listing 4.18 Loading the connector offset from Kafka in the method `start` of the class `AnalyticsApiSourceTask`.

```
public class AnalyticsApiSourceTask extends SourceTask {
    private String lastProcessedTimestamp = null;

    @Override
    public void start(Map<String, String> settings) {
        this.config = new AnalyticsApiSourceConnectorConfig(settings);
        this.objectMapper = new ObjectMapper();

        Map<String, ?> offset = null;
        if (this.context != null) {
            this.context.offsetStorageReader().offset( #1
                Collections.singletonMap("baseUri", config.getBaseUri())
            );
        }

        this.lastProcessedTimestamp = (offset == null) #2
            ? null
            : (String) offset.get("baseUri");
    }
}
```

#1 Read the persisted offset from Kafka Connect's offset store.

#2 Fill the class attribute `lastProcessedTimestamp` with the persisted offset if available.

Next, we should add the `lastProcessedTimestamp` to the HTTP request that is sent to the Analytics API. Navigate to the method `poll` of the class `AnalyticsApiSourceTask` and replace the line `String fullPath = String.format("%s/visitors", config.getBaseUri());` with the code snippet in Listing 4.19, which sets the query parameter `since` to the value of `lastProcessedTimestamp` if there is any:

Listing 4.19 Filtering for visitor data that occurred since the last seen timestamp in the method `poll` of the class `AnalyticsApiSourceTask`.

```
String fullPath = (this.lastProcessedTimestamp == null)
    ? String.format("%s/visitors", config.getBaseUri())
    : String.format("%s/visitors?since=%s", config.getBaseUri(),
        ↪ this.lastProcessedTimestamp);
```

Congratulations! Your connector is now able to recover from restarts without producing duplicate data or losing records. It is keeping track of the data processing by persisting the most recently-processed timestamp in the offset storage of Kafka Connect.

4.5 Testing connectors

The great thing about Kafka Connect is that connectors are regular Java programs. That is, we can make use of existing test frameworks from the Java community to write software tests. As visualized in Figure 4.3, we must differentiate between unit and integration tests. Unit tests address the functionality of single Java classes of a connector and verify their correctness when working in isolation; for instance, they do not let connectors interact with Kafka. Integration tests verify the functionality of fully integrated connectors and, to this end, integrate them with Kafka and other external systems. While unit tests are easier to develop and faster to execute, integration tests are more helpful for validating the end-to-end functionality of connectors.

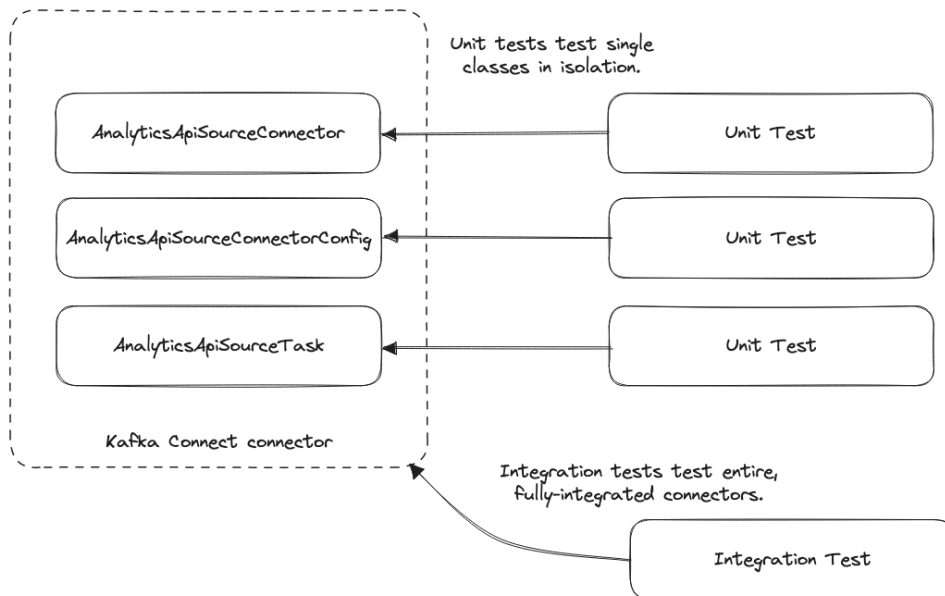


Figure 4.3 Differences between unit and integration tests when testing Kafka Connect connectors.

Let's get started with testing connectors by writing a unit test for the class `AnalyticsApiSourceConnector`, which acts as the entry point of our connector. We want to verify if it passes the connector's configuration to the tasks correctly. To this end, create the file `AnalyticsApiSourceConnectorTest.java` in the folder `src/test/java/com/manning/streamingdatapipelines` and fill it with the code from Listing 4.20:

Listing 4.20 Testing the configuration of connector tasks.

```

package com.manning.streamingdatapipelines;

import org.junit.jupiter.api.Test;
import java.util.Map;
import static org.junit.jupiter.api.Assertions.assertEquals;

public class AnalyticsApiSourceConnectorTest {
    @Test
    public void testTaskConfiguration() {
        AnalyticsApiSourceConnector apiConnector = #1
            new AnalyticsApiSourceConnector();
        Map<String, String> connectorConfig = Map.of( #2
            "base.uri",          "http://localhost:8080",
            "poll.interval.ms",  "5000",
            "topic.name",        "online_shop.analytics.visitors"
        );
        apiConnector.start(connectorConfig); #3
        Map<String, String> taskConfig =
            apiConnector.taskConfigs(1).get(0); #4

        assertEquals("http://localhost:8080", taskConfig.get("base.uri")); #5
        assertEquals("5000", taskConfig.get("poll.interval.ms"));
        assertEquals("online_shop.analytics.visitors",
            taskConfig.get("topic.name"));
    }
}

```

#1 Create a new instance of your connector.

#2 Store the configuration of the connector in a Map object.

#3 Pass the configuration to the connector instance and start it.

#4 Access the first connector task of the connector.

#5 Verify that the connector task holds the expected configuration.

You can execute the tests by running the command `mvn test` in the root directory of your connector's project folder.

The core functionality of the connector is implemented in the `poll` method of the class `AnalyticsApiSourceTask`: It performs an HTTP call to extract visitor data from the Analytics API and returns the data as a list of `SourceRecord` objects. Listing 4.21 shows a unit test, which uses the assertion `assertFalse` to verify that the list returned by the method `poll` is not empty. Place the code from the listing in the file `src/test/java/com/manning/streamingdatapipelines/AnalyticsApiSourceTaskTest.java`. Note that the successful execution of the test requires the Analytics API to be available at the configured base URI (here <http://localhost:8080>).

Listing 4.21 Testing the polling of data from the Analytics API.

```

package com.manning.streamingdatapipelines;

import java.util.Map;
import java.util.List;
import org.apache.kafka.connect.source.SourceRecord;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.assertFalse;

public class AnalyticsApiSourceTaskTest {
    @Test
    public void testPolling() throws InterruptedException {
        AnalyticsApiSourceTask sourceTask = new AnalyticsApiSourceTask();
        Map<String, String> connectorConfig = Map.of(
            "base.uri", "http://localhost:8080",
            "poll.interval.ms", "5000",
            "topic.name", "online_shop.analytics.visitors"
        );
        sourceTask.start(connectorConfig);
        List<SourceRecord> sourceRecords = sourceTask.poll();

        assertFalse(sourceRecords.isEmpty());
    }
}

```

Feel free to increase the test coverage by introducing more unit tests. For instance, you might want to test the behavior of your connector in the case of failing HTTP requests.

INTEGRATION TESTS

In this chapter, we tested two classes of our custom connector in isolation. It might make sense to implement integration tests that cover the end-to-end functionality of a connector, too. Integration tests are much harder to implement and take much longer to execute than unit tests because they cover lots of different systems.

Testcontainers (<https://testcontainers.com/>) is an open-source framework that lets you create and use containers in your tests and is well-suited for spinning up the different systems used in integration tests. In the case of a connector, it would make sense to let the tests spin up a Kafka cluster, a Kafka Connect cluster, and the external data system integrated by the connector. Before executing the actual test, we would also have to package the current connector implementation as a JAR and deploy it onto the Kafka Connect cluster.

Due to their complexity, we refrained from covering integration tests in this chapter but encourage you to explore the topic and try out frameworks, such as Testcontainers.

4.6 Summary

- Kafka Connect supports the development of custom connectors for integrating data systems that are not yet covered by existing connectors.
- Kafka Connect provides a Java library for implementing custom connectors as regular Java applications.
- Custom connectors are packaged and shipped as JAR files, which can be installed onto a Kafka Connect cluster by adding the JAR file to its classpath.
- Kafka Connect offers connector offsets as a means to maintain the state of the data processing across restarts reliably. Internally, Kafka Connect stores connector offsets in a Kafka topic.
- By default, connectors fail in the occurrence of an error, i.e., a Java exception, stop their work, and must be manually restarted.
- Connectors can implement sophisticated retry mechanisms for dealing with failures, such that they can cope with temporary outages of an external data system.
- Since custom connectors are regular Java applications, single components (or classes) of a connector can be tested with unit tests.

5

Stateless transformations with Kafka Streams

This chapter covers

- Setting up and configuring a Kafka Streams application
- Mapping Kafka records
- Filtering Kafka records
- Routing Kafka records

So far, we have focused on replicating data changes from data sources to data sinks without processing them on the way. A lot of use cases require transforming the data while streaming them. For instance, to become compliant with data protection regulations, Excellent Toys might want to mask the IP addresses of website visitors when streaming the data from their analytics tool to downstream consumers.

Kafka Streams is a Java library that helps you build stream applications that consume data from Kafka topics as their input, apply processing logic of arbitrary complexity to the data, and produce the processed data to Kafka topics as their output. Kafka Streams applications can be developed, tested, packaged, and deployed as regular Java applications, which makes stream processing accessible to software developers and anyone with basic programming experience.

This chapter focuses on using Kafka Streams for implementing stateless transformations, which process event by event. Examples are *map* functions that take a single event and yield a new version of the event ([Section 5.2](#)), *filter* functions that ignore all events for further processing that do not match a defined predicate ([Section 5.3](#)), and *routing* functions that produce events to Kafka topics depending on a custom routing logic ([Section 5.4](#)). The next chapter covers stateful transformations, like aggregations or joins.

Let's have a look at how to set up and configure a Kafka Streams application before we dive deep into the implementation of stateless transformations.

NOTE This chapter assumes that you have successfully implemented the custom connector from Chapter 4. It works with the analytics data stored in the topic `online_shop.analytics.visitors`. If you got stuck implementing the connector, please download the JAR from <https://github.com/flippingbits/streaming-data-pipelines-book/raw/main/analytics-api-source-connector-0.0.1-jar-with-dependencies.jar>, install it as shown in Section 4.2.3, and instantiate the connector as presented in Listing 4.3, so the topic `online_shop.analytics.visitors` is filled with data.

5.1 Setting up a Kafka Streams application

Figure [5.1](#) visualizes a Kafka Streams application with

- A source node consuming records from the Kafka topic `online_shop.analytics.visitors` that is populated by the custom connector we developed in the last chapter.
- One processing node, which accepts records from the source node, masks IP addresses stored in the record values, and passes the processed records to the sink node.
- A sink node that accepts records from the processing node and produces them to the topic `online_shop.analytics.visitors_processed`.

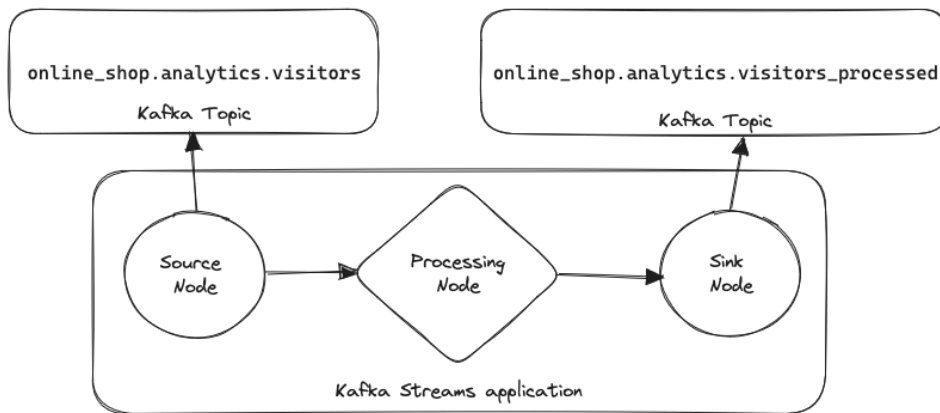


Figure 5.1 Kafka Streams application with one processing node.

Let's take a look at a potential implementation of the application. We prepared a project template that you can use to quickly set up a Kafka Streams application. Please follow the instructions in Listing 5.1 to download it. After downloading the .zip file, unpack it and rename the directory to a name of your choice, e.g., `kafka-streams-app`.

Listing 5.1 Downloading the project template for the Kafka Streams application.

```
$ wget https://github.com/flippingbits/streaming-data-pipelines-book-
streams-template/archive/refs/heads/main.zip
$ unzip main.zip
$ mv streaming-data-pipelines-book-streams-template-main kafka-streams-app
```

Navigate to the folder `src/main/java/com/manning/streamingdatapipelines` and open the file `StreamsApp.java`, which will serve as the entry point of the Java project. The method `main` is called when running the built Java project and might be a reasonable location for instantiating the Kafka Streams application. Given that Kafka Streams is an ordinary Java library, you have full flexibility to organize your code differently.

5.1.1 Configuration

In the first step, we need to take care of configuring the Kafka Streams application. Kafka Streams requires two configuration parameters:

- The configuration option `application.id` defines the identifier or name for the Kafka Streams application. It must be unique within a specific Kafka cluster because it is used for isolating different Kafka Streams applications from each other and serves as the `group.id` of the underlying Kafka consumer. You can scale Kafka Streams applications by running multiple instances of the same application (using the same `application.id`).

- The configuration option `bootstrap.servers` lets you point the Kafka Streams application to the Kafka cluster that it should work with.

Additionally, you can define configuration options, like, for instance, the default SerDe classes used for serializing and deserializing the record keys and values. While Kafka Streams tries to set reasonable default values for the config of the underlying Kafka consumer and producer, you can adapt the configuration to your own needs by overriding them. See the documentation of Kafka Streams for a complete list of the available configuration options: <https://kafka.apache.org/21/documentation/streams/developer-guide/config-streams#optional-configuration-parameters>

For managing the configuration of a Kafka Streams application, we create a new instance of Java's `Properties` class and store all configuration options in it:

Listing 5.2 Creating configuration of the Kafka Streams application.

```
Properties props = new Properties();
props.put("application.id", "mask-ip-addresses");
props.put("bootstrap.servers", "localhost:9092");
```

NOTE Kafka Streams sets the `group.id` of the Kafka consumers to the value of `application.id`. When restarting a Kafka Streams application, the source nodes continue consuming records from the Kafka topics at the last committed offset, i.e., they do neither re-process data nor lose data (unless the application has been unavailable longer than the retention time of the Kafka topic or the consumer offsets; by default, the broker configuration `offsets.retention.minutes` is set to 10080, which means that Kafka cleans up consumer offsets after seven days of inactivity). If you introduce fundamental changes to your Kafka Streams application, you might want to re-process existing records. In this case, include a version in the `application.id` that is updated in case of, for instance, major releases.

5.1.2 Processing topology

Next, we need to define the processing topology of the Kafka Streams application. Listing 5.3 shows a topology that is defined as an instance of Kafka Streams' `StreamsBuilder` class. It consists of one source node, which consumes records from the Kafka topic `online_shop.analytics.visitors`, one processing node, which maps the record values to the return value of the method `maskIpAddresses`, and one sink node, which produces the records to the topic `online_shop.analytics.visitors_processed`.

When defining the source node, we need to define a deserializer for converting the data from the byte array, which Kafka internally uses for persisting data, into the format used by our application. In this example, we convert both the keys and the values to a `String` by passing `Serdes.String()` to both parameters of `Consumed.with()` (the first parameter defines the serializer for the key, and the second one for the value). Similarly, when defining the sink node, we need to define a serializer for converting the data from our application's format into a byte array, which is then produced to Kafka. Kafka provides serializers and deserializers (SerDe) for many data formats. You can also implement your own SerDe classes by developing a class that inherits from the class `org.apache.kafka.common.serialization.Serdes WrapperSerde<T>`.

Listing 5.3 Processing topology of the Kafka Streams application.

```
final StreamsBuilder builder = new StreamsBuilder();
builder
    .stream( #1
        "online_shop.analytics.visitors",
        Consumed.with(Serdes.String(), Serdes.String())
    )
    .mapValues(StreamsApp::maskIpAddressInRecord) #2
    .to( #3
        "online_shop.analytics.visitors_processed",
        Produced.with(Serdes.String(), Serdes.String())
    );
```

#1 The source node

#2 The processing node

#3 The sink node

The processor node calls the method `maskIpAddressInRecord` (see Listing [5.4](#)), which accepts the deserialized record value, parses the JSON object stored in the `String recordValue`, masks the JSON object's key `ip` by setting the last 8 bits of an IPv4 address (or last 80 bits of an IPv6 address) to zero, and returns the JSON object as a string value. The method handles errors, like deserialization failures, by logging them and returning the original record value unchanged.

Listing 5.4 Masking the IP address.

```
private static String maskIpAddressInRecord(String recordValue) {
    try {
        JsonNode valueObject = objectMapper.readTree(recordValue); #1
        JsonNode payload = valueObject.get("payload"); #2

        String ip = payload.get("ip").textValue(); #3
        InetAddress ipAddress = InetAddress.getByName(ip);
        byte[] address = ipAddress.getAddress();
        int anonymizedBytes = ipAddress instanceof InetAddress ? 10 : 1;
        for (int i = 0; i < anonymizedBytes; ++i) {
            address[address.length - i - 1] = 0; #4
        }
        ip = InetAddress.getByAddress(address).getHostAddress();
        ((ObjectNode) payload).put("ip", ip);

        return valueObject.toString(); #5
    } catch (UnknownHostException | JsonProcessingException e) { #6
        logger.error("Could not apply maskIpAddressInRecord()", e);
        return recordValue;
    }
}
```

#1 Deserialize the record value to a JSON object.

#2 Get the key payload of the JSON object, which is a JSON object itself.

#3 Get the key ip of the payload.

#4 Mask the last 8 (or 80 bytes) of the IPv4 (or IPv6) address by replacing all bits with zeros.

#5 Return the updated JSON object.

#6 Handle errors, like invalid IP addresses.

Figure 5.2 visualizes the masking of the IP address by the developed processing node.

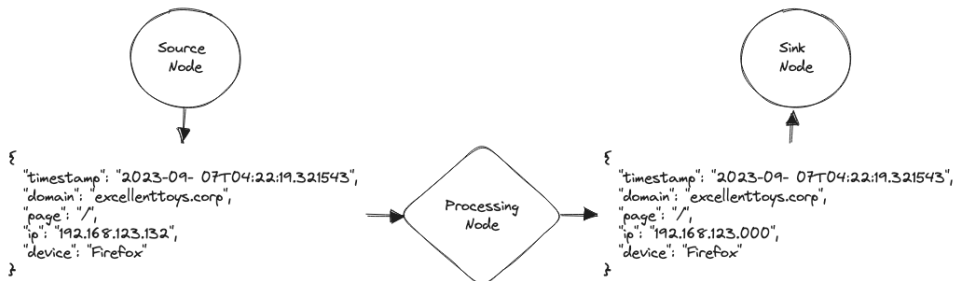


Figure 5.2 Masking of the IP address by the processing node.

5.1.3 Executing the Kafka Streams application

We can instantiate the Kafka Streams application by creating a new instance of the class `KafkaStreams` and passing the `Properties` and `StreamsBuilder` objects to it:

Listing 5.5 Instantiating the Kafka Streams application.

```
final KafkaStreams streams = new KafkaStreams(builder.build(), props);
```

After creating the instance, we first clean up any existing local state from prior runs with `streams.cleanUp()` and finally start the Kafka Streams application by calling `streams.start()`. In the case of an exception, we log the exception's message and stop the Kafka Streams application.

Listing 5.6 Starting the Kafka Streams application.

```
try {
    streams.cleanUp();
    streams.start();
} catch (Throwable e) {
    logger.error("Error while running Kafka Streams app", e);
    System.exit(1);
}
```

Congratulations! You built your first stream processing application with Kafka Streams.

Now, it's time to execute it. You can compile and run the Java application by navigating to the root folder of the Java project and executing the following statement on the command line:

Listing 5.7 Running the Kafka Streams application.

```
$ mvn compile exec:java \
  -Dexec.mainClass="com.manning.streamingdatapipelines.StreamsApp"
```

While calling `mvn compile exec:java` works great when trying out a Kafka Streams application on your local machine, it is clearly not feasible for production-grade deployments on remote machines or orchestration engines like Kubernetes. Chapter 7 will cover how to package your Kafka Streams application for production.

Watch the results of your first Kafka Streams application by consuming the records from the sink topic `online_shop.analytics.visitors_processed`:

Listing 5.8 Consuming records from the topic `online_shop.analytics.visitors_processed`.

```
$ docker exec kafka \
  /bin/kafka-console-consumer \
  --bootstrap-server=kafka:29092 \
  --topic=online_shop.analytics.visitors_processed \
  --from-beginning \
  --property print.key=true
```

5.2 Mapping data

Mapping data from one format to another while streaming them between Kafka topics is one of the most common requirements for Kafka Streams application. Example use cases include

- Changing the schema of records, such that records match the format expected by the data sink.
- Masking sensitive data, e.g., IP addresses, credit card numbers, email addresses, etc.
- Enriching records with data from other data sources, e.g., adding the geographical region of an IP address by looking it up with an API call.
- Changing the data format, e.g., converting record values from JSON to Avro.

5.2.1 `map()` and `mapValues()`

In [Section 5.1](#), we used the `mapValues` operator to mask IP addresses stored in the record values, which allows Excellent Toys to provide their analytics data to downstream data consumers without exposing personal identifiable information. The operator `mapValues` allows you to transform the values of records by applying a Java function to them while keeping the record key and headers as they are.

Recall that the source connector from Chapter 4 stores the IP address of visitors in the record key. While Kafka Streams' `mapValues` allows us to transform the record values, it does not support the processing of record keys. Excellent Toys aims at being fully compliant with data protection regulations and, to this end, must mask IP addresses in record keys, too. We can deal with the requirement by using `map` instead of `mapValues`. The operator `map` is a more general form of `mapValues` and allows you to transform the record keys, too.

Listing [5.9](#) shows how to adapt the topology in the `main` method of the `StreamsApp` class so that it makes use of the `map` operator:

Listing 5.9 Updated processing topology of the Kafka Streams application.

```
builder
    .stream(
        "online_shop.analytics.visitors",
        Consumed.with(Serdes.String(), Serdes.String())
    )
    .map(StreamsApp::maskIpAddressInRecord)
    .to(
        "online_shop.analytics.visitors_processed",
        Produced.with(Serdes.String(), Serdes.String())
    );
```

Next, adjust the method `maskIpAddressInRecord`, such that it accepts two arguments, `recordKey` and `recordValue`, and returns an instance of KafkaStreams' `KeyValue`, which is a pair of two objects, in our case string values. Let's also move the implementation of the data masking to another method, called `maskIpAddress`.

Listing 5.10 Updated method for masking IP addresses in the record key and value.

```

private static KeyValue<String, String> maskIpAddressInRecord(
    String recordKey, String recordValue) {

    try {
        JsonNode valueObject = objectMapper.readTree(recordValue);
        JsonNode payload = valueObject.get("payload");

        String ip = payload.get("ip").textValue();
        String maskedIp = maskIpAddress(ip); #1
        ((ObjectNode) payload).put("ip", maskedIp);

        return KeyValue.pair(maskedIp, valueObject.toString()); #2
    } catch (UnknownHostException | JsonProcessingException e) {
        logger.error("Could not apply maskIpAddressInRecord()", e);
        return KeyValue.pair(recordKey, recordValue);
    }
}

private static String maskIpAddress(String ip)
    throws UnknownHostException {

    InetAddress ipAddress = InetAddress.getByName(ip);
    byte[] address = ipAddress.getAddress();
    int anonymizedBytes = ipAddress instanceof Inet6Address ? 10 : 1;
    for (int i = 0; i < anonymizedBytes; ++i) {
        address[address.length - i - 1] = 0;
    }
    return InetAddress.getByAddress(address).getHostAddress();
}

```

#1 Apply the method `maskIpAddress` to the value of the key `ip`.

#2 Return the new version of the Kafka record as a key-value pair.

While `map` is more powerful than `mapValues`, it causes a re-partitioning of the data, because the record keys are affected by the operator. For this reason, `mapValues` is preferable unless you need to process the record keys.

NOTE The DSL of Kafka Streams does not provide mechanisms for processing the record headers. If you need to modify them, you would have to make use of the low-level processor API.

5.2.2 flatMap() and flatMapValues()

In the last section, we used the `map` operator to mask IP addresses in record keys and values. `map` takes a single record, processes it with a Java function, and returns the processed version of the record. What if we require more flexibility with regard to the output of the operator though?

Some of the sites on Excellent Toys' online shop are accessible via different URI paths. For instance, the paths `/products` and `/catalog` lead to the same page. When a visitor visits one of both paths, Excellent Toys wants to produce a visit event for the other path as well. Technically, they need to turn one Kafka record into two.

While the `map` operator is restricted to mapping records one-to-one, the `flatMap` operator is much more flexible with regard to its output. It takes a single record, processes it with a Java function, and returns zero, one, or multiple records. Similar to `mapValues`, you can use `flatMapValues` if you do not need to operate on the keys. As visualized in Figure 5.3, Excellent Toys could apply the `flatMapValues` operator to take events, look up all aliases of the visited path, and produce one event for each alias.

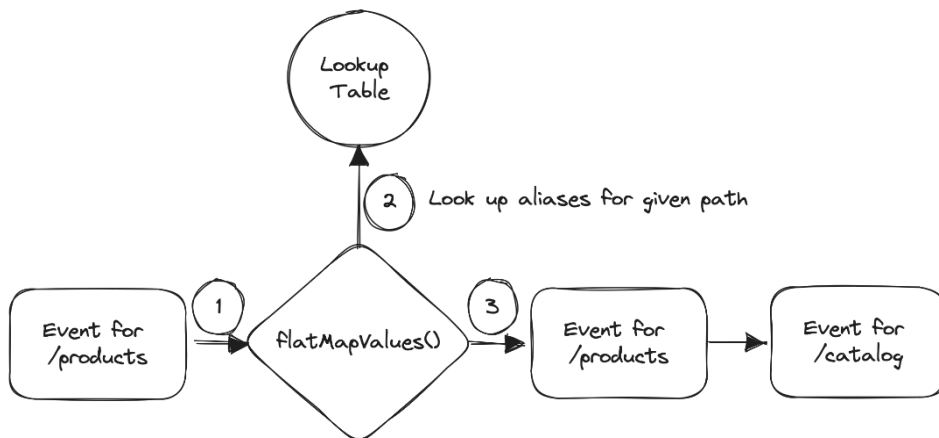


Figure 5.3 In this example, the processing node `flatMapValues` (1) takes an event that occurred on the website path `/products`, (2) looks up all aliases for the given path, and (3) returns one event for each alias.

Let's adapt the topology of our streaming application and introduce a second processor node. With Kafka Streams, you can build chains (or graphs) of processor nodes that are applied to the data in sequential order while streaming them from source to sink nodes. We introduce the `flatMapValues` operator, which takes record values and must return a Java `List`. Kafka Streams produces one record for each element of the `List` object.

Listing 5.11 Using the flatMapValues() operator to produce one event for each alias of the path.

```

builder
    .stream(
        "online_shop.analytics.visitors",
        Consumed.with(Serdes.String(), Serdes.String())
    )
    .map(StreamsApp::maskIpAddressInRecord)
    .flatMapValues(StreamsApp::handlePathAliases)
    .to(
        "online_shop.analytics.visitors_processed",
        Produced.with(Serdes.String(), Serdes.String())
    );

```

We store the path aliases in a static class variable and define the implementation of the mapping logic in the method `handlePathAliases`:

Listing 5.12 Using the flatMapValues() operator to produce one event for each alias of the path.

```

private static final List<List<String>> PATH_ALIASES_LIST = Arrays.asList( #1
    Arrays.asList("/products", "/catalog"),
    Arrays.asList("/", "/home")
);

private static List<String> handlePathAliases(String recordValue) {
    List<String> records = new ArrayList<>();

    try {
        JsonNode valueObject = objectMapper.readTree(recordValue);
        JsonNode payload = valueObject.get("payload");
        String path = payload.get("page").textValue();

        for (List<String> aliases : PATH_ALIASES_LIST) { #2
            if (aliases.contains(path)) { #3
                for (String alias : aliases) {
                    ((ObjectNode) payload).put("page", alias);
                    records.add(valueObject.toString());
                }
            }
        }
        if (records.isEmpty()) { #4
            records.add(recordValue);
        }
    } catch (JsonProcessingException e) {
        logger.error("Could not apply handlePathAliases()", e);
    }
}

```

```

        records.add(recordValue); #5
    }

    return records;
}

```

- #1 Store path aliases as a list of lists.
- #2 Iterate over lists of aliases.
- #3 If the list includes the path, generate one record for each alias.
- #4 If no alias has been found, keep the original record.
- #5 In case of errors, return the original record value.

5.3 Filtering data

The marketing department of Excellent Toys would appreciate it if we filtered out visitor data that have been produced by non-human website visitors, i.e., bots, before replicating the data to the sink topic. In practice, the detection of bots and other non-human website visitors would definitely be a huge challenge. For now, let's assume that bots use the device name `bot`. How could we filter out these events and prevent them from being sent to the sink topic?

We could use the `flatMap` operator, which we introduced in the prior section, to filter out visitor data from bots by returning an empty `List` object if the field `device` equals `bot`. Using the `flatMap` operator for filtering out records might harm the readability of the code though. Fortunately, Kafka Streams provides an even more expressive operator for filtering data, `filter`.

The operator `filter` takes one record, evaluates a Java function returning a boolean value against it, and forwards the record to downstream processors or sink nodes if the Java function evaluates to `true`. Figure 5.4 visualizes the behavior of a `filter` operator that evaluates to `true` if `device` is set to `Bot`.

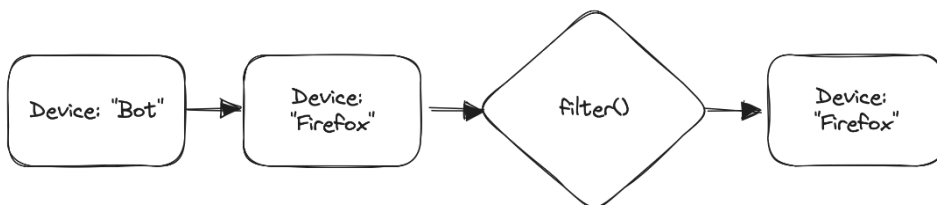


Figure 5.4 Filtering out events generated by bots.

Let's see how this could look in action. We introduce a third processor node to the topology of our Kafka Streams application, which filters out all events from bots. We let it be the first processing node of the topology, such that the remaining processing nodes need to process only relevant records. Due to its small size, we provide the `filter` function inline:

Listing 5.13 Filtering out events that have been produced by bots.

```

builder
  .stream(
    "online_shop.analytics.visitors",
    Consumed.with(Serdes.String(), Serdes.String())
  )
  .filter((key, value) -> {
    try {
      JsonNode valueObject = objectMapper.readTree(value);
      JsonNode payload = valueObject.get("payload");
      String device = payload.get("device").textValue(); #1
      return !"bot".equals(device); #2
    } catch (JsonProcessingException e) {
      logger.error("Could not apply filter()", e);
      return true;
    }
  })
  .map(StreamsApp::maskIpAddressInRecord)
  .flatMapValues(StreamsApp::handlePathAliases)
  .to(
    "online_shop.analytics.visitors_processed",
    Produced.with(Serdes.String(), Serdes.String())
  );

```

#1 Get the field `device` from the record value.

#2 Return true if the field `device` is null or does not equal `bot`.

Using negated predicates in filters, as in our example, might harm the readability of the code. To this end, Kafka Streams provides the `filterNot`, which can be used to filter for all records that do not satisfy a given predicate. We could simplify our code by replacing the usage of `filter` with `filterNot` and updating the predicate, as shown in Listing [5.14](#):

Listing 5.14 Using the `filterNot` operator to filter out records that have the field `device` set to `bot`.

```
.filterNot((key, value) -> {
    try {
        JsonNode valueObject = objectMapper.readTree(value);
        JsonNode payload = valueObject.get("payload");
        String device = payload.get("device").textValue();
        return "bot".equals(device);
    } catch (JsonProcessingException e) {
        logger.error("Could not apply filterNot()", e);
        return false;
    }
})
```

5.4 Routing data to different data sinks

Excellent Toys offers their online shop in three countries: `excellenttoys.com` (United States), `excellenttoys.co.uk` (United Kingdom), and `excellenttoys.de` (Germany). The analytics tool stores the domain of the online shop in the field `domain`. Instead of aggregating events from all countries in one Kafka topic, the marketing department prefers to keep them in different topics: events from the domain `excellenttoys.com` in the Kafka topic `online_shop.analytics.visitors_us`, events from `excellenttoys.co.uk` in `online_shop.analytics.visitors_uk`, and events from `excellenttoys.de` in `online_shop.analytics.visitors_de`.

Figure 5.5 visualizes Kafka Streams' `split` operator, which can be used to split the records into different branches and route them to different processor or sink nodes. Each branch must define a predicate that is evaluated against the record key and value. The record is assigned to the first branch for which the predicate evaluates to true. It is possible to define a default branch, similar to the `else` clause in an `if` condition.

Let's take a look at Listing 5.15, which presents an update of our Kafka Streams application that uses the `split` operator. Instead of replicating all records to a single sink node (or Kafka topic), the application splits the record stream into three branches named `country-us`, `country-uk`, and `country-de`. The branches are consumed by different sink nodes. The splitting of the records depends on the field `domain`. By default, records are assigned to the branch, which is passed to the Kafka topic `online_shop.analytics.visitors_us`.

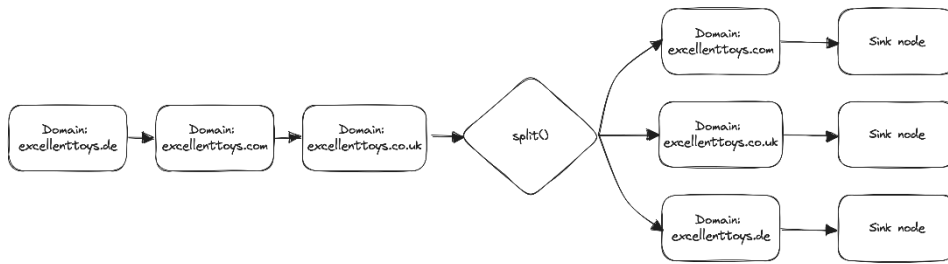


Figure 5.5 Route records to different sink nodes (or Kafka topics) depending on the key and/or value of the record.

Listing 5.15 Split the stream of records into three different branches.

```

public static void main(final String[] args) {
    ...

    final StreamsBuilder builder = new StreamsBuilder();
    Map<String, KStream<String, String>> branches = builder
        .stream(
            "online_shop.analytics.visitors",
            Consumed.with(Serdes.String(), Serdes.String())
        )
        .filterNot((key, value) -> {
            try {
                JsonNode valueObject = objectMapper.readTree(value);
                JsonNode payload = valueObject.get("payload");
                return "bot".equals(payload.get("device").textValue());
            } catch (JsonProcessingException e) {
                logger.error("Could not apply filterNot()", e);
                return false;
            }
        })
        .map(StreamsApp::maskIpAddressInRecord)
        .flatMapValues(StreamsApp::handlePathAliases)
        .split(Named.as("country-")) #1
        .branch( #2
            (key, value) ->
                StreamsApp.getDomain(value).equals("excellenttoys.co.uk"),
                Branched.as("uk")
        )
        .branch( #3
            (key, value) ->
                StreamsApp.getDomain(value).equals("excellenttoys.de"),
                Branched.as("de")
        )
    }
}

```

```

    )
    .defaultBranch( #4
        Branched.as("us")
    );
branches #5
    .get("country-us")
    .to("online_shop.analytics.visitors_us",
        Produced.with(Serdes.String(), Serdes.String()));
branches
    .get("country-uk")
    .to("online_shop.analytics.visitors_uk",
        Produced.with(Serdes.String(), Serdes.String()));
branches
    .get("country-de")
    .to("online_shop.analytics.visitors_de",
        Produced.with(Serdes.String(), Serdes.String()));

final KafkaStreams streams = new KafkaStreams(builder.build(), props);
try {
    streams.cleanUp();
    streams.start();
} catch (Throwable e) {
    logger.error("Error while running Kafka Streams app", e);
    System.exit(1);
}
}

private static final String DEFAULT_DOMAIN = "excellenttoys.com";

private static String getDomain(String recordValue) {
    try {
        JsonNode valueObject = objectMapper.readTree(recordValue);
        JsonNode payload = valueObject.get("payload");
        return payload.get("domain").textValue();
    } catch (JsonProcessingException e) {
        logger.error("Could not apply getDomain()", e);
        return DEFAULT_DOMAIN;
    }
}
}

```

#1 The operator `split`, which adds the prefix `country-` to the name of all branches

#2 Branch for events from the domain `excellenttoys.co.uk`.

#3 Branch for events from the domain `excellenttoys.de`.

#4 Default branch, implicitly used for events from the domain `excellenttoys.com`.

#5 Records from the branch country-us are produced to the topic
online_shop.analytics.visitors_us.

In our example, we passed the records of each branch directly to sink nodes (or Kafka topics). If you need to apply further processing logic to the records it would be perfectly fine to chain the `branch`, e.g., `branches.get("country-uk")`, with further processing nodes.

That's it. You have met and successfully applied the most important stateless transformation operators of Kafka Streams. By combining `map`, `flatMap`, `filter`, and other operators, you can build powerful, flexible streaming applications with a few lines of Java code. We will build on top of these operators and dive into stateful stream processing in the next chapter!

If you are interested in visualizing the topology of the Kafka Streams application, replace the line `final KafkaStreams streams = new KafkaStreams(builder.build(), props);` with the content from Listing 5.16.

Listing 5.16 Topology of the Kafka Streams application.

```
final Topology topology = builder.build();
final KafkaStreams streams = new KafkaStreams(topology, props);

System.out.println(topology.describe());
```

Now, your application will print out its topology when starting up:

Listing 5.17 Topology of the Kafka Streams application.

Topologies:

```

Sub-topology: 0
  Source: KSTREAM-SOURCE-000000000 (topics: [online_shop.analytics.visitors])
    --> KSTREAM-FILTER-000000001
  Processor: KSTREAM-FILTER-000000001 (stores: [])
    --> KSTREAM-MAPVALUES-000000002
    <-- KSTREAM-SOURCE-000000000
  Processor: KSTREAM-MAPVALUES-000000002 (stores: [])
    --> KSTREAM-FLATMAPVALUES-000000003
    <-- KSTREAM-FILTER-000000001
  Processor: KSTREAM-FLATMAPVALUES-000000003 (stores: [])
    --> country-
    <-- KSTREAM-MAPVALUES-000000002
  Processor: country- (stores: [])
    --> country-de, country-uk, country-us
    <-- KSTREAM-FLATMAPVALUES-000000003
  Processor: country-de (stores: [])
    --> KSTREAM-SINK-000000010
    <-- country-
  Processor: country-uk (stores: [])
    --> KSTREAM-SINK-000000009
    <-- country-
  Processor: country-us (stores: [])
    --> KSTREAM-SINK-000000008
    <-- country-
  Sink: KSTREAM-SINK-000000008 (topic: online_shop.analytics.visitors_us)
    <-- country-us
  Sink: KSTREAM-SINK-000000009 (topic: online_shop.analytics.visitors_uk)
    <-- country-uk
  Sink: KSTREAM-SINK-000000010 (topic: online_shop.analytics.visitors_de)
    <-- country-de

```

See the official documentation of Kafka Streams for a complete list of all available operators: <https://kafka.apache.org/36/javadoc/org/apache/kafka/streams/kstream/KStream.html>

5.5 Summary

- Kafka Streams is a Java library that lets you build stream processing applications, which replicate data between Kafka topics and transform them on the way.

- Kafka Streams applications are modeled as directed graphs of source, processor, and sink nodes.
- Kafka Streams applications can work with topics of the same Kafka cluster. They do not support streaming data between different Kafka clusters.
- The `map` operator lets you transform single records by applying a Java function to them.
- The operators `map` and `flatMap` process entire records, while `mapValues` and `flatMapValues` operate only on the record values, which is preferable, if possible, because they do not cause a re-partitioning of the records.
- The operator `filter` (or `filterNot`) can be used to filter for records for which a given predicate evaluates to `true` (or `false`).
- Using the `split` operator, you can define predicates to split streams into different branches and, for instance, route records to different Kafka topics.

6

Stateful transformations with Kafka Streams

This chapter covers

- The state management of Kafka Streams
- Aggregating records
- Deduplicating data
- Joining records of different topics
- A comparison of Kafka Streams and Apache Flink

In the last chapter, we applied stateless stream processing to transform, filter, and route single events from the analytics API at a time. If you need to process multiple Kafka messages from one or multiple topics at a time, for instance, when joining the analytics data with records from your user database, you need to maintain state across the processing of multiple events. This chapter covers the latter case and introduces you to stateful stream processing.

TIP This chapter does not contain the complete code of the Kafka Streams applications but shows only the topology. You can add the examples to the application that you built in Chapter 5 or start from scratch using the project template from <https://github.com/flippingbits/streaming-data-pipelines-book-streams-template>. As discussed in the introduction of Chapter 5, please make sure that the topic `online_shop.analytics.visitors` holds data before you start implementing the examples from this chapter.

Let's assume that Excellent Toys wants to gather statistics about its website traffic, so they know at which times the website is most frequented. To this end, they decide to group the events from the analytics API by website, assign them to temporal windows of the same size, and count the number of events per window. Figure 6.1 visualizes the topology of the corresponding Kafka Streams app.

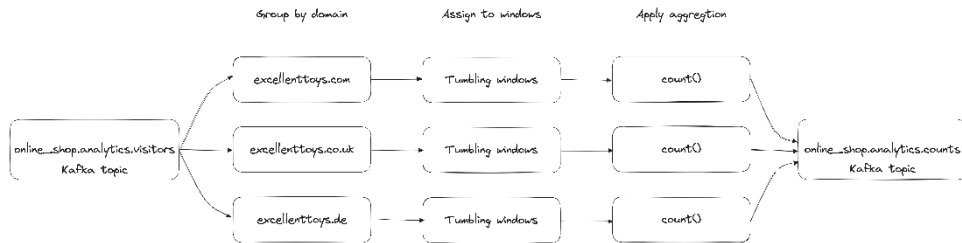


Figure 6.1 Topology of a stateful Kafka Streams app that counts the events per domain and tumbling window.

Listing 6.1 shows the corresponding Kafka Streams app, which consumes events from the Kafka topic `online_shop.analytics.visitors`, groups them into tumbling windows with a size of 5 minutes (see Section 2.3 for a detailed description of window types), applies the `count` aggregation to each window, and produces the results to the Kafka topic `online_shop.analytics.counts`.

Listing 6.1 Grouping events into non-overlapping 5-minute windows and counting the number of events per window.

```
builder
  .stream(
    "online_shop.analytics.visitors",
    Consumed.with(Serdes.String(), Serdes.String()))
  .groupBy( #1
    (key, value) -> getDomain(value),
    Grouped.with(Serdes.String(), Serdes.String()))
  .windowedBy(TimeWindows.of(Duration.ofMinutes(5))) #2
  .count(Materialized.as("count_visits")) #3
  .toStream() #4
  .map( #5
    (window, value) ->
      KeyValue.pair(
        String.format(
          "%s, %d-%d",
          window.key(), #6
          window.window().start(), #7
          window.window().end(), #8
          value.toString()), #9
        Named.as("format_windows")) #10
  ).to(
    "online_shop.analytics.counts",
    Produced.with(Serdes.String(), Serdes.String()));
```

#1 Group records by the field domain stored in the value.

#2 Create tumbling windows of 5 minutes.

#3 Count the number of records per window and group. Name the state store count_visits.

#4 Convert the result of the count into a stream.

#5 Change the format of the records of the stream.

#6 Name of the group, i.e., the domain.

#7 UNIX timestamp of the start of the window.

#8 UNIX timestamp of the end of the window.

#9 The count.

#10 Name of the stateless map() operator.

Listing [6.2](#) shows the results of the stateful processor. For each window and domain, the processor publishes the number of events to the Kafka topic `online_shop.analytics.counts`.

Listing 6.2 Results of the stateful processor from Listing 6.1.

```
$ docker exec kafka \
  /bin/kafka-console-consumer \
  --bootstrap-server=kafka:29092 \
  --topic=online_shop.analytics.counts \
  --from-beginning \
  --property print.key=true

excellenttoys.com, 1702193400000-1702193520000 31
excellenttoys.co.uk, 1702193400000-1702193520000 29
excellenttoys.de, 1702193400000-1702193520000 40
...
```

TIP You might see multiple records for the same key. If this happens, Kafka Streams refined (or updated) the result of the aggregation. While Kafka takes care of cleaning up older versions of the same key when compacting the topic, such behavior produces inconsistent results temporarily. We will discuss the matter in more detail when comparing Kafka Streams to Apache Flink in [Section 6.6](#).

[Section 6.1](#) kicks off our journey through stateful stream processing by explaining the state management of Kafka Streams. [Section 6.2](#) presents aggregating multiple events belonging to the same window, using an operator such as `count`. In [Section 6.3](#), we discuss out-of-order streams and how late events impact the results of stateful stream operators. [Section 6.4](#) shows you how to deduplicate identical events and [Section 6.5](#) discusses joining (or combining) the data of multiple Kafka topics. We finish this chapter by comparing Kafka Streams to Apache Flink, another popular stream processing tool from the Apache ecosystem, in [Section 6.6](#).

6.1 How does Kafka Streams manage state?

In the last section, we discussed a Kafka Streams application that employs an aggregation operator to count the number of events per group and window. To implement this use case, Kafka Streams must internally maintain the number of events it has seen in the past, i.e., it must manage the state. When processing a record, the operator must update its state, so it reflects the current state of the processing.

Recall that Kafka Streams applications can be executed with multiple instances on different physical machines, where each instance is responsible for processing one or multiple topics and topic partitions. Furthermore, Kafka Streams applications can consist of multiple stateful operators that are chained together. As a consequence, Kafka Streams applications must maintain a separate state store for each operator and topic partition.

By default, Kafka Streams uses RocksDB as the storage engine for its state store. The main motivation for using RocksDB is its read/write performance and its ability to manage state stores larger than the available main memory by moving parts of the store to disk. While RocksDB is a great choice for most use cases, Kafka Streams allows you to replace it with any other key-value store, as long as you implement the `KeyValueStore` interface (see <https://kafka.apache.org/37/javadoc/org/apache/kafka/streams/state/KeyValueStore.html>). For instance, if you need to be able to access the state with a very low latency and if the state fits into the main memory, the in-memory state store (see <https://kafka.apache.org/21/javadoc/org/apache/kafka/streams/state/Stores.html#inMemoryKeyValueStore-java.lang.String->) might be the store engine of your choice.

When executing Kafka Streams applications in a distributed setting, solely relying on local disks is not sufficient for persisting the state in a reliable and fault-tolerant manner. When migrating instances between different physical machines, we would have to completely rebuild the state store, which would lead to duplicates in the processed data, if it's even possible (we might have already exceeded the retention time of the consumed Kafka topics). For this reason, Kafka Streams produces every state change, e.g., updating the counter value of the key `excellenttoys.co.uk` from 3 to 4, to an internal changelog topic. Kafka Streams uses *changelog topics* to recover the state when restarting an application instance. You can list the changelog topics available on your Kafka cluster using the following command (here, the changelog topic is named `stateful-app-count_visits-changelog`):

Listing 6.3 Listing the internal changelog topics of the Kafka Streams application.

```
$ docker exec kafka \
  /bin/kafka-topics \
  --bootstrap-server=kafka:29092 \
  --topic='.*.changelog' \
  --list

stateful-app-count_visits-changelog
```

Note that the name of the state store, here `count_visits`, becomes part of the changelog topic name. If you do not name your state stores, Kafka Streams uses automatically generated topic names, such as `stateful-app-KSTREAM-AGGREGATE-STATE-STORE-0000000001-changelog`, for the changelog topics.

TIP In general, it's a best practice to name the state stores and the stateful operators in Kafka Streams applications because custom names help you to identify the *internal* topics of your application and ease mapping them to the topology of your application. Note that it's recommended to use custom names from the beginning; changing the name of a state store or operator would change the name of the underlying topic. Throughout this chapter, we will name stateless operators with `Named.as(String name)` (see <https://kafka.apache.org/37/javadoc/org/apache/kafka/streams/kstream/Named.html>), state stores with `Materialized.as(String name)` (see <https://kafka.apache.org/37/javadoc/org/apache/kafka/streams/kstream/Materialized.html>), and joins with `Joined.as(String name)` and `Joined.with(..., String name)` (see <https://kafka.apache.org/37/javadoc/org/apache/kafka/streams/kstream/Joined.html>).

Figure 6.2 shows the aggregation operator of a Kafka Streams application that holds the counter values for three keys in its state store. When processing a record for the key `excellenttoys.co.uk`, the Kafka Streams application updates its local state store and produces a change event to its changelog topic. For demonstration purposes, we assume that the Kafka Streams application executes a single instance consuming a Kafka topic with only one partition.

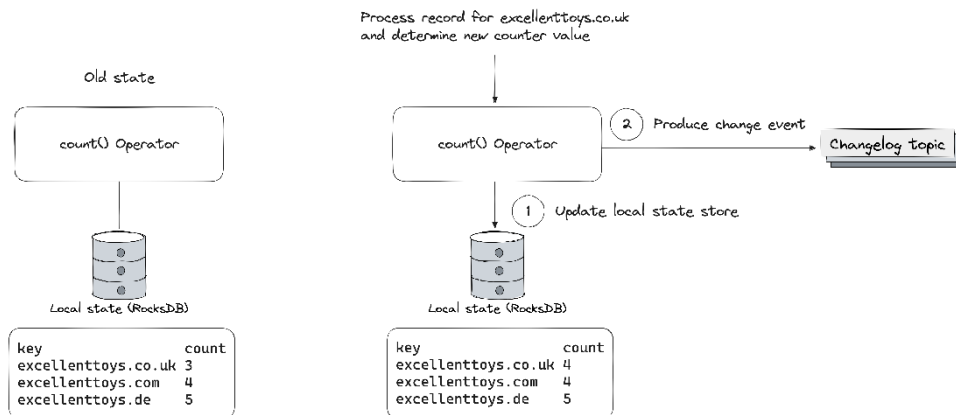


Figure 6.2 How Kafka Streams applications persist and update their local state.

Kafka Streams supports a variety of other stateful operators for use cases beyond counting records. All of them build on top of the state management capabilities of Kafka Streams. We will cover them in more detail in the next sections.

6.2 Aggregating data

In the introduction of this chapter, we have seen the operator `count` that returns the number of events of a specific group and window. The operator `count` initializes its state variable with the value 0 and increments that value for each processed record. Finally, it produces the value of the variable to a downstream operator or sink topic.

Kafka Streams provides a generalized implementation of aggregations, the `aggregate` operator. Let's take a look at an implementation of the example from Listing [6.1](#) using the `aggregate` operator. The `aggregate` function takes multiple parameters. The first parameter defines a lambda function, which initializes the aggregation value. While we use a built-in Java data type, `Long`, here, you can bring your own class for the aggregation value if you provide a valid Serdes implementation when setting up the state store. The second parameter lets you define a lambda function that is called when the `aggregate` operator processes a record. It takes three parameters, the `key` and the `value` of the processed record, as well as the current state of the aggregation value. It returns a new version of the aggregation value. In our example, we increment the aggregation value by 1 without touching the key or the value. The third parameter of the `aggregate` function lets you set up the state store. In our example, we provide a name and define the Serdes classes for the keys and the values.

Listing 6.4 Using the generalized aggregate operator to implement the use case from Listing 6.1.

```

builder
  .stream(
    "online_shop.analytics.visitors",
    Consumed.with(Serdes.String(), Serdes.String()))
  .groupByKey( #1
    (key, value) -> getDomain(value),
    Grouped.with(Serdes.String(), Serdes.String()))
  .windowedBy(TimeWindows.of(Duration.ofMinutes(5))) #2
  .aggregate(
    () -> 0L, #3
    (key, value, count) -> count + 1L, #4
    Materialized.as("counters") #5
    .with(Serdes.String(), Serdes.Long()))
  .toStream() #6
  .map( #7
    (window, value) ->
      KeyValue.pair(
        String.format(
          "%s, %d-%d",
          window.key(),
          window.window().start(),
          window.window().end()),
        value.toString()),
    Named.as("format_windows"))
  .to(
    "online_shop.analytics.counts",
    Produced.with(Serdes.String(), Serdes.String()));

```

#1 Group records by the domain of the analytics event.

#2 Create tumbling windows of 5 minutes.

#3 Initialize the counter with 0.

#4 Increase the counter by 1.

#5 Let Kafka Streams know that we store the aggregation value, the counter, as a Long value.

#6 Turn the results of the aggregate operator into a continuous stream of updates.

#7 Enrich the results with information about the key and the window.

Let us assume that, for each page of our website, we want to create a list of the IP addresses that have accessed it. The lists should be created for 1-minute tumbling windows. We could use the `aggregate` operator by initializing the aggregate value with an empty string and adding the IP address of each processed record to it. There is a simpler, more precise way though. Similar to the `count` operator that we discussed in the introduction of this chapter, Kafka Streams provides another specialization of the `aggregate` operator, the `reduce` operator. The `reduce` operator allows you to combine multiple values. Listing 6.5 first maps the records, such that the key holds the page and the value holds the IP address. Next, it uses the `reduce` operator to concatenate all values per key and 1-minute tumbling window.

Listing 6.5 Using the `reduce` operator to concatenate IP addresses.

```
builder
    .stream(
        "online_shop.analytics.visitors",
        Consumed.with(Serdes.String(), Serdes.String()))
    .flatMap(
        (key, value) -> { #1
            try {
                JsonNode valueObject = objectMapper.readTree(value);
                JsonNode payload = valueObject.get("payload");
                return List.of(
                    KeyValue.pair(
                        payload.get("page").textValue(),
                        payload.get("ip").textValue()));
            } catch (JsonProcessingException e) {
                return Collections.emptyList();
            }
        },
        Named.as("map_ip_addresses"))
    .groupByKey(Grouped.with(Serdes.String(), Serdes.String())) #2
    .windowedBy(TimeWindows.of(Duration.ofMinutes(1))) #3
    .reduce(
        (aggregationValue, ipAddress) -> #4
            String.format("%s,%s", aggregationValue, ipAddress),
        Materialized.as("combine_ip_addresses"))
    .toStream() #5
    .map( #6
        (window, value) ->
            KeyValue.pair(
                String.format(
                    "%s, %d-%d",
                    window.key(),
```

```

        window.window().start(),
        window.window().end()),
        value.toString()),
        Named.as("format_windows"))
    .to(
        "online_shop.analytics.page_ips",
        Produced.with(Serdes.String(), Serdes.String()));

```

#1 Map records, such that keys hold the page and values hold the ip.

#2 Group records by their key, i.e., the page.

#3 Create tumbling windows of 1 minute.

#4 Combine the IP addresses of the same key and window into a String value; IP addresses are delimited by a comma.

#5 Turn the results of the reduce operator into a continuous stream of updates.

#6 Enrich the results with information about the key and the window.

You can watch the results of the `reduce` operator by consuming records from the sink topic, here `online_shop.analytics.page_ips`, and printing them:

Listing 6.6 Consuming the results of the `reduce` operator.

```

$ docker exec kafka \
  /bin/kafka-console-consumer \
  --bootstrap-server=kafka:29092 \
  --topic=online_shop.analytics.page_ips \
  --from-beginning \
  --property print.key=true

/products, 1702567140000-1702567200000 222.137.152.72,216.76.49.32,...
/privacy-policy, 1702567140000-1702567200000 58.171.180.80,...
/, 1702567140000-1702567200000 131.60.189.61,29.117.193.175,...
/contact, 1702567140000-1702567200000 90.242.149.162,16.28.93.182,...
/cart, 1702567140000-1702567200000 199.25.117.81,79.73.190.251,...

```

Figure 6.3 visualizes the evolution of the state when processing records with the `reduce` operator and Kafka Streams topology from Listing 6.5. Note that the figure leaves out windows for presentation purposes.

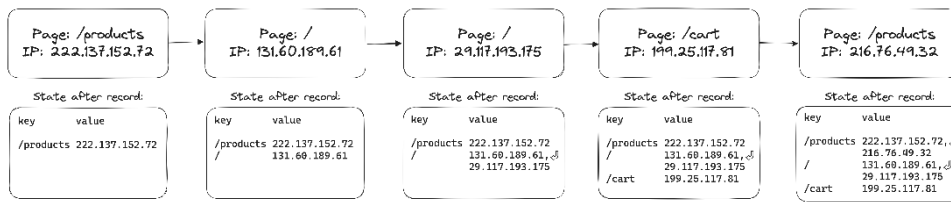


Figure 6.3 Evolution of the state when processing records with the reduce operator.

The `reduce` operator implicitly initializes the aggregation value with the first value that is processed. For instance, the key `/products` of the state gets initialized with the value `222.137.152.72` when processing the first record. Similarly, it implicitly fixes the data type of the aggregation value to the type of the processed value. In this example, the aggregation value is stored as a `String`. Compared to the `aggregate` operator, using the `reduce` operator leads to more precise code at the cost of less flexibility with regard to the aggregation value.

Did you notice that we called `toStream()` on the return value of the `aggregate`, `count`, and `reduce` operator? These operators return a `KTable`, which resembles the latest view of the results of the operator (see Section 2.3 for the differences between `KStream` and `KTable`). Since we want to stream the results of the operator to a downstream Kafka topic, we need to convert the `KTable` to a `KStream`.

6.3 Suppressing partial results and handling late events

When running the examples from the last sections, you might have noticed that stateful operators sometimes emit multiple versions for the same key and window. The last version, i.e., the record with the highest offset, is the correct one holding the final result of the operator. As opposed to other stream processing frameworks that use watermarks to mark the boundaries of a window (we will have a closer look at how Kafka Streams compares to such frameworks in [Section 6.6](#)), Kafka Streams implements the concept of *continuous refinements* and emits (partial) results whenever it processes records.

Many applications benefit from the fast and early results of continuous refinements. Consider, for instance, a real-time analytics dashboard that should be updated as soon as (partial) results are available. Other applications might prefer to wait for the final version of the result of a stateful operator. Assume, for instance, that a downstream application sends notifications to users based on the results of a `count` operator. You neither want to send notifications for incorrect, partial results nor want to send multiple notifications for the same matter.

Kafka Streams provides the `suppress` operator as a means to wait until the end of a window before passing the results to downstream operators or sink nodes:

Listing 6.7 Suppressing partial results with the suppress operator.

```
.groupByKey(Grouped.with(Serdes.String(), Serdes.String()))
.windowedBy(TimeWindows.of(Duration.ofMinutes(1)))
.reduce(
    (aggregationValue, ipAddress) ->
        String.format("%s,%s", aggregationValue, ipAddress),
        Materialized.as("combine_ip_addresses"))
.suppress(Suppressed.untilWindowCloses(Suppressed.BufferConfig.unbounded()))
```

There might be times when events arrive after the window that they belong to closes. This should be no surprise because Kafka does not guarantee any temporal ordering of records. Also, producers might lag behind and produce *old* events. By default, when using the `suppress` operator with the configuration shown in Listing [6.7](#), it rejects any events that are processed after their window has closed. Kafka Streams allows us to define a grace period up to which late events are accepted and added to the window. They produce an update of the result of the stateful operator. You can define the grace period when setting up the window:

Listing 6.8 Handling late events up to a defined grace period.

```
.groupByKey(Grouped.with(Serdes.String(), Serdes.String()))
.windowedBy(
    TimeWindows.of(Duration.ofMinutes(1)).grace(Duration.ofSeconds(10)))
.reduce(
    (aggregationValue, ipAddress) ->
        String.format("%s,%s", aggregationValue, ipAddress),
        Materialized.as("combine_ip_addresses"))
.suppress(Suppressed.untilWindowCloses(Suppressed.BufferConfig.unbounded()))
```

6.4 Deduplicating data

You might face situations where different records hold data for the same event. While they are technically not the same record in Kafka, they can still be considered duplicates. Duplicates might, for instance, occur when source systems hold duplicate data, when source connectors extract the same source record multiple times, or when producers publish the same record multiple times to a Kafka cluster (for instance, due to connection problems). Depending on your use case, you might want to filter out duplicates before processing the data. For instance, when counting website visitors, you probably do not want to count the same event multiple times.

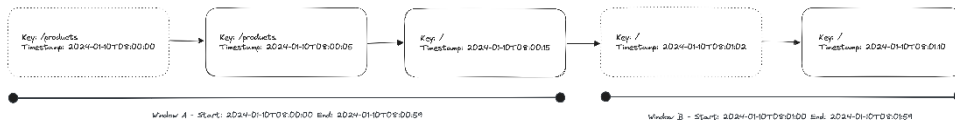


Figure 6.4 Window-based deduplication of records with the same key. Records that are ignored for further processing are visualized with a dotted stroke.

Figure 6.4 illustrates a Kafka Streams application that avoids passing duplicate data to downstream operators or sink nodes by sorting out records that have the same key and belong to the same window (they are visualized with a dotted stroke in the figure). For each window and key, only one record is kept. Listing 6.9 shows the corresponding implementation. First, we group records by their key and assign them to tumbling windows with a size of 1 minute. Next, we use the `reduce` operator to aggregate all records having the same record key. We keep only the value of the record that has been processed last. Finally, we apply the `suppress` operator, so we do not publish incomplete results before the window closes. We wait up to a grace period of ten seconds for late events. The application produces the deduplicated records to the topic `online_shop.analytics.dedup`.

Listing 6.9 Filtering out duplicate records by combining the reduce aggregation with the suppress operator.

```
builder
  .stream(
    "online_shop.analytics.visitors",
    Consumed.with(Serdes.String(), Serdes.String()))
  .groupByKey(Grouped.with(Serdes.String(), Serdes.String()))
  .windowedBy(
    TimeWindows.of(Duration.ofMinutes(1)).grace(Duration.ofSeconds(10)))
  .reduce(
    (valueState, value) -> value,
    Materialized.as("unique_ip_addresses"))
  .suppress(Suppressed.untilWindowCloses(
    ➔ Suppressed.BufferConfig.unbounded()))
  .toStream()
  .map(
    (window, value) ->
    KeyValue.pair(
      String.format(
        "%s, %d-%d",
        window.key(),
        window.window().start(),
        window.window().end()),
        value.toString()),
    Named.as("format_windows"))
  .to(
    "online_shop.analytics.dedup",
    Produced.with(Serdes.String(), Serdes.String()));
```

Figure 6.5 visualizes how the Kafka Streams application from Listing 6.9 deduplicates multiple records with the same key. For each record key, the application keeps the value of the most recent record. We ignore windows for the purpose of a cleaner visualization.

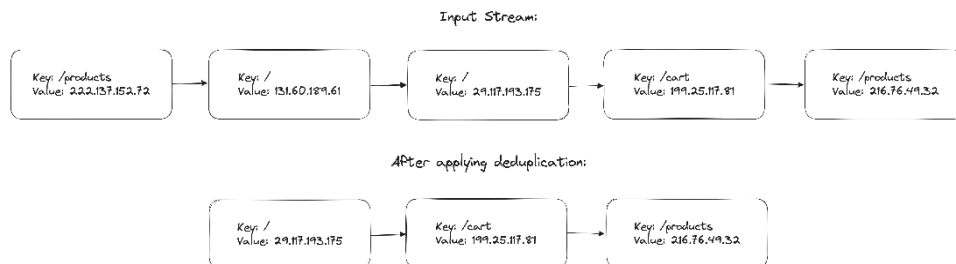


Figure 6.5 Results of the deduplication implemented by Listing 6.9.

In this section, we performed the duplicate detection based on the record keys. If you need to perform duplicate detection based on data in the record values, consider replacing `groupByKey` with `groupBy` and use any data from the record value for grouping the records.

6.5 Joining data

Excellent Toys keeps their customer data in the PostgreSQL table `customers`:

Listing 6.10 Schema of the PostgreSQL table `customers`.

```
CREATE TABLE customers (
  id          SERIAL NOT NULL PRIMARY KEY,
  name        TEXT   NOT NULL,
  email       TEXT   NOT NULL,
  last_sign_in_ip TEXT NOT NULL
);
```

The management of Excellent Toys decided that they want to enrich the analytics data, stored in the topic `online_shop.analytics.visitors`, with customer data if possible, such that they can determine how much traffic on which page of their website is coming from customers (and guests). The customer data should be present in the topic `book.online_shop.customers` of your local Kafka cluster, because we created a source connector in Chapter 3 that replicates all PostgreSQL tables of the database `postgres` to Kafka:

Listing 6.11 Viewing the records from the Kafka topic `book.online_shop.customers`.

```
$ docker exec kafka \
  /bin/kafka-console-consumer \
  --bootstrap-server=kafka:9092 \
  --topic=book.online_shop.customers \
  --from-beginning \
  --property print.key=true
```

TIP In case of an empty topic, you might need to perform a new snapshot, as shown in Listing 3.15, to re-fill the topic with data.

Taking inspiration from traditional database systems, Kafka Streams supports different types of joins for combining data from two topics, namely *inner joins*, *left outer joins*, and *full outer joins*. Similar to database systems, we make reference to the first topic as the *left side* of the join and to the second topic as the *right side* of the join. In the example of Excellent Toys, the topic `online_shop.analytics.visitors` would be the left side of the join and the topic `book.online_shop.customers` would be the right side. Joins mostly operate on the record key, so you might need to repartition your data if you need to use data from the record values in the join condition.

- The inner join produces a result for each pair of records from the left and right sides with a matching key.
- The left outer join produces a result for each pair of records from the left and right sides with a matching key. If there is no matching key on the right side, the left outer join still produces a result with the value from the left side but sets the right side to `NULL`.
- The full outer join produces a result for each pair of records from the left and right sides with a matching key. If there is no matching key on the left (or the right side), the full outer join still produces a result with the value from the left side (or right side) but sets the right side (or left side) to `NULL`.

Figure 6.6 visualizes the differences between the three join types.

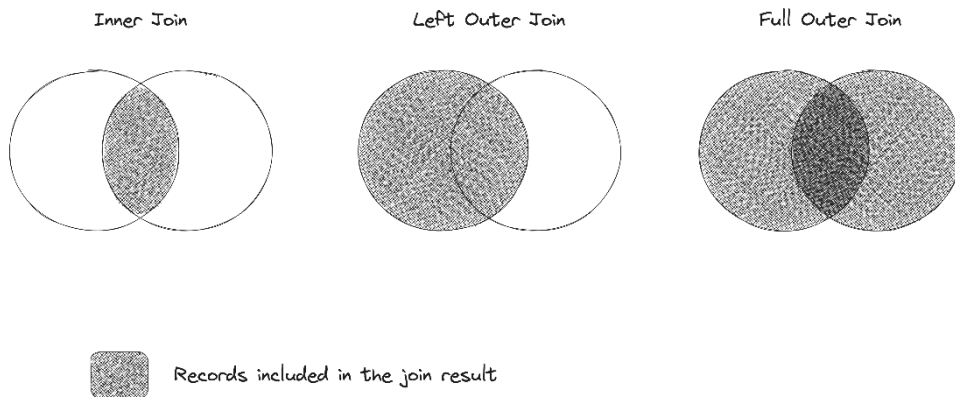


Figure 6.6 Join types supported by Kafka Streams.

Comparing the different join types, choosing the left outer join makes the most sense for the use case of Excellent Toys. They want to stream the analytics data to downstream consumers regardless of the presence of a matching customer record. If there is a customer with a matching IP address, they want to enrich the analytics event with the data from the customer table. Data from customers without matching analytics events should not be passed to downstream operators or sink nodes.

In Section 2.3, we discussed that Kafka Streams supports three different abstractions for working with data from Kafka topics: the `KStream`, the `KTable`, and the `GlobalKTable`. When joining a `KStream` with a `KStream` or a `KTable` with a `KTable`, Kafka Streams supports all three join types. When joining a `KStream` with a `KTable` or a `GlobalKTable`, Kafka Streams supports only *inner joins* or *left outer joins* but not *full outer joins*. Kafka Streams does not support joins of a `KTable` and a `GlobalKTable`. Note that the `KStream:KStream` joins are the only ones that require operating on windows, while all other combinations work without windows (`KTable` and `GlobalKTable` hold the latest version of each record key of a topic and thus ignore the concept of windows). [Table 6.1](#) summarizes the combinations of the topic abstractions supported by the different join types.

Table 6.1 Support of topic abstractions by the different join types.

Left side:Right side	Inner Join	Left Outer Join	Full Outer Join
<code>KStream:KStream</code>	Supported	Supported	Supported
<code>KTable:KTable</code>	Supported	Supported	Supported
<code>KStream:KTable</code>	Supported	Supported	Not supported
<code>KStream:GlobalKTable</code>	Supported	Supported	Not supported
<code>KTable:GlobalKTable</code>	Not supported	Not supported	Not supported

As shown above, the left outer join supports the following combinations between the different topic abstractions: `KStream:KStream`, `KTable:KTable`, `KStream:KTable`, `KStream:GlobalKTable`. Since every new record from the topic `online_shop.analytics.visitors` should invoke the join operator, we should certainly choose a `KStream` for the *left* side of the join. But what about the right side, which holds data from the topic `book.online_shop.customers`? If we use a `KStream` for the right side, we would need to work with windows and could only join customer data that have been created or updated in the same window as the analytics event. Since we do not expect customer data to change very frequently, we should consider using a `KTable` or `GlobalKTable` for the right side.

When joining a `KStream` with a `KTable`, we would need to co-partition the data of both topics:

- Both topics would need to have the same number of partitions.
- Producers would need to produce records to both topics using the same partitioning strategy, so records with the same key end up in the same partition.

Co-partitioning the topics is needed to allow the processing of different partitions by different application instances. When using a `GlobalKTable` for the right side, we can ignore the partitioning of the topics, as each application instance gets a full copy of the topic backing the `GlobalKTable`, which includes all topic partitions. When using a `GlobalKTable` with very large topics, you would build up a lot of state and should consider using a `KTable` (at the cost of having to take care of co-partitioning the topics).

Listing [6.12](#) shows a left outer join of the topic `online_shop.analytics.visitors` (represented by `KStream`) and the topic `book.online_shop.customers` (represented by a `GlobalKTable`). Results are produced to the topic `online_shop.analytics.visitors_customers`.

Listing 6.12 Joining the topics `online_shop.analytics.visitors` and `book.online_shop.customers` with a left outer join.

```
KStream<String, String> visitors = builder.stream( #1
    "online_shop.analytics.visitors",
    Consumed.with(Serdes.String(), Serdes.String()));
GlobalKTable<String, String> customers = builder.globalTable( #2
    "book.online_shop.customers",
    Consumed.with(Serdes.String(), Serdes.String()));

visitors
    .leftJoin(
        customers, #3
        (visitorKey, visitorValue) -> visitorKey, #4
        (visitorValue, customerValue) -> { #5
            try {
                JsonNode visitor = objectMapper.readTree(visitorValue);
                ObjectNode joinResult = objectMapper.createObjectNode();
                joinResult.put("visitor", visitor);
                if (customerValue != null) {
                    JsonNode customer = objectMapper.readTree(customerValue);
                    joinResult.put("customer", customer);
                }
                return joinResult.textValue();
            } catch (Exception e) {
                logger.error(e.getMessage());
                return "";
            }
        },
        Named.as("join_visitors_customers"))
    .to(
        "online_shop.analytics.visitors_customers",
        Produced.with(Serdes.String(), Serdes.String()));
```

#1 Representing the topic `online_shop.analytics.visitors` as a `KStreams`.

#2 Representing the topic `book.online_shop.customers` as a `GlobalKTable`.

#3 Right side of the left outer join.

#4 Preparing the left side of the join condition (the join uses the key of the right side without any preprocessing).

#5 Combining the value of the left side (`online_shop.analytics.visitors`) and the right side (`book.online_shop.customers`).

6.6 Kafka Streams vs. Apache Flink

In Chapter 2, we briefly discussed Apache Flink, another open-source stream processing tool. We highlighted the main differences compared to Kafka Streams:

- Flink supports many different data sources and sinks, like event brokers, message queues, or database systems, while Kafka Streams works with Apache Kafka exclusively. If you want to integrate a Kafka Streams application with other data systems, you need to use additional tooling, like Kafka Connect.
- Flink ships its own orchestrator and is designed to run on a distributed cluster of different physical machines. When using Flink, you need to deploy both the cluster management service (called `JobManager`) and the actual stream processing application (Flink offers a special deployment mode though, which combines both components into a single service). Flink can scale its processing independently from the characteristics of the data source. In contrast, Kafka Streams applications are regular Java applications. Although they can be scaled horizontally by adding more instances of the same application, their degree of parallelism is limited by the number of partitions of the consumed source topic.
- In addition to stream processing, Flink supports batch processing, too. Given that Flink supports other data sources and sinks besides Apache Kafka, this makes a lot of sense.

In this section, I want to dive deeper into the differences between Kafka Streams and Apache Flink when it comes to stateful stream processing.

6.6.1 Watermarks

We have seen that, by default, Kafka Streams employs the concept of continuous refinements (or updates). When a stateful operator, like `aggregate` or `reduce`, processes a record, it publishes a new version of the result to the downstream operators and sink nodes. While this minimizes the latency of the data processing, it leads to incorrect results, at least temporarily.

Kafka Streams introduced the `suppress` operator as a means to hold back partial results until the window closes. The `suppress` operator supports grace periods for dealing with late events. While the `suppress` operator improves the correctness (or consistency) of a lot of use cases, it still has drawbacks for others, especially applications that feature multiple stateful operators:

1. The `suppress` operator affects only a single operator. If the topology of your Kafka Streams application contains multiple stateful operators, you need to chain every stateful operator with a `suppress` operator. This does not only increase the complexity of your topology significantly but also bears the risk of forgetting to call `suppress`, potentially leading to your application producing incorrect results.
2. If your application uses the `suppress` operator multiple times, it is very likely that the different instances of the operator are not in sync, which increases the chances of incorrect results.

Apache Flink uses a different concept for ensuring the consistency of the results of stateful operators, called *watermarks*. Recall that there are different notions of time in stream processing (see Section 2.3): event time, processing time, and ingestion time. When working with event time, the stream processing framework must have knowledge about the progression of time to decide if a window has started and/or finished. Especially the knowledge about the end of a window is essential if the framework should not produce results before it has been reached.

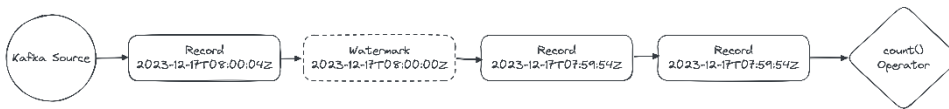


Figure 6.7 Flink source connectors produce watermarks to signalize that all records up to a specific timestamp have been seen.

In Apache Flink, source connectors produce special events, watermarks, from time to time, which hold a timestamp and signalize that it can be assumed that the time has reached the timestamp of the watermark (see Figure 6.7). When, for instance, a stateful operator, like `count`, processes a watermark for the timestamp `2023-12-17T08:00:00Z`, it can safely assume that it has seen all records that belong to the time before the timestamp and thus produce all results of windows ending before that timestamp. Note that watermarks do not have any impact on the results of the operators.

Watermarks are produced by source connectors and flow through entire topologies from sources over operators to sinks. This offers a couple of benefits: (1) When defining an operator, we no longer have to take care of chaining it with the `suppress` operator and (2) we have a *global*, in-sync definition of the progression of time.

How does Flink decide when to publish a watermark? There are different strategies available for publishing watermarks that are highly dependent on the underlying data source. In the case of Apache Kafka, it is very common to publish watermarks based on the timestamps of the processed Kafka records. To allow for late arrival events and cope with out-of-order streams, Kafka data sources often publish watermarks with a bounded grace period of, for instance, 30 seconds, i.e., when the source connector processed a Kafka record with the timestamp `2023-12-17T08:00:00Z`, it can publish the watermark `2023-12-17T07:59:30Z`, thus forcing all operators of the topology of the Flink application to produce results for windows ending before the timestamp of the watermark.

Compared to Kafka Streams's concept of dealing with progressing in event time (or the absence of it), Apache Flink provides a much more powerful concept via watermarks.

6.6.2 Checkpoints

In [Section 6.1](#), we discussed that Kafka Streams produces all state changes to an internal changelog topic, so it can recover from failures by rebuilding the state from this topic. This strategy is also used when, for instance, migrating an instance of a Kafka Streams application from one physical machine to another. Over time, streaming applications might accumulate GBs to TBs of state, which takes a lot of time to rebuild from change events. Users from the community report that their applications sometimes need hours to rebuild their state before being able to continue processing data after a restart. Kafka Streams introduced standby applications, configurable via the option `num.standby.replicas`, as a means to mitigate the pains of long state rebuilds. They are essential *secondary* instances of your Kafka Streams application that keep a fully-replicate copy of the state of the *primary* instance, so they can take over its role in the case of a failure.

Instead of replicating the state of a streaming application to a secondary instance, Flink periodically snapshots the states and stores it on local disk or on an external data storage, like AWS S3. To make the process of taking and saving these snapshots as efficient as possible, Flink can perform them incrementally, so it only has to store the changes applied since the last snapshot. Snapshots are essential to the fault-tolerant processing of data: In the case of a failure, the new application instance can read the latest state from the external data store without having to manually rebuild the state. Flink coordinates the process of taking snapshots, by injecting so-called checkpoint barriers into the streams of the data sources. When an operator processes a checkpoint barrier, it performs an (incremental) snapshot of the state and persists it.

Flink's way of storing snapshots of the state on external data storage systems, like S3, is probably much cheaper than the standby replicas that Kafka Streams employs for achieving faster state recovery (do the math yourself and compare \$/GB of S3 with main memory). For very large states and applications with a very high throughput, taking snapshots in Flink might take a considerable amount of time and slow down the processing of the data though.

6.7 Summary

- Kafka Streams offers stateful operators for processing multiple events together, e.g., `count`, `aggregate`, `reduce`, and `join`.
- Most stateful operators group records into time windows for processing, e.g., `count`, or `aggregate`. Some operators can work without windows, e.g., certain types of `join`.
- Kafka Streams persists the application state in a state store, like RocksDB, and produces state changes to an internal changelog topic for fault tolerance.
- While the `aggregate` operator provides a generic and flexible means to aggregate records, Kafka Streams also offers specialized aggregation operators, like `reduce` or `count`.
- When stateful operators process records, they immediately produce refinements (or updates) of their result. Kafka Streams offers the `suppress` operator for holding back partial results until the window closes.
- Inspired by traditional database systems, Kafka Streams offers three types of joins: inner joins, left outer joins, and full outer joins.
- While `KStream:KStream` joins operate on windows, joins with any other combination of `KStream`, `KTable`, or `GlobalKTable` can work without windows.
- When joining a `KStream` or `KTable`, topics must be co-partitioned.
- Apache Flink offers advanced concepts for stateful stream processing: watermarks for handling progression in time and checkpoints for achieving fault tolerance.